

CT60A2600-TIIMI-A

Generated by Doxygen 1.16.1

1 Directory Documentation	1
1.1 src Directory Reference	1
2 Class Documentation	3
2.1 jono Struct Reference	3
2.1.1 Detailed Description	3
2.1.2 Member Data Documentation	3
2.1.2.1 pSeuraava	3
2.1.2.2 pSolmu	3
2.2 puu Struct Reference	4
2.2.1 Detailed Description	4
2.2.2 Member Data Documentation	4
2.2.2.1 aNimi	4
2.2.2.2 iArvo	4
2.2.2.3 iPituus	4
2.2.2.4 pOikea	4
2.2.2.5 pVasen	5
2.3 RBSolmu Struct Reference	5
2.3.1 Detailed Description	5
2.3.2 Member Data Documentation	5
2.3.2.1 aNimi	5
2.3.2.2 iArvo	5
2.3.2.3 iVariBitti	5
2.3.2.4 pOikea	6
2.3.2.5 pVanhempi	6
2.3.2.6 pVasen	6
2.4 tiedot Struct Reference	6
2.4.1 Detailed Description	6
2.4.2 Member Data Documentation	6
2.4.2.1 aNimi	6
2.4.2.2 iYhteensa	7
2.4.2.3 pEdellinen	7
2.4.2.4 pSeuraava	7
3 File Documentation	9
3.1 binaaripuu.c File Reference	9
3.1.1 Function Documentation	10
3.1.1.1 etsiPienin()	10
3.1.1.2 kirjoitaBinaaripuu()	10
3.1.1.3 kirjoitaTiedostoon()	11
3.1.1.4 lisaaSolmu()	11
3.1.1.5 luoPuu()	12
3.1.1.6 nimenArvo()	13

3.1.1.7 oikeaPuoli()	14
3.1.1.8 onkoLuku()	14
3.1.1.9 paivitaPuu()	15
3.1.1.10 poistaSolmu()	16
3.1.1.11 puunPituus()	17
3.1.1.12 suurempiLukuVertailu()	17
3.1.1.13 tasapainoitaPuu()	18
3.1.1.14 vapautaMuistiPuu()	18
3.1.1.15 varaaMuistiaPuulle()	19
3.1.1.16 vasenPuoli()	19
3.2 binaaripuu.c	20
3.3 binaaripuu.h File Reference	24
3.3.1 Typedef Documentation	25
3.3.1.1 PUU	25
3.3.2 Function Documentation	25
3.3.2.1 etsiPienin()	25
3.3.2.2 kirjoitaBinaaripuu()	26
3.3.2.3 kirjoitaTiedostoon()	26
3.3.2.4 lisaaSolmu()	27
3.3.2.5 luoPuu()	28
3.3.2.6 nimenArvo()	29
3.3.2.7 oikeaPuoli()	29
3.3.2.8 onkoLuku()	30
3.3.2.9 paivitaPuu()	30
3.3.2.10 poistaSolmu()	31
3.3.2.11 puunPituus()	32
3.3.2.12 suurempiLukuVertailu()	33
3.3.2.13 tasapainoitaPuu()	33
3.3.2.14 vapautaMuistiPuu()	34
3.3.2.15 varaaMuistiaPuulle()	34
3.3.2.16 vasenPuoli()	35
3.4 binaaripuu.h	35
3.5 haut.c File Reference	36
3.5.1 Function Documentation	36
3.5.1.1 binaarihaku()	36
3.5.1.2 leveyshaku()	37
3.5.1.3 syvyyshaku()	38
3.5.1.4 tarkistaLoytyykoSyvyyshaula()	39
3.5.1.5 vapautaMuistiJono()	39
3.5.1.6 varaaMuistiaJonolle()	40
3.6 haut.c	40
3.7 haut.h File Reference	42

3.7.1 Typedef Documentation	42
3.7.1.1 JONO	42
3.7.2 Function Documentation	43
3.7.2.1 binaarihaku()	43
3.7.2.2 leveyshaku()	43
3.7.2.3 syvyyshaku()	44
3.7.2.4 tarkistaLoytvykoSyvyysaulla()	45
3.7.2.5 vapautaMuistiJono()	46
3.7.2.6 varaaMuistiaJonolle()	46
3.8 haut.h	47
3.9 linkitettylista.c File Reference	47
3.9.1 Function Documentation	48
3.9.1.1 halkaise()	48
3.9.1.2 kirjoitaTiedostoAlustaLoppuun()	49
3.9.1.3 kirjoitaTiedostoLopustaAlkuun()	49
3.9.1.4 laskeListanPituus()	50
3.9.1.5 lisaaListaan()	50
3.9.1.6 lisaysLajittelu()	52
3.9.1.7 lomitusp()	52
3.9.1.8 lomituspLajittelu()	54
3.9.1.9 lue()	54
3.9.1.10 onkoLukuVaiNimi()	55
3.9.1.11 poistaListastaAlkio()	56
3.9.1.12 poistaListastaLkmPerusteella()	57
3.9.1.13 poistaListastaNimenPerusteella()	57
3.9.1.14 useammallaAlkiollaSamaLKM()	58
3.9.1.15 vapautaMuisti()	59
3.9.1.16 varaaMuistia()	59
3.10 linkitettylista.c	60
3.11 linkitettylista.h File Reference	66
3.11.1 Typedef Documentation	67
3.11.1.1 TIEDOT	67
3.11.2 Function Documentation	68
3.11.2.1 halkaise()	68
3.11.2.2 kirjoitaTiedostoAlustaLoppuun()	68
3.11.2.3 kirjoitaTiedostoLopustaAlkuun()	69
3.11.2.4 laskeListanPituus()	70
3.11.2.5 lisaaListaan()	70
3.11.2.6 lisaysLajittelu()	71
3.11.2.7 lomitusp()	72
3.11.2.8 lomituspLajittelu()	74
3.11.2.9 lue()	74

3.11.2.10 onkoLukuVaiNimi()	75
3.11.2.11 poistaListastaAlkio()	76
3.11.2.12 poistaListastaLkmPeruusteella()	76
3.11.2.13 poistaListastaNimenPerusteella()	77
3.11.2.14 useammallaAlkiollaSamaLKM()	78
3.11.2.15 vapautaMuisti()	78
3.11.2.16 varaaMuistia()	79
3.12 linkitettylista.h	80
3.13 paaohjelma.c File Reference	80
3.13.1 Function Documentation	80
3.13.1.1 main()	80
3.14 paaohjelma.c	81
3.15 punamustapuu.c File Reference	81
3.15.1 Function Documentation	82
3.15.1.1 kierraOikealle()	82
3.15.1.2 kierraVasemmalle()	83
3.15.1.3 kirjoitaRB()	83
3.15.1.4 kirjoitaRBTiedostoon()	84
3.15.1.5 korjaaLisays()	84
3.15.1.6 lisaaRBSolmu()	85
3.15.1.7 luoRBPuu()	86
3.15.1.8 vapautaMuistiRB()	87
3.15.1.9 varaaMuistiaRB()	87
3.16 punamustapuu.c	88
3.17 punamustapuu.h File Reference	90
3.17.1 Typedef Documentation	91
3.17.1.1 RBSOLMU	91
3.17.2 Function Documentation	91
3.17.2.1 kierraOikealle()	91
3.17.2.2 kierraVasemmalle()	92
3.17.2.3 kirjoitaRB()	92
3.17.2.4 kirjoitaRBTiedostoon()	93
3.17.2.5 korjaaLisays()	93
3.17.2.6 lisaaRBSolmu()	94
3.17.2.7 luoRBPuu()	95
3.17.2.8 vapautaMuistiRB()	96
3.17.2.9 varaaMuistiaRB()	96
3.18 punamustapuu.h	97
3.19 yleiset.c File Reference	97
3.19.1 Function Documentation	98
3.19.1.1 binaaripuuValikko()	98
3.19.1.2 kysyArvo()	99

3.19.1.3 kysyNimi()	99
3.19.1.4 listaValikko()	100
3.19.1.5 mainBinaaripuu()	100
3.19.1.6 mainLista()	101
3.19.1.7 valikko()	102
3.20 yleiset.c	103
3.21 yleiset.h File Reference	106
3.21.1 Macro Definition Documentation	106
3.21.1.1 LEN	106
3.21.2 Function Documentation	107
3.21.2.1 binaaripuuValikko()	107
3.21.2.2 kysyArvo()	107
3.21.2.3 kysyNimi()	108
3.21.2.4 listaValikko()	108
3.21.2.5 mainBinaaripuu()	109
3.21.2.6 mainLista()	110
3.21.2.7 valikko()	111
3.22 yleiset.h	111
Index	113

Chapter 1

Directory Documentation

1.1 src Directory Reference

Files

- file [binaaripuu.c](#)
- file [binaaripuu.h](#)
- file [haut.c](#)
- file [haut.h](#)
- file [linkitettylista.c](#)
- file [linkitettylista.h](#)
- file [paaohjelma.c](#)
- file [punamustapuu.c](#)
- file [punamustapuu.h](#)
- file [yleiset.c](#)
- file [yleiset.h](#)

Chapter 2

Class Documentation

2.1 jono Struct Reference

```
#include <haut.h>
```

Public Attributes

- [PUU](#) * [pSolmu](#)
- struct [jono](#) * [pSeuraava](#)

2.1.1 Detailed Description

Definition at line [5](#) of file [haut.h](#).

2.1.2 Member Data Documentation

2.1.2.1 pSeuraava

```
struct jono* jono::pSeuraava
```

Definition at line [7](#) of file [haut.h](#).

2.1.2.2 pSolmu

```
PUU* jono::pSolmu
```

Definition at line [6](#) of file [haut.h](#).

The documentation for this struct was generated from the following file:

- [haut.h](#)

2.2 puu Struct Reference

```
#include <binaaripuu.h>
```

Public Attributes

- char [aNimi](#) [[LEN](#)]
- int [iArvo](#)
- int [iPituus](#)
- struct [puu](#) * [pOikea](#)
- struct [puu](#) * [pVasen](#)

2.2.1 Detailed Description

Definition at line 5 of file [binaaripuu.h](#).

2.2.2 Member Data Documentation

2.2.2.1 aNimi

```
char puu::aNimi [LEN]
```

Definition at line 6 of file [binaaripuu.h](#).

2.2.2.2 iArvo

```
int puu::iArvo
```

Definition at line 7 of file [binaaripuu.h](#).

2.2.2.3 iPituus

```
int puu::iPituus
```

Definition at line 8 of file [binaaripuu.h](#).

2.2.2.4 pOikea

```
struct puu* puu::pOikea
```

Definition at line 9 of file [binaaripuu.h](#).

2.2.2.5 pVasen

```
struct puu* puu::pVasen
```

Definition at line 10 of file [binaaripuu.h](#).

The documentation for this struct was generated from the following file:

- [binaaripuu.h](#)

2.3 RBSolmu Struct Reference

```
#include <punamustapuu.h>
```

Public Attributes

- char [aNimi](#) [[LEN](#)]
- int [iArvo](#)
- int [iVariBitti](#)
- struct [RBSolmu](#) * [pOikea](#)
- struct [RBSolmu](#) * [pVasen](#)
- struct [RBSolmu](#) * [pVanhempi](#)

2.3.1 Detailed Description

Definition at line 5 of file [punamustapuu.h](#).

2.3.2 Member Data Documentation

2.3.2.1 aNimi

```
char RBSolmu::aNimi[LEN]
```

Definition at line 6 of file [punamustapuu.h](#).

2.3.2.2 iArvo

```
int RBSolmu::iArvo
```

Definition at line 7 of file [punamustapuu.h](#).

2.3.2.3 iVariBitti

```
int RBSolmu::iVariBitti
```

Definition at line 8 of file [punamustapuu.h](#).

2.3.2.4 pOikea

```
struct RBSolmu* RBSolmu::pOikea
```

Definition at line 9 of file [punamustapuu.h](#).

2.3.2.5 pVanhempi

```
struct RBSolmu* RBSolmu::pVanhempi
```

Definition at line 11 of file [punamustapuu.h](#).

2.3.2.6 pVasen

```
struct RBSolmu* RBSolmu::pVasen
```

Definition at line 10 of file [punamustapuu.h](#).

The documentation for this struct was generated from the following file:

- [punamustapuu.h](#)

2.4 tiedot Struct Reference

```
#include <linkitettylista.h>
```

Public Attributes

- char [aNimi](#) [[LEN](#)]
- int [iYhteensa](#)
- struct [tiedot](#) * [pSeuraava](#)
- struct [tiedot](#) * [pEdellinen](#)

2.4.1 Detailed Description

Definition at line 5 of file [linkitettylista.h](#).

2.4.2 Member Data Documentation

2.4.2.1 aNimi

```
char tiedot::aNimi [LEN]
```

Definition at line 6 of file [linkitettylista.h](#).

2.4.2.2 iYhteensa

```
int tiedot::iYhteensa
```

Definition at line 7 of file [linkitettylista.h](#).

2.4.2.3 pEdellinen

```
struct tiedot* tiedot::pEdellinen
```

Definition at line 9 of file [linkitettylista.h](#).

2.4.2.4 pSeuraava

```
struct tiedot* tiedot::pSeuraava
```

Definition at line 8 of file [linkitettylista.h](#).

The documentation for this struct was generated from the following file:

- [linkitettylista.h](#)

Chapter 3

File Documentation

3.1 binaaripuu.c File Reference

```
#include "binaaripuu.h"
#include <ctype.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

Functions

- **PUU * varaaMuistiaPuulle** (char *pSolmu, int iArvo)
Muistin varaaminen puu structin alkioille.
- **PUU * vapautaMuistiPuu** (PUU *pJuuriSolmu)
Muistin vapauttaminen puu structin alkioista Juurisolmun ja kaikkien sen lapsie solmujen muisti vapautetaan.
- **PUU * lisaaSolmu** (PUU *pAlku, char *pSolmu, int iArvo)
Lisaa solmun puuhun.
- **PUU * luoPuu** (char *pNimi, PUU *pJuuriSolmu)
Luetaan tiedosto ja luodaan puu rakenne.
- void **kirjoitaBinaaripuu** (char *pNimi, PUU *pAlku)
Tasapainoitettun binaaripuun kirjoittaminen tiedostoon.
- void **kirjoitaTiedostoon** (char *pNimi, PUU *pAlku)
Kirjoittaa binaaripuun tiedostoon.
- int **tasapainoitaPuu** (PUU *pAlku)
Palauttaa solmun tasapainokertoimen.
- **PUU * oikeaPuoli** (PUU *pAlku)
Oikean puolen kierto.
- **PUU * vasenPuoli** (PUU *pAlku)
Vasemman puolen kierto.
- int **puunPituus** (PUU *pAlku)
Hakee solmun korkeuden.
- int **suurempiLukuVertailu** (int iLuku1, int iLuku2)
Vertaa kahta lukua ja palauttaa suuremman.
- **PUU * poistaSolmu** (char *pNimi, PUU *pJuuriSolmu)
Lehtisolmun poistaminen.

- int `onkoLuku` (char *`pNimi`)
Testaa, onko käyttäjän antama arvo nimi vai numeroarvo.
- int `nimenArvo` (char *`pNimi`, `PUU` *`pJuuriSolmu`)
Etsii nimen perusteella sen arvon.
- `PUU` * `etsiPienin` (`PUU` *`pUusiSolmu`)
Etsii pienimmän solmun arvon binääripuusta.
- `PUU` * `paivitaPuu` (`PUU` *`pJuuriSolmu`)
Tasapainotetaan puu poiston jälkeen.

3.1.1 Function Documentation

3.1.1.1 etsiPienin()

```
PUU * etsiPienin (
    PUU * pUusiSolmu)
```

Parameters

<code>pUusiSolmu</code>	Osoitin kasiteltavaan solmuun.
-------------------------	--------------------------------

Returns

`PUU*` Palauttaa osoittimen puun pienimpaan solmuun.

Definition at line 444 of file `binaaripuu.c`.

```
00444     {
00445         PUU *pNykyinenSolmu = pUusiSolmu;
00446
00447         while (pNykyinenSolmu->pVasen != NULL) {
00448             pNykyinenSolmu = pNykyinenSolmu->pVasen;
00449         }
00450
00451         return (pNykyinenSolmu);
00452     }
```

3.1.1.2 kirjoitaBinaaripuu()

```
void kirjoitaBinaaripuu (
    char * pNimi,
    PUU * pAlku)
```

Parameters

<code>pNimi</code>	Kirjoitettavan tiedoston nimi.
<code>pAlku</code>	

Definition at line 177 of file `binaaripuu.c`.

```
00177     {
00178         if (pAlku != NULL) {
00179             kirjoitaTiedostoon(pNimi, pAlku);
00180             kirjoitaBinaaripuu(pNimi, pAlku->pVasen);
00181             kirjoitaBinaaripuu(pNimi, pAlku->pOikea);
00182         }
00183         return;
00184     }
```

3.1.1.3 kirjoitaTiedostoon()

```
void kirjoitaTiedostoon (
    char * pNimi,
    PUU * pAlku)
```

Parameters

<i>pNimi</i>	Kirjoitettavan tiedoston nimi
<i>pAlku</i>	Solmu, joka kirjoitetaan tiedostoon

Definition at line 192 of file [binaaripuu.c](#).

```
00192                                     {
00193     FILE *Tiedosto = NULL;
00194
00195     if (pAlku == NULL) {
00196         return;
00197     }
00198
00199     /* Tiedoston avaaminen. */
00200     if ((Tiedosto = fopen(pNimi, "a")) == NULL) {
00201         perror("Tiedoston avaaminen epäonnistui, lopetetaan");
00202         exit(0);
00203     }
00204     /* Tiedostoon kirjoittaminen. */
00205     fprintf(Tiedosto, "%s,%d\n", pAlku->aNimi, pAlku->iArvo);
00206
00207     /* Tiedoston sulkeminen. */
00208     fclose(Tiedosto);
00209     return;
00210 }
```

3.1.1.4 lisaaSolmu()

```
PUU * lisaaSolmu (
    PUU * pAlku,
    char * pSolmu,
    int iArvo)
```

Tasapainottaa puun.

Parameters

<i>pAlku</i>	
<i>pSolmu</i>	Lisattava solmu.
<i>iArvo</i>	Solmun arvo.

Returns

PUU*

Definition at line 58 of file [binaaripuu.c](#).

```
00058                                     {
00059     int iVertailu = 0;
00060     int tasapaino = 0;
00061
00062     // Varataan muistia, jos muisti tyhjä.
00063     if (pAlku == NULL) {
00064         return varaaMuistiaPuulle(pSolmu, iArvo);
00065     }
```

```

00066
00067     iVertailu = strcmp(pSolmu, pAlku->aNimi);
00068
00069     // Solmujen lisääminen.
00070     if (iArvo < pAlku->iArvo) {
00071         pAlku->pVasen = lisaaSolmu(pAlku->pVasen, pSolmu, iArvo);
00072     } else if (iArvo > pAlku->iArvo) {
00073         pAlku->pOikea = lisaaSolmu(pAlku->pOikea, pSolmu, iArvo);
00074
00075         // Testataan, ovatko arvot samat.
00076     } else if (iArvo == pAlku->iArvo) {
00077         if (iVertailu < 0) {
00078             pAlku->pVasen = lisaaSolmu(pAlku->pVasen, pSolmu, iArvo);
00079         } else if (iVertailu > 0) {
00080             pAlku->pOikea = lisaaSolmu(pAlku->pOikea, pSolmu, iArvo);
00081         }
00082     }
00083
00084     // Selvitetaan paikka, johon solmu lisataan. Tasapainotetaan puu.
00085     pAlku->iPituus = 1 + suurempiLukuVertailu(puunPituus(pAlku->pVasen), puunPituus(pAlku->pOikea));
00086     tasapaino = tasapainoitaPuu(pAlku);
00087
00088     // vasen vasen tasapainotus
00089     if ((tasapaino > 1) && (iArvo < pAlku->pVasen->iArvo)) {
00090         return (oikeaPuoli(pAlku));
00091     }
00092
00093     // vasen oikea tasapainotus
00094     if ((tasapaino > 1) && (iArvo > pAlku->pVasen->iArvo)) {
00095         pAlku->pVasen = vasenPuoli(pAlku->pVasen);
00096         return (oikeaPuoli(pAlku));
00097     }
00098
00099     // oikea vasen tasapainotus
00100     if ((tasapaino < -1) && (iArvo < pAlku->pOikea->iArvo)) {
00101         pAlku->pOikea = oikeaPuoli(pAlku->pOikea);
00102         return (vasenPuoli(pAlku));
00103     }
00104
00105     // oikea oikea tasapainotus
00106     if ((tasapaino < -1) && (iArvo > pAlku->pOikea->iArvo)) {
00107         return (vasenPuoli(pAlku));
00108     }
00109
00110     return (pAlku);
00111 }

```

3.1.1.5 luoPuu()

```

PUU * luoPuu (
    char * pNimi,
    PUU * pJuuriSolmu)

```

Pilkotaan luetut rivit.

Parameters

<i>pNimi</i>	Luettavan tiedoston nimi.
<i>pJuuriSolmu</i>	Puun juurisolmu.

Returns

PUU* Palauttaa juuriSolmun.

Definition at line 120 of file [binaaripuu.c](#).

```

00120                                     {
00121     FILE *Tiedosto = NULL;
00122     char aRivi[LEN] = "";
00123     int iOtsikko = 0;
00124     char *p1 = NULL, *p2 = NULL;
00125     int iArvo = 0;

```

```

00126
00127 // Tiedoston avaus
00128 if ((Tiedosto = fopen(pNimi, "r")) == NULL) {
00129     perror("Tiedoston avaaminen epäonnistui, lopetetaan");
00130     exit(0);
00131 }
00132
00133 // Tiedoston lukeminen
00134 while ((fgets(aRivi, LEN, Tiedosto)) != NULL) {
00135     aRivi[strlen(aRivi) - 1] = '\0';
00136
00137     // Ohitetaan otsikko
00138     if (iOtsikko == 0) {
00139         iOtsikko++;
00140         continue;
00141     }
00142
00143     // Pilkotaan rivit
00144     if ((p1 = strtok(aRivi, ";")) == NULL) {
00145         perror("Tiedoston pilkkominen epäonnistui, lopetetaan");
00146         exit(0);
00147     }
00148
00149     if ((p2 = strtok(NULL, ";")) == NULL) {
00150         perror("Tiedoston pilkkominen epäonnistui, lopetetaan");
00151         exit(0);
00152     }
00153
00154     iArvo = atoi(p2);
00155
00156     // Maaritetaan juuri solmu jos sitä ei ole määritetty.
00157     if (pJuuriSolmu == NULL) {
00158         pJuuriSolmu = lisaaSolmu(pJuuriSolmu, p1, iArvo);
00159         iOtsikko++;
00160         continue;
00161     }
00162
00163     pJuuriSolmu = lisaaSolmu(pJuuriSolmu, p1, iArvo);
00164 }
00165
00166 // Suljetaan tiedosto
00167 fclose(Tiedosto);
00168 return (pJuuriSolmu);
00169 }

```

3.1.1.6 nimenArvo()

```

int nimenArvo (
    char * pNimi,
    PUU * pJuuriSolmu)

```

Parameters

<i>pNimi</i>	Poistettava nimi merkkijonona.
<i>pJuuriSolmu</i>	Osoitin kasiteltavaan solmuun.

Returns

int Palauttaa 0 tai 1, riippuen siitä, löytyykö nimen nimen pohjalta arvoa.

Definition at line 419 of file [binaaripuu.c](#).

```

00419
00420 if (pJuuriSolmu == NULL) {
00421     return (0);
00422 }
00423
00424 if (strcmp(pJuuriSolmu->aNimi, pNimi) == 0) {
00425     int iArvo = pJuuriSolmu->iArvo;
00426     return (iArvo);
00427 }
00428

```

```

00429     int iArvo = nimenArvo(pNimi, pJuuriSolmu->pVasen);
00430     if (iArvo != 0) {
00431         return (iArvo);
00432     } else {
00433         int iArvo = nimenArvo(pNimi, pJuuriSolmu->pOikea);
00434         return (iArvo);
00435     }
00436 }

```

3.1.1.7 oikeaPuoli()

```

PUU * oikeaPuoli (
    PUU * pAlku)

```

Puu järjestetään, jotta se on tasapainossa.

Parameters

<i>pAlku</i>	
--------------	--

Returns

PUU* Palauttaa solmun.

Definition at line 234 of file [binaaripuu.c](#).

```

00234     {
00235         // Tarkistetaan, onko pAlku tai pAlku oikea puoli NULL
00236         if ((pAlku->pVasen == NULL) || (pAlku == NULL)) {
00237             return (pAlku);
00238         }
00239
00240         // Muuttujien ja vakioden alustaminen
00241         PUU *pMuuttujal = pAlku->pVasen;
00242         PUU *pMuuttuja2 = pMuuttujal->pOikea;
00243
00244         // Sijoitetetaan arvoja
00245         pMuuttujal->pOikea = pAlku;
00246         pAlku->pVasen = pMuuttuja2;
00247
00248         // Verrataan ja sijoitetaan suuremmat korkeudet
00249         pAlku->iPituus = suurempiLukuVertailu(puunPituus(pAlku->pVasen), puunPituus(pAlku->pOikea)) + 1;
00250         pMuuttujal->iPituus = suurempiLukuVertailu(puunPituus(pMuuttujal->pVasen), puunPituus(pMuuttujal->pOikea)) + 1;
00251
00252         return pMuuttujal;
00253     }
00254 }

```

3.1.1.8 onkoLuku()

```

int onkoLuku (
    char * pNimi)

```

Ilmoittaa, onko käyttäjän antama numero vai ei.

Parameters

<i>pNimi</i>	Poistettava nimi tai numeroarvo merkkijonona.
--------------	---

Returns

int Palauttaa 0 tai 1, riippuen siitä onko käyttäjän syöte numero.

Definition at line 398 of file [binaaripuu.c](#).

```
00398         {
00399     int tosi = 0;
00400     char *ptr = pNimi;
00401
00402     while (*ptr) {
00403         if (!isdigit(*ptr)) {
00404             return tosi;
00405         }
00406         ptr++;
00407     }
00408     tosi = 1;
00409     return (tos);
00410 }
```

3.1.1.9 paivitaPuu()

```
PUU * paivitaPuu (
    PUU * pJuuriSolmu)
```

Parameters

<i>pJuuriSolmu</i>	Osoitin kasiteltavaan solmuun.
--------------------	--------------------------------

Returns

PUU* palauttaa tasapainotetun solmun.

Definition at line 460 of file [binaaripuu.c](#).

```
00460         {
00461     // Puun korkeuden päivittäminen.
00462     if (pJuuriSolmu == NULL) {
00463         return (pJuuriSolmu);
00464     }
00465
00466     int iLuku1 = puunPituus(pJuuriSolmu->pVasen);
00467     int iLuku2 = puunPituus(pJuuriSolmu->pOikea);
00468     pJuuriSolmu->iPituus = suurempiLukuVertailu(iLuku1, iLuku2) + 1;
00469
00470     // Puun tasapanottaminen.
00471     int iTasapaino = tasapainoitaPuu(pJuuriSolmu);
00472     // Vasen-vasen tapaus.
00473     if (iTasapaino > 1 && tasapainoitaPuu(pJuuriSolmu->pVasen) >= 0) {
00474         return oikeaPuoli(pJuuriSolmu);
00475     }
00476
00477     // Oikea-oikea tapaus.
00478     if (iTasapaino < -1 && tasapainoitaPuu(pJuuriSolmu->pOikea) <= 0) {
00479         return vasenPuoli(pJuuriSolmu);
00480     }
00481
00482     // Vasen-oikea tapaus.
00483     if (iTasapaino > 1 && tasapainoitaPuu(pJuuriSolmu->pVasen) < 0) {
00484         pJuuriSolmu->pVasen = vasenPuoli(pJuuriSolmu->pVasen);
00485         return oikeaPuoli(pJuuriSolmu);
00486     }
00487
00488     // Oikea-vasen tapaus.
00489     if (iTasapaino < -1 && tasapainoitaPuu(pJuuriSolmu->pOikea) > 0) {
00490         pJuuriSolmu->pOikea = oikeaPuoli(pJuuriSolmu->pOikea);
00491         return vasenPuoli(pJuuriSolmu);
00492     }
00493
00494     return (pJuuriSolmu);
00495 }
```

3.1.1.10 poistaSolmu()

```
PUU * poistaSolmu (
    char * pNimi,
    PUU * pJuuriSolmu)
```

Poistaa lehtisolmun joko numeroarvon tai nimen perusteella.

Parameters

<i>pNimi</i>	Poistettava nimi tai numeroarvo merkkijonona.
<i>pJuuriSolmu</i>	Osoitin kasiteltavaan puun solmuun.
<i>iArvo</i>	Haettava numeroarvo.

Returns

PUU* Uusi osoitin puun solmuun, NULL jos puu tyhjeni.

Definition at line 331 of file [binaaripuu.c](#).

```
00331                                     {
00332     int iArvo = 0;
00333     PUU *pUusiSolmu = NULL;
00334
00335     if (pJuuriSolmu == NULL) {
00336         printf("Solmua ei löytynyt.\n");
00337         return (pJuuriSolmu);
00338     }
00339
00340     // Testataan, onko syöte nimi vai lukuarvo, ja määritetään lukuarvo puun läpikäymiseksi.
00341     if (onkoLuku(pNimi)) {
00342         iArvo = atoi(pNimi);
00343     } else {
00344         iArvo = nimenArvo(pNimi, pJuuriSolmu);
00345
00346         if (iArvo == 0) {
00347             printf("Solmua ei löytynyt tällä nimellä.\n");
00348             return (pJuuriSolmu);
00349         }
00350     }
00351
00352     // Lehtisolmun poistaminen.
00353     if (iArvo < pJuuriSolmu->iArvo) {
00354         pJuuriSolmu->pVasen = poistaSolmu(pNimi, pJuuriSolmu->pVasen);
00355     } else if (iArvo > pJuuriSolmu->iArvo) {
00356         pJuuriSolmu->pOikea = poistaSolmu(pNimi, pJuuriSolmu->pOikea);
00357     } else {
00358
00359         if (pJuuriSolmu->pVasen == NULL && pJuuriSolmu->pOikea == NULL) {
00360             free(pJuuriSolmu);
00361             pJuuriSolmu = NULL;
00362             printf("Solmu poistettu.\n");
00363             return (pJuuriSolmu);
00364
00365         } else if (pJuuriSolmu->pVasen == NULL || pJuuriSolmu->pOikea == NULL) {
00366             if (pJuuriSolmu->pVasen != NULL) {
00367                 pUusiSolmu = pJuuriSolmu->pVasen;
00368             } else {
00369                 pUusiSolmu = pJuuriSolmu->pOikea;
00370             }
00371             free(pJuuriSolmu);
00372             pJuuriSolmu = NULL;
00373             printf("Solmu poistettu.\n");
00374             return (pUusiSolmu);
00375
00376         } else {
00377             pUusiSolmu = etsiPienin(pJuuriSolmu->pOikea);
00378             pJuuriSolmu->iArvo = pUusiSolmu->iArvo;
00379             strcpy(pJuuriSolmu->aNimi, pUusiSolmu->aNimi);
00380             pJuuriSolmu->pOikea = poistaSolmu(pUusiSolmu->aNimi, pJuuriSolmu->pOikea);
00381             return (pJuuriSolmu);
00382         }
00383     }
00384
00385     pJuuriSolmu = paivitaPuu(pJuuriSolmu);
00386     return (pJuuriSolmu);
00387 }
```


3.1.1.11 puunPituus()

```
int puunPituus (  
    PUU * pAlku)
```

Parameters

<i>pAlku</i>	Kohta, josta korkeus haetaan
--------------	------------------------------

Returns

int Palauttaa solmun korkeuden

Definition at line 290 of file [binaaripuu.c](#).

```
00290                                     {  
00291     // Tarkistetaan, onko Solmu Null  
00292     if (pAlku == NULL) {  
00293         return (0);  
00294     }  
00295  
00296     // Palauttaa solmun korkeuden  
00297     return (pAlku->iPituus);  
00298 }
```

3.1.1.12 suurempiLukuVertailu()

```
int suurempiLukuVertailu (  
    int iLuku1,  
    int iLuku2)
```

Parameters

<i>iLuku1</i>	Ensimmäinen verrattava kokonaisluku.
<i>iLuku2</i>	Toinen verrattava kokonaisluku.

Returns

int Palauttaa suuremman luvun.

Definition at line 307 of file [binaaripuu.c](#).

```
00307                                     {  
00308     int iPalautus = 0;  
00309  
00310     // Verrataan kumpi luvuista on suurempi.  
00311     if (iLuku1 > iLuku2) {  
00312         iPalautus = iLuku1;  
00313     } else {  
00314         iPalautus = iLuku2;  
00315     }  
00316  
00317     // Palauttaa suuremman luvun.  
00318     return (iPalautus);  
00319 }
```

3.1.1.13 tasapainoitaPuu()

```
int tasapainoitaPuu (
    PUU * pAlku)
```

Parameters

<i>pAlku</i>	Solmu, josta tasapainokerroin halutaan
--------------	--

Returns

int

Definition at line 218 of file [binaaripuu.c](#).

```
00218                                     {
00219     // Tarkistetaan, onko Null
00220     if (pAlku == NULL) {
00221         return (0);
00222     }
00223
00224     // Palautetaan tasapainokerroin
00225     return (puunPituus(pAlku->pVasen) - puunPituus(pAlku->pOikea));
00226 }
```

3.1.1.14 vapautaMuistiPuu()

```
PUU * vapautaMuistiPuu (
    PUU * pJuuriSolmu)
```

Parameters

<i>pJuuriSolmu</i>	Puun juurisolmu
--------------------	-----------------

Returns

PUU*

Definition at line 39 of file [binaaripuu.c](#).

```
00039                                     {
00040     // Muistin vapauttaminen
00041     if (pJuuriSolmu != NULL) {
00042         vapautaMuistiPuu(pJuuriSolmu->pVasen);
00043         vapautaMuistiPuu(pJuuriSolmu->pOikea);
00044         free(pJuuriSolmu);
00045     }
00046     pJuuriSolmu = NULL;
00047     return (pJuuriSolmu);
00048 }
```

3.1.1.15 varaaMuistiaPuulle()

```

PUU * varaaMuistiaPuulle (
    char * pSolmu,
    int iArvo)

```

Parameters

<i>pSolmu</i>	Solmu, jolle muistia varataan
<i>iArvo</i>	Arvo, jolle muistia varataan

Returns

PUU*

Definition at line 16 of file [binaaripuu.c](#).

```

00016                                     {
00017     PUU *pUusi = NULL;
00018
00019     // Muistin varaaminen
00020     if ((pUusi = (PUU *)malloc(sizeof(PUU))) == NULL) {
00021         perror("Muistin varaus epäonnistui, lopetetaan");
00022         exit(0);
00023     }
00024
00025     strcpy(pUusi->aNimi, pSolmu);
00026     pUusi->iArvo = iArvo;
00027     pUusi->iPituus = 1;
00028     pUusi->pVasen = NULL;
00029     pUusi->pOikea = NULL;
00030     return (pUusi);
00031 }

```

3.1.1.16 vasenPuoli()

```

PUU * vasenPuoli (
    PUU * pAlku)

```

Puu järjestetään, jotta se on tasapainossa.

Parameters

<i>pAlku</i>	
--------------	--

Returns

PUU* Palauttaa solmun.

Definition at line 262 of file [binaaripuu.c](#).

```

00262                                     {
00263     // Tarkistetaan, onko pAlku tai pAlku oikea puoli NULL
00264     if ((pAlku->pOikea == NULL) || (pAlku == NULL)) {
00265         return (pAlku);
00266     }
00267
00268     // Muuttujien ja vakioiden alustaminen
00269     PUU *pMuuttuja1 = pAlku->pOikea;
00270     PUU *pMuuttuja2 = pMuuttuja1->pVasen;
00271
00272     // Sijoitetaan arvoja
00273     pMuuttuja1->pVasen = pAlku;
00274     pAlku->pOikea = pMuuttuja2;
00275
00276     // Verrataan ja sijoitetaan suuremmat korkeudet
00277     pAlku->iPituus = suurempiLukuVertailu(puunPituus(pAlku->pVasen), puunPituus(pAlku->pOikea)) + 1;
00278     pMuuttuja1->iPituus =
00279         suurempiLukuVertailu(puunPituus(pMuuttuja1->pVasen), puunPituus(pMuuttuja1->pOikea)) + 1;
00280
00281     return pMuuttuja1;
00282 }

```

3.2 binaaripuu.c

[Go to the documentation of this file.](#)

```

00001 #include "binaaripuu.h"
00002 #include <ctype.h>
00003 #include <stdio.h>
00004 #include <stdlib.h>
00005 #include <string.h>
00006
00007 // Tasapainoitettu binääripuu AVL:
00008
00016 PUU *varaaMuistiaPuulle(char *pSolmu, int iArvo) {
00017     PUU *pUusi = NULL;
00018
00019     // Muistin varaaminen
00020     if ((pUusi = (PUU *)malloc(sizeof(PUU))) == NULL) {
00021         perror("Muistin varaus epäonnistui, lopetetaan");
00022         exit(0);
00023     }
00024
00025     strcpy(pUusi->aNimi, pSolmu);
00026     pUusi->iArvo = iArvo;
00027     pUusi->iPituus = 1;
00028     pUusi->pVasen = NULL;
00029     pUusi->pOikea = NULL;
00030     return (pUusi);
00031 }
00032
00039 PUU *vapautaMuistiPuu(PUU *pJuuriSolmu) {
00040     // Muistin vapauttaminen
00041     if (pJuuriSolmu != NULL) {
00042         vapautaMuistiPuu(pJuuriSolmu->pVasen);
00043         vapautaMuistiPuu(pJuuriSolmu->pOikea);
00044         free(pJuuriSolmu);
00045     }
00046     pJuuriSolmu = NULL;
00047     return (pJuuriSolmu);
00048 }
00049
00058 PUU *lisaaSolmu(PUU *pAlku, char *pSolmu, int iArvo) {
00059     int iVertailu = 0;
00060     int tasapaino = 0;
00061
00062     // Varataan muistia, jos muisti tyhjä.
00063     if (pAlku == NULL) {
00064         return varaaMuistiaPuulle(pSolmu, iArvo);
00065     }
00066
00067     iVertailu = strcmp(pSolmu, pAlku->aNimi);
00068
00069     // Solmujen lisääminen.
00070     if (iArvo < pAlku->iArvo) {
00071         pAlku->pVasen = lisaaSolmu(pAlku->pVasen, pSolmu, iArvo);
00072     } else if (iArvo > pAlku->iArvo) {
00073         pAlku->pOikea = lisaaSolmu(pAlku->pOikea, pSolmu, iArvo);
00074     }
00075
00076     // Testataan, ovatko arvot samat.
00077     } else if (iArvo == pAlku->iArvo) {
00078         if (iVertailu < 0) {
00079             pAlku->pVasen = lisaaSolmu(pAlku->pVasen, pSolmu, iArvo);
00080         } else if (iVertailu > 0) {
00081             pAlku->pOikea = lisaaSolmu(pAlku->pOikea, pSolmu, iArvo);
00082         }
00083     }
00084
00085     // Selvitetaan paikka, johon solmu lisataan. Tasapainotetaan puu.
00086     pAlku->iPituus = 1 + suurempiLukuVertailu(puunPituus(pAlku->pVasen), puunPituus(pAlku->pOikea));
00087     tasapaino = tasapainoitaPuu(pAlku);
00088
00089     // vasen vasen tasapainotus
00090     if ((tasapaino > 1) && (iArvo < pAlku->pVasen->iArvo)) {
00091         return (oikeaPuoli(pAlku));
00092     }
00093
00094     // vasen oikea tasapainotus
00095     if ((tasapaino > 1) && (iArvo > pAlku->pVasen->iArvo)) {
00096         pAlku->pVasen = vasenPuoli(pAlku->pVasen);
00097         return (oikeaPuoli(pAlku));
00098     }
00099
00100     // oikea vasen tasapainotus
00101     if ((tasapaino < -1) && (iArvo < pAlku->pOikea->iArvo)) {
00102         pAlku->pOikea = oikeaPuoli(pAlku->pOikea);
00103         return (vasenPuoli(pAlku));
00104     }

```

```

00104
00105 // oikea oikea tasapainotus
00106 if ((tasapaino < -1) && (iArvo > pAlku->pOikea->iArvo)) {
00107     return (vasenPuoli(pAlku));
00108 }
00109
00110 return (pAlku);
00111 }
00112
00120 PUU *luoPuu(char *pNimi, PUU *pJuuriSolmu) {
00121     FILE *Tiedosto = NULL;
00122     char aRivi[LEN] = "";
00123     int iOtsikko = 0;
00124     char *p1 = NULL, *p2 = NULL;
00125     int iArvo = 0;
00126
00127     // Tiedoston avaus
00128     if ((Tiedosto = fopen(pNimi, "r")) == NULL) {
00129         perror("Tiedoston avaaminen epäonnistui, lopetetaan");
00130         exit(0);
00131     }
00132
00133     // Tiedoston lukeminen
00134     while ((fgets(aRivi, LEN, Tiedosto)) != NULL) {
00135         aRivi[strlen(aRivi) - 1] = '\0';
00136
00137         // Ohitetaan otsikko
00138         if (iOtsikko == 0) {
00139             iOtsikko++;
00140             continue;
00141         }
00142
00143         // Pilkkotaan rivit
00144         if ((p1 = strtok(aRivi, ";")) == NULL) {
00145             perror("Tiedoston pilkkominen epäonnistui, lopetetaan");
00146             exit(0);
00147         }
00148
00149         if ((p2 = strtok(NULL, ";")) == NULL) {
00150             perror("Tiedoston pilkkominen epäonnistui, lopetetaan");
00151             exit(0);
00152         }
00153
00154         iArvo = atoi(p2);
00155
00156         // Maaritetaan juuri solmu jos sitä ei ole määritetty.
00157         if (pJuuriSolmu == NULL) {
00158             pJuuriSolmu = lisaaSolmu(pJuuriSolmu, p1, iArvo);
00159             iOtsikko++;
00160             continue;
00161         }
00162
00163         pJuuriSolmu = lisaaSolmu(pJuuriSolmu, p1, iArvo);
00164     }
00165
00166     // Suljetaan tiedosto
00167     fclose(Tiedosto);
00168     return (pJuuriSolmu);
00169 }
00170
00177 void kirjoitaBinaaripuu(char *pNimi, PUU *pAlku) {
00178     if (pAlku != NULL) {
00179         kirjoitaTiedostoon(pNimi, pAlku);
00180         kirjoitaBinaaripuu(pNimi, pAlku->pVasen);
00181         kirjoitaBinaaripuu(pNimi, pAlku->pOikea);
00182     }
00183     return;
00184 }
00185
00192 void kirjoitaTiedostoon(char *pNimi, PUU *pAlku) {
00193     FILE *Tiedosto = NULL;
00194
00195     if (pAlku == NULL) {
00196         return;
00197     }
00198
00199     /* Tiedoston avaaminen. */
00200     if ((Tiedosto = fopen(pNimi, "a")) == NULL) {
00201         perror("Tiedoston avaaminen epäonnistui, lopetetaan");
00202         exit(0);
00203     }
00204     /* Tiedostoon kirjoittaminen. */
00205     fprintf(Tiedosto, "%s,%d\n", pAlku->aNimi, pAlku->iArvo);
00206
00207     /* Tiedoston sulkeminen. */
00208     fclose(Tiedosto);
00209     return;

```

```

00210 }
00211
00218 int tasapainoitaPuu(PUU *pAlku) {
00219     // Tarkistetaan, onko Null
00220     if (pAlku == NULL) {
00221         return (0);
00222     }
00223
00224     // Palautetaan tasapainokerroin
00225     return (puunPituus(pAlku->pVasen) - puunPituus(pAlku->pOikea));
00226 }
00227
00234 PUU *oikeaPuoli(PUU *pAlku) {
00235     // Tarkistetaan, onko pAlku tai pAlku oikea puoli NULL
00236     if ((pAlku->pVasen == NULL) || (pAlku == NULL)) {
00237         return (pAlku);
00238     }
00239
00240     // Muuttujien ja vakioiden alustaminen
00241     PUU *pMuuttuja1 = pAlku->pVasen;
00242     PUU *pMuuttuja2 = pMuuttuja1->pOikea;
00243
00244     // Sijoitetetaan arvoja
00245     pMuuttuja1->pOikea = pAlku;
00246     pAlku->pVasen = pMuuttuja2;
00247
00248     // Verrataan ja sijoitetaan suuremmat korkeudet
00249     pAlku->iPituus = suurempiLukuVertailu(puunPituus(pAlku->pVasen), puunPituus(pAlku->pOikea)) + 1;
00250     pMuuttuja1->iPituus =
00251         suurempiLukuVertailu(puunPituus(pMuuttuja1->pVasen), puunPituus(pMuuttuja1->pOikea)) + 1;
00252
00253     return pMuuttuja1;
00254 }
00255
00262 PUU *vasenPuoli(PUU *pAlku) {
00263     // Tarkistetaan, onko pAlku tai pAlku oikea puoli NULL
00264     if ((pAlku->pOikea == NULL) || (pAlku == NULL)) {
00265         return (pAlku);
00266     }
00267
00268     // Muuttujien ja vakioiden alustaminen
00269     PUU *pMuuttuja1 = pAlku->pOikea;
00270     PUU *pMuuttuja2 = pMuuttuja1->pVasen;
00271
00272     // Sijoitetetaan arvoja
00273     pMuuttuja1->pVasen = pAlku;
00274     pAlku->pOikea = pMuuttuja2;
00275
00276     // Verrataan ja sijoitetaan suuremmat korkeudet
00277     pAlku->iPituus = suurempiLukuVertailu(puunPituus(pAlku->pVasen), puunPituus(pAlku->pOikea)) + 1;
00278     pMuuttuja1->iPituus =
00279         suurempiLukuVertailu(puunPituus(pMuuttuja1->pVasen), puunPituus(pMuuttuja1->pOikea)) + 1;
00280
00281     return pMuuttuja1;
00282 }
00283
00290 int puunPituus(PUU *pAlku) {
00291     // Tarkistetaan, onko Solmu Null
00292     if (pAlku == NULL) {
00293         return (0);
00294     }
00295
00296     // Palauttaa solmun korkeuden
00297     return (pAlku->iPituus);
00298 }
00299
00307 int suurempiLukuVertailu(int iLuku1, int iLuku2) {
00308     int iPalautus = 0;
00309
00310     // Verrataan kumpi luvuista on suurempi.
00311     if (iLuku1 > iLuku2) {
00312         iPalautus = iLuku1;
00313     } else {
00314         iPalautus = iLuku2;
00315     }
00316
00317     // Palauttaa suuremman luvun.
00318     return (iPalautus);
00319 }
00320
00331 PUU *poistaSolmu(char *pNimi, PUU *pJuuriSolmu) {
00332     int iArvo = 0;
00333     PUU *pUusiSolmu = NULL;
00334
00335     if (pJuuriSolmu == NULL) {
00336         printf("Solmua ei löytynyt.\n");
00337         return (pJuuriSolmu);

```

```

00338     }
00339
00340     // Testataan, onko syöte nimi vai lukuarvo, ja määritetään lukuarvo puun läpikäymiseksi.
00341     if (onkoLuku(pNimi)) {
00342         iArvo = atoi(pNimi);
00343     } else {
00344         iArvo = nimenArvo(pNimi, pJuuriSolmu);
00345
00346         if (iArvo == 0) {
00347             printf("Solmua ei löytynyt tällä nimellä.\n");
00348             return (pJuuriSolmu);
00349         }
00350     }
00351
00352     // Lehtisolmun poistaminen.
00353     if (iArvo < pJuuriSolmu->iArvo) {
00354         pJuuriSolmu->pVasen = poistaSolmu(pNimi, pJuuriSolmu->pVasen);
00355     } else if (iArvo > pJuuriSolmu->iArvo) {
00356         pJuuriSolmu->pOikea = poistaSolmu(pNimi, pJuuriSolmu->pOikea);
00357     } else {
00358
00359         if (pJuuriSolmu->pVasen == NULL && pJuuriSolmu->pOikea == NULL) {
00360             free(pJuuriSolmu);
00361             pJuuriSolmu = NULL;
00362             printf("Solmu poistettu.\n");
00363             return (pJuuriSolmu);
00364
00365         } else if (pJuuriSolmu->pVasen == NULL || pJuuriSolmu->pOikea == NULL) {
00366             if (pJuuriSolmu->pVasen != NULL) {
00367                 pUusiSolmu = pJuuriSolmu->pVasen;
00368             } else {
00369                 pUusiSolmu = pJuuriSolmu->pOikea;
00370             }
00371             free(pJuuriSolmu);
00372             pJuuriSolmu = NULL;
00373             printf("Solmu poistettu.\n");
00374             return (pUusiSolmu);
00375
00376         } else {
00377             pUusiSolmu = etsiPienin(pJuuriSolmu->pOikea);
00378             pJuuriSolmu->iArvo = pUusiSolmu->iArvo;
00379             strcpy(pJuuriSolmu->aNimi, pUusiSolmu->aNimi);
00380             pJuuriSolmu->pOikea = poistaSolmu(pUusiSolmu->aNimi, pJuuriSolmu->pOikea);
00381             return (pJuuriSolmu);
00382         }
00383     }
00384
00385     pJuuriSolmu = paivitaPuu(pJuuriSolmu);
00386     return (pJuuriSolmu);
00387 }
00388
00398 int onkoLuku(char *pNimi) {
00399     int tosi = 0;
00400     char *ptr = pNimi;
00401
00402     while (*ptr) {
00403         if (!isdigit(*ptr)) {
00404             return tosi;
00405         }
00406         ptr++;
00407     }
00408     tosi = 1;
00409     return (tos);
00410 }
00411
00419 int nimenArvo(char *pNimi, PUU *pJuuriSolmu) {
00420     if (pJuuriSolmu == NULL) {
00421         return (0);
00422     }
00423
00424     if (strcmp(pJuuriSolmu->aNimi, pNimi) == 0) {
00425         int iArvo = pJuuriSolmu->iArvo;
00426         return (iArvo);
00427     }
00428
00429     int iArvo = nimenArvo(pNimi, pJuuriSolmu->pVasen);
00430     if (iArvo != 0) {
00431         return (iArvo);
00432     } else {
00433         int iArvo = nimenArvo(pNimi, pJuuriSolmu->pOikea);
00434         return (iArvo);
00435     }
00436 }
00437
00444 PUU *etsiPienin(PUU *pUusiSolmu) {
00445     PUU *pNykyinenSolmu = pUusiSolmu;
00446

```

```

00447     while (pNykyinenSolmu->pVasen != NULL) {
00448         pNykyinenSolmu = pNykyinenSolmu->pVasen;
00449     }
00450
00451     return (pNykyinenSolmu);
00452 }
00453
00460 PUU *paivitaPuu(PUU *pJuuriSolmu) {
00461     // Puun korkeuden päivittäminen.
00462     if (pJuuriSolmu == NULL) {
00463         return (pJuuriSolmu);
00464     }
00465
00466     int iLuku1 = puunPituus(pJuuriSolmu->pVasen);
00467     int iLuku2 = puunPituus(pJuuriSolmu->pOikea);
00468     pJuuriSolmu->iPituus = suurempiLukuVertailu(iLuku1, iLuku2) + 1;
00469
00470     // Puun tasapanottaminen.
00471     int iTasapaino = tasapainoitaPuu(pJuuriSolmu);
00472     // Vasen-vasen tapaus.
00473     if (iTasapaino > 1 && tasapainoitaPuu(pJuuriSolmu->pVasen) >= 0) {
00474         return oikeaPuoli(pJuuriSolmu);
00475     }
00476
00477     // Oikea-oikea tapaus.
00478     if (iTasapaino < -1 && tasapainoitaPuu(pJuuriSolmu->pOikea) <= 0) {
00479         return vasenPuoli(pJuuriSolmu);
00480     }
00481
00482     // Vasen-oikea tapaus.
00483     if (iTasapaino > 1 && tasapainoitaPuu(pJuuriSolmu->pVasen) < 0) {
00484         pJuuriSolmu->pVasen = vasenPuoli(pJuuriSolmu->pVasen);
00485         return oikeaPuoli(pJuuriSolmu);
00486     }
00487
00488     // Oikea-vasen tapaus.
00489     if (iTasapaino < -1 && tasapainoitaPuu(pJuuriSolmu->pOikea) > 0) {
00490         pJuuriSolmu->pOikea = oikeaPuoli(pJuuriSolmu->pOikea);
00491         return vasenPuoli(pJuuriSolmu);
00492     }
00493
00494     return (pJuuriSolmu);
00495 }

```

3.3 binaaripuu.h File Reference

```
#include "yleiset.h"
```

Classes

- struct [puu](#)

Typedefs

- typedef struct [puu](#) [PUU](#)

Functions

- [PUU](#) * [varaaMuistiaPuulle](#) (char *pSolmu, int iArvo)
Muistin varaaminen puu structin alkioille.
- [PUU](#) * [vapautaMuistiPuu](#) ([PUU](#) *pJuuriSolmu)
Muistin vapauttaminen puu structin alkioista JuuriSolmun ja kaikkien sen lapsie solmujen muisti vapautetaan.
- [PUU](#) * [lisaaSolmu](#) ([PUU](#) *pAlku, char *pSolmu, int iArvo)
Lisaa solmun puuhun.

- `PUU * luoPuu` (char *pNimi, `PUU *pJuuriSolmu`)
Luetaan tiedosto ja luodaan puu rakenne.
- int `tasapainoitaPuu` (`PUU *pAlku`)
Palauttaa solmun tasapainokertoimen.
- `PUU * oikeaPuoli` (`PUU *pAlku`)
Oikean puolen kierto.
- `PUU * vasenPuoli` (`PUU *pAlku`)
Vasemman puolen kierto.
- int `puunPituus` (`PUU *pAlku`)
Hakee solmun korkeuden.
- int `suurempiLukuVertailu` (int iLuku1, int iLuku2)
Vertaa kahta lukua ja palauttaa suuremman.
- `PUU * poistaSolmu` (char *pNimi, `PUU *pJuuriSolmu`)
Lehtisolmun poistaminen.
- int `onkoLuku` (char *pNimi)
Testaa, onko käyttäjän antama arvo nimi vai numeroarvo.
- int `nimenArvo` (char *pNimi, `PUU *pJuuriSolmu`)
Etsii nimen perusteella sen arvon.
- `PUU * etsiPienin` (`PUU *pUusiSolmu`)
Etsii pienimmän solmun arvon binääripuusta.
- `PUU * paivitaPuu` (`PUU *pJuuriSolmu`)
Tasapainotetaan puu poiston jälkeen.
- void `kirjoitaBinaaripuu` (char *pNimi, `PUU *pAlku`)
Tasapainoitettun binaaripuun kirjoittaminen tiedostoon.
- void `kirjoitaTiedostoon` (char *pKirjoitaTiedostoNimi, `PUU *pAlku`)
Kirjoittaa binaaripuun tiedostoon.

3.3.1 Typedef Documentation

3.3.1.1 PUU

```
typedef struct puu PUU
```

3.3.2 Function Documentation

3.3.2.1 etsiPienin()

```
PUU * etsiPienin (  
    PUU * pUusiSolmu)
```

Parameters

<code>pUusiSolmu</code>	Osoitin kasiteltavaan solmuun.
-------------------------	--------------------------------

Returns

PUU* Palauttaa osoittimen puun pienimpaan solmuun.

Definition at line 444 of file [binaaripuu.c](#).

```
00444 {
00445     PUU *pNykyinenSolmu = pUusiSolmu;
00446
00447     while (pNykyinenSolmu->pVasen != NULL) {
00448         pNykyinenSolmu = pNykyinenSolmu->pVasen;
00449     }
00450
00451     return(pNykyinenSolmu);
00452 }
```

3.3.2.2 kirjoitaBinaaripuu()

```
void kirjoitaBinaaripuu (
    char * pNimi,
    PUU * pAlku)
```

Parameters

<i>pNimi</i>	Kirjoitettavan tiedoston nimi.
<i>pAlku</i>	

Definition at line 177 of file [binaaripuu.c](#).

```
00177 {
00178     if (pAlku != NULL) {
00179         kirjoitaTiedostoon(pNimi, pAlku);
00180         kirjoitaBinaaripuu(pNimi, pAlku->pVasen);
00181         kirjoitaBinaaripuu(pNimi, pAlku->pOikea);
00182     }
00183     return;
00184 }
```

3.3.2.3 kirjoitaTiedostoon()

```
void kirjoitaTiedostoon (
    char * pNimi,
    PUU * pAlku)
```

Parameters

<i>pNimi</i>	Kirjoitettavan tiedoston nimi
<i>pAlku</i>	Solmu, joka kirjoitetaan tiedostoon

Definition at line 192 of file [binaaripuu.c](#).

```
00192 {
00193     FILE *Tiedosto = NULL;
00194
00195     if (pAlku == NULL) {
00196         return;
00197     }
00198
00199     /* Tiedoston avaaminen. */
00200     if ((Tiedosto = fopen(pNimi, "a")) == NULL) {
00201         perror("Tiedoston avaaminen epäonnistui, lopetetaan");
00202         exit(0);
00203     }
00204     /* Tiedostoon kirjoittaminen. */
00205     fprintf(Tiedosto, "%s,%d\n", pAlku->aNimi, pAlku->iArvo);
00206
00207     /* Tiedoston sulkeminen. */
00208     fclose(Tiedosto);
00209     return;
00210 }
```

3.3.2.4 lisaaSolmu()

```
PUU * lisaaSolmu (
    PUU * pAlku,
    char * pSolmu,
    int iArvo)
```

Tasapainottaa puun.

Parameters

<i>pAlku</i>	
<i>pSolmu</i>	Lisattava solmu.
<i>iArvo</i>	Solmun arvo.

Returns

PUU*

Definition at line 58 of file [binaaripuu.c](#).

```
00058                                     {
00059     int iVertailu = 0;
00060     int tasapaino = 0;
00061
00062     // Varataan muistia, jos muisti tyhjä.
00063     if (pAlku == NULL) {
00064         return varaaMuistiaPuulle(pSolmu, iArvo);
00065     }
00066
00067     iVertailu = strcmp(pSolmu, pAlku->aNimi);
00068
00069     // Solmujen lisääminen.
00070     if (iArvo < pAlku->iArvo) {
00071         pAlku->pVasen = lisaaSolmu(pAlku->pVasen, pSolmu, iArvo);
00072     } else if (iArvo > pAlku->iArvo) {
00073         pAlku->pOikea = lisaaSolmu(pAlku->pOikea, pSolmu, iArvo);
00074
00075         // Testataan, ovatko arvot samat.
00076     } else if (iArvo == pAlku->iArvo) {
00077         if (iVertailu < 0) {
00078             pAlku->pVasen = lisaaSolmu(pAlku->pVasen, pSolmu, iArvo);
00079         } else if (iVertailu > 0) {
00080             pAlku->pOikea = lisaaSolmu(pAlku->pOikea, pSolmu, iArvo);
00081         }
00082     }
00083
00084     // Selvitetaan paikka, johon solmu lisataan. Tasapainotetaan puu.
00085     pAlku->iPituus = 1 + suurempiLukuVertailu(puunPituus(pAlku->pVasen), puunPituus(pAlku->pOikea));
00086     tasapaino = tasapainoitaPuu(pAlku);
00087
00088     // vasen vasen tasapainotus
00089     if ((tasapaino > 1) && (iArvo < pAlku->pVasen->iArvo)) {
00090         return (oikeaPuoli(pAlku));
00091     }
00092
00093     // vasen oikea tasapainotus
00094     if ((tasapaino > 1) && (iArvo > pAlku->pVasen->iArvo)) {
00095         pAlku->pVasen = vasenPuoli(pAlku->pVasen);
00096         return (oikeaPuoli(pAlku));
00097     }
00098
00099     // oikea vasen tasapainotus
00100     if ((tasapaino < -1) && (iArvo < pAlku->pOikea->iArvo)) {
00101         pAlku->pOikea = oikeaPuoli(pAlku->pOikea);
00102         return (vasenPuoli(pAlku));
00103     }
00104
00105     // oikea oikea tasapainotus
00106     if ((tasapaino < -1) && (iArvo > pAlku->pOikea->iArvo)) {
00107         return (vasenPuoli(pAlku));
00108     }
00109
00110     return (pAlku);
00111 }
```

3.3.2.5 luoPuu()

```
PUU * luoPuu (
    char * pNimi,
    PUU * pJuuriSolmu)
```

Pilkotaan luetut rivit.

Parameters

<i>pNimi</i>	Luettavan tiedoston nimi.
<i>pJuuriSolmu</i>	Puun juurisolmu.

Returns

PUU* Palauttaa juuriSolmun.

Definition at line 120 of file [binaaripuu.c](#).

```
00120                                     {
00121     FILE *Tiedosto = NULL;
00122     char aRivi[LEN] = "";
00123     int iOtsikko = 0;
00124     char *p1 = NULL, *p2 = NULL;
00125     int iArvo = 0;
00126
00127     // Tiedoston avaus
00128     if ((Tiedosto = fopen(pNimi, "r")) == NULL) {
00129         perror("Tiedoston avaaminen epäonnistui, lopetetaan");
00130         exit(0);
00131     }
00132
00133     // Tiedoston lukeminen
00134     while ((fgets(aRivi, LEN, Tiedosto)) != NULL) {
00135         aRivi[strlen(aRivi) - 1] = '\0';
00136
00137         // Ohitetaan otsikko
00138         if (iOtsikko == 0) {
00139             iOtsikko++;
00140             continue;
00141         }
00142
00143         // Pilkotaan rivit
00144         if ((p1 = strtok(aRivi, ";")) == NULL) {
00145             perror("Tiedoston pilkkominen epäonnistui, lopetetaan");
00146             exit(0);
00147         }
00148
00149         if ((p2 = strtok(NULL, ";")) == NULL) {
00150             perror("Tiedoston pilkkominen epäonnistui, lopetetaan");
00151             exit(0);
00152         }
00153
00154         iArvo = atoi(p2);
00155
00156         // Maaritetaan juuri solmu jos sitä ei ole määritelty.
00157         if (pJuuriSolmu == NULL) {
00158             pJuuriSolmu = lisaaSolmu(pJuuriSolmu, p1, iArvo);
00159             iOtsikko++;
00160             continue;
00161         }
00162
00163         pJuuriSolmu = lisaaSolmu(pJuuriSolmu, p1, iArvo);
00164     }
00165
00166     // Suljetaan tiedosto
00167     fclose(Tiedosto);
00168     return (pJuuriSolmu);
00169 }
```

3.3.2.6 nimenArvo()

```
int nimenArvo (
    char * pNimi,
    PUU * pJuuriSolmu)
```

Parameters

<i>pNimi</i>	Poistettava nimi merkkijonona.
<i>pJuuriSolmu</i>	Osoitin kasiteltavaan solmuun.

Returns

int Palauttaa 0 tai 1, riippuen siitä, löytyykö nimen nimen pohjalta arvoa.

Definition at line 419 of file [binaaripuu.c](#).

```
00419                                     {
00420     if (pJuuriSolmu == NULL) {
00421         return (0);
00422     }
00423
00424     if (strcmp(pJuuriSolmu->aNimi, pNimi) == 0) {
00425         int iArvo = pJuuriSolmu->iArvo;
00426         return (iArvo);
00427     }
00428
00429     int iArvo = nimenArvo(pNimi, pJuuriSolmu->pVasen);
00430     if (iArvo != 0) {
00431         return (iArvo);
00432     } else {
00433         int iArvo = nimenArvo(pNimi, pJuuriSolmu->pOikea);
00434         return (iArvo);
00435     }
00436 }
```

3.3.2.7 oikeaPuoli()

```
PUU * oikeaPuoli (
    PUU * pAlku)
```

Puu järjestetään, jotta se on tasapainossa.

Parameters

<i>pAlku</i>	
--------------	--

Returns

PUU* Palauttaa solmun.

Definition at line 234 of file [binaaripuu.c](#).

```
00234                                     {
00235     // Tarkistetaan, onko pAlku tai pAlku oikea puoli NULL
00236     if ((pAlku->pVasen == NULL) || (pAlku == NULL)) {
00237         return (pAlku);
00238     }
00239
00240     // Muuttujien ja vakioden alustaminen
00241     PUU *pMuuttuja1 = pAlku->pVasen;
00242     PUU *pMuuttuja2 = pMuuttuja1->pOikea;
```

```

00243
00244 // Sijoitetetaan arvoja
00245 pMuuttuja1->pOikea = pAlku;
00246 pAlku->pVasen = pMuuttuja2;
00247
00248 // Verrataan ja sijoitetaan suuremmat korkeudet
00249 pAlku->iPituus = suurempiLukuVertailu(puunPituus(pAlku->pVasen), puunPituus(pAlku->pOikea)) + 1;
00250 pMuuttuja1->iPituus =
00251     suurempiLukuVertailu(puunPituus(pMuuttuja1->pVasen), puunPituus(pMuuttuja1->pOikea)) + 1;
00252
00253 return pMuuttuja1;
00254 }

```

3.3.2.8 onkoLuku()

```

int onkoLuku (
    char * pNimi)

```

Ilmoittaa, onko käyttäjän antama numero vai ei.

Parameters

<i>pNimi</i>	Poistettava nimi tai numeroarvo merkkijonona.
--------------	---

Returns

int Palauttaa 0 tai 1, riippuen siitä onko käyttäjän syöte numero.

Definition at line 398 of file [binaaripuu.c](#).

```

00398 {
00399     int tosi = 0;
00400     char *ptr = pNimi;
00401
00402     while (*ptr) {
00403         if (!isdigit(*ptr)) {
00404             return tosi;
00405         }
00406         ptr++;
00407     }
00408     tosi = 1;
00409     return (tos);
00410 }

```

3.3.2.9 paivitaPuu()

```

PUU * paivitaPuu (
    PUU * pJuuriSolmu)

```

Parameters

<i>pJuuriSolmu</i>	Osoitin kasiteltavaan solmuun.
--------------------	--------------------------------

Returns

PUU* palauttaa tasapainotetun solmun.

Definition at line 460 of file [binaaripuu.c](#).

```

00460                                     {
00461     // Puun korkeuden päivittäminen.
00462     if (pJuuriSolmu == NULL) {
00463         return (pJuuriSolmu);
00464     }
00465
00466     int iLuku1 = puunPituus(pJuuriSolmu->pVasen);
00467     int iLuku2 = puunPituus(pJuuriSolmu->pOikea);
00468     pJuuriSolmu->iPituus = suurempiLukuVertailu(iLuku1, iLuku2) + 1;
00469
00470     // Puun tasapanottaminen.
00471     int iTasapaino = tasapainoitaPuu(pJuuriSolmu);
00472     // Vasen-vasen tapaus.
00473     if (iTasapaino > 1 && tasapainoitaPuu(pJuuriSolmu->pVasen) >= 0) {
00474         return oikeaPuoli(pJuuriSolmu);
00475     }
00476
00477     // Oikea-oikea tapaus.
00478     if (iTasapaino < -1 && tasapainoitaPuu(pJuuriSolmu->pOikea) <= 0) {
00479         return vasenPuoli(pJuuriSolmu);
00480     }
00481
00482     // Vasen-oikea tapaus.
00483     if (iTasapaino > 1 && tasapainoitaPuu(pJuuriSolmu->pVasen) < 0) {
00484         pJuuriSolmu->pVasen = vasenPuoli(pJuuriSolmu->pVasen);
00485         return oikeaPuoli(pJuuriSolmu);
00486     }
00487
00488     // Oikea-vasen tapaus.
00489     if (iTasapaino < -1 && tasapainoitaPuu(pJuuriSolmu->pOikea) > 0) {
00490         pJuuriSolmu->pOikea = oikeaPuoli(pJuuriSolmu->pOikea);
00491         return vasenPuoli(pJuuriSolmu);
00492     }
00493
00494     return (pJuuriSolmu);
00495 }
```

3.3.2.10 poistaSolmu()

```

PUU * poistaSolmu (
    char * pNimi,
    PUU * pJuuriSolmu)
```

Poistaa lehtisolmun joko numeroarvon tai nimen perusteella.

Parameters

<i>pNimi</i>	Poistettava nimi tai numeroarvo merkkijonona.
<i>pJuuriSolmu</i>	Osoitin kasiteltavaan puun solmuun.
<i>iArvo</i>	Haettava numeroarvo.

Returns

PUU* Uusi osoitin puun solmuun, NULL jos puu tyhjeni.

Definition at line 331 of file [binaaripuu.c](#).

```

00331                                     {
00332     int iArvo = 0;
00333     PUU *pUusiSolmu = NULL;
00334
00335     if (pJuuriSolmu == NULL) {
00336         printf("Solmua ei löytynyt.\n");
```

```

00337         return (pJuuriSolmu);
00338     }
00339
00340     // Testataan, onko syöte nimi vai lukuarvo, ja määritetään lukuarvo puun läpikäymiseksi.
00341     if (onkoLuku(pNimi)) {
00342         iArvo = atoi(pNimi);
00343     } else {
00344         iArvo = nimenArvo(pNimi, pJuuriSolmu);
00345
00346         if (iArvo == 0) {
00347             printf("Solmua ei löytynyt tällä nimellä.\n");
00348             return (pJuuriSolmu);
00349         }
00350     }
00351
00352     // Lehtisolmun poistaminen.
00353     if (iArvo < pJuuriSolmu->iArvo) {
00354         pJuuriSolmu->pVasen = poistaSolmu(pNimi, pJuuriSolmu->pVasen);
00355     } else if (iArvo > pJuuriSolmu->iArvo) {
00356         pJuuriSolmu->pOikea = poistaSolmu(pNimi, pJuuriSolmu->pOikea);
00357     } else {
00358
00359         if (pJuuriSolmu->pVasen == NULL && pJuuriSolmu->pOikea == NULL) {
00360             free(pJuuriSolmu);
00361             pJuuriSolmu = NULL;
00362             printf("Solmu poistettu.\n");
00363             return (pJuuriSolmu);
00364
00365         } else if (pJuuriSolmu->pVasen == NULL || pJuuriSolmu->pOikea == NULL) {
00366             if (pJuuriSolmu->pVasen != NULL) {
00367                 pUusiSolmu = pJuuriSolmu->pVasen;
00368             } else {
00369                 pUusiSolmu = pJuuriSolmu->pOikea;
00370             }
00371             free(pJuuriSolmu);
00372             pJuuriSolmu = NULL;
00373             printf("Solmu poistettu.\n");
00374             return (pUusiSolmu);
00375
00376         } else {
00377             pUusiSolmu = etsiPienin(pJuuriSolmu->pOikea);
00378             pJuuriSolmu->iArvo = pUusiSolmu->iArvo;
00379             strcpy(pJuuriSolmu->aNimi, pUusiSolmu->aNimi);
00380             pJuuriSolmu->pOikea = poistaSolmu(pUusiSolmu->aNimi, pJuuriSolmu->pOikea);
00381             return (pJuuriSolmu);
00382         }
00383     }
00384
00385     pJuuriSolmu = paivitaPuu(pJuuriSolmu);
00386     return (pJuuriSolmu);
00387 }

```

3.3.2.11 puunPituus()

```

int puunPituus (
    PUU * pAlku)

```

Parameters

<i>pAlku</i>	Kohta, josta korkeus haetaan
--------------	------------------------------

Returns

int Palauttaa solmun korkeuden

Definition at line 290 of file [binaaripuu.c](#).

```

00290     {
00291         // Tarkistetaan, onko Solmu Null
00292         if (pAlku == NULL) {
00293             return (0);
00294         }
00295
00296         // Palauttaa solmun korkeuden
00297         return (pAlku->iPituus);
00298     }

```


3.3.2.12 suurempiLukuVertailu()

```
int suurempiLukuVertailu (
    int iLuku1,
    int iLuku2)
```

Parameters

<i>iLuku1</i>	Ensimmäinen verrattava kokonaisluku.
<i>iLuku2</i>	Toinen verrattava kokonaisluku.

Returns

int Palauttaa suuremman luvun.

Definition at line 307 of file [binaaripuu.c](#).

```
00307                                     {
00308     int iPalautus = 0;
00309
00310     // Verrataan kumpi luvuista on suurempi.
00311     if (iLuku1 > iLuku2) {
00312         iPalautus = iLuku1;
00313     } else {
00314         iPalautus = iLuku2;
00315     }
00316
00317     // Palauttaa suuremman luvun.
00318     return (iPalautus);
00319 }
```

3.3.2.13 tasapainoitaPuu()

```
int tasapainoitaPuu (
    PUU * pAlku)
```

Parameters

<i>pAlku</i>	Solmu, josta tasapainokerroin halutaan
--------------	--

Returns

int

Definition at line 218 of file [binaaripuu.c](#).

```
00218                                     {
00219     // Tarkistetaan, onko Null
00220     if (pAlku == NULL) {
00221         return (0);
00222     }
00223
00224     // Palautetaan tasapainokerroin
00225     return (puunPituus(pAlku->pVasen) - puunPituus(pAlku->pOikea));
00226 }
```

3.3.2.14 vapautaMuistiPuu()

```
PUU * vapautaMuistiPuu (
    PUU * pJuuriSolmu)
```

Parameters

<i>pJuuriSolmu</i>	Puun juurisolmu
--------------------	-----------------

Returns

PUU*

Definition at line 39 of file [binaaripuu.c](#).

```
00039                                     {
00040     // Muistin vapauttaminen
00041     if (pJuuriSolmu != NULL) {
00042         vapautaMuistiPuu(pJuuriSolmu->pVasen);
00043         vapautaMuistiPuu(pJuuriSolmu->pOikea);
00044         free(pJuuriSolmu);
00045     }
00046     pJuuriSolmu = NULL;
00047     return (pJuuriSolmu);
00048 }
```

3.3.2.15 varaaMuistiaPuulle()

```
PUU * varaaMuistiaPuulle (
    char * pSolmu,
    int iArvo)
```

Parameters

<i>pSolmu</i>	Solmu, jolle muistia varataan
<i>iArvo</i>	Arvo, jolle muistia varataan

Returns

PUU*

Definition at line 16 of file [binaaripuu.c](#).

```
00016                                     {
00017     PUU *pUusi = NULL;
00018
00019     // Muistin varaaminen
00020     if ((pUusi = (PUU *)malloc(sizeof(PUU))) == NULL) {
00021         perror("Muistin varaus epäonnistui, lopetetaan");
00022         exit(0);
00023     }
00024
00025     strcpy(pUusi->aNimi, pSolmu);
00026     pUusi->iArvo = iArvo;
00027     pUusi->iPituus = 1;
00028     pUusi->pVasen = NULL;
00029     pUusi->pOikea = NULL;
00030     return (pUusi);
00031 }
```

3.3.2.16 vasenPuoli()

```
PUU * vasenPuoli (
    PUU * pAlku)
```

Puu järjestetään, jotta se on tasapainossa.

Parameters

<i>pAlku</i>	
--------------	--

Returns

PUU* Palauttaa solmun.

Definition at line 262 of file [binaaripuu.c](#).

```
00262     {
00263     // Tarkistetaan, onko pAlku tai pAlku oikea puoli NULL
00264     if ((pAlku->pOikea == NULL) || (pAlku == NULL)) {
00265         return (pAlku);
00266     }
00267
00268     // Muuttujien ja vakioiden alustaminen
00269     PUU *pMuuttuja1 = pAlku->pOikea;
00270     PUU *pMuuttuja2 = pMuuttuja1->pVasen;
00271
00272     // Sijoitetetaan arvoja
00273     pMuuttuja1->pVasen = pAlku;
00274     pAlku->pOikea = pMuuttuja2;
00275
00276     // Verrataan ja sijoitetaan suuremmat korkeudet
00277     pAlku->iPituus = suurempiLukuVertailu(puunPituus(pAlku->pVasen), puunPituus(pAlku->pOikea)) + 1;
00278     pMuuttuja1->iPituus =
00279         suurempiLukuVertailu(puunPituus(pMuuttuja1->pVasen), puunPituus(pMuuttuja1->pOikea)) + 1;
00280
00281     return pMuuttuja1;
00282 }
```

3.4 binaaripuu.h

[Go to the documentation of this file.](#)

```
00001 #ifndef BINAARIPUU_H
00002 #define BINAARIPUU_H
00003 #include "yleiset.h"
00004
00005 typedef struct puu {
00006     char aNimi[LEN];
00007     int iArvo;
00008     int iPituus;
00009     struct puu *pOikea;
00010     struct puu *pVasen;
00011 } PUU;
00012
00013 PUU *varaaMuistiaPuulle(char *pSolmu, int iArvo);
00014 PUU *vapautaMuistiPuu(PUU *pJuuriSolmu);
00015 PUU *lisaaSolmu(PUU *pAlku, char *pSolmu, int iArvo);
00016 PUU *luoPuu(char *pNimi, PUU *pJuuriSolmu);
00017
00018 int tasapainoitaPuu(PUU *pAlku);
00019 PUU *oikeaPuoli(PUU *pAlku);
00020 PUU *vasenPuoli(PUU *pAlku);
00021 int puunPituus(PUU *pAlku);
00022 int suurempiLukuVertailu(int iLuku1, int iLuku2);
00023
00024 PUU *poistaSolmu(char *pNimi, PUU *pJuuriSolmu);
00025 int onkoLuku(char *pNimi);
00026 int nimenArvo(char *pNimi, PUU *pJuuriSolmu);
00027 PUU *etsiPienin(PUU *pUusiSolmu);
00028 PUU *paivitaPuu(PUU *pJuuriSolmu);
00029
00030 void kirjoitaBinaaripuu(char *pNimi, PUU *pAlku);
00031 void kirjoitaTiedostoon(char *pKirjoitaTiedostoNimi, PUU *pAlku);
00032
00033 #endif
```

3.5 haut.c File Reference

```
#include "haut.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

Functions

- int [syvyyshaku](#) (char *pNimi, [PUU](#) *pAlku, int iArvo)
Tekee syvyysshaun numeroarvon pohjalta.
- void [tarkistaLoytyykoSyvyysshaulla](#) (char *aNimiKirjoitettava, [PUU](#) *pJuuriSolmu, int iArvo)
Tarkistaa, loytyyko etsitty arvo syvyysshaussa.
- void [leveyshaku](#) (char *pNimi, [PUU](#) *pJuuriSolmu, char *pHaku)
Tekee leveysshaun nimen pohjalta.
- [JONO](#) * [varaaMuistiaJonolle](#) ([PUU](#) *pSolmu)
Varaa muistia jonolle.
- [JONO](#) * [vapautaMuistiJono](#) ([JONO](#) *pAlku)
Vapauttaa jonon varaaman muistin.
- int [binaarihaku](#) (char *pNimi, [PUU](#) *pJuuriSolmu, int iArvo)
Binaarihaku perustuen numeroarvoon.

3.5.1 Function Documentation

3.5.1.1 binaarihaku()

```
int binaarihaku (
    char * pNimi,
    PUU * pJuuriSolmu,
    int iArvo)
```

Binaarihaku, joka tehdään käyttäjän antaman numeroarvon perusteella.

Parameters

<i>pNimi</i>	Kirjoitettavan tiedoston nimi.
<i>pJuuriSolmu</i>	Osoitin kasiteltavaan puun solmuun.
<i>iArvo</i>	Haettava numeroarvo.

Returns

int Palauttaa joko 0 tai 1, riippuen siitä, löytyykö numeroarvo binaaripuusta.

Definition at line 176 of file [haut.c](#).

```

00176                                     {
00177
00178     if (pJuuriSolmu == NULL) {
00179         return (0);
00180     }
00181
00182     kirjoitaTiedostoon(pNimi, pJuuriSolmu);
00183
00184     if (iArvo == pJuuriSolmu->iArvo) {
00185         printf("Hakemasi arvo '%d' löytyi!\n", iArvo);
00186         return (1);
00187     } else if (iArvo < pJuuriSolmu->iArvo) {
00188         return binaarihaku(pNimi, pJuuriSolmu->pVasen, iArvo);
00189     } else {
00190         return binaarihaku(pNimi, pJuuriSolmu->pOikea, iArvo);
00191     }
00192 }
```

3.5.1.2 leveyshaku()

```

void leveyshaku (
    char * pNimi,
    PUU * pJuuriSolmu,
    char * pHaku)
```

Tekee leveyshaun käyttäjän antaman nimen pohjalta. Kirjoittaa leveyshaun polun käyttäjän määrittelemään tiedoston.

Parameters

<i>pNimi</i>	Käyttäjän antama kirjoitettavan tiedoston nimi.
<i>pAlku</i>	Osoitin puu tietueen alkuun, eli ensimmäiseen solmuun.
<i>pHaku</i>	Osoitin haettavan nimen merkkijonoon.

Returns

void

Definition at line 74 of file [haut.c](#).

```

00074                                     {
00075     // Leveyshaku nimen mukaan, kirjoitetaan käydyt solmut tiedostoon.
00076     JONO *pAlku = varaaMuistiaJonolle(pJuuriSolmu);
00077     JONO *pLoppu = pAlku;
00078     JONO *pEdellinen = NULL;
00079     int iVertailu = 0;
00080     int iLoytyi = 0;
00081
00082     if (pJuuriSolmu == NULL) {
00083         printf("Puu on tyhjä, luo puurakenne ennen leveyshakua.\n");
00084         return;
00085     }
00086
00087     while (pAlku != NULL) {
00088         PUU *ptr = pAlku->pSolmu;
00089         kirjoitaTiedostoon(pNimi, ptr);
00090
00091         iVertailu = strcmp(ptr->aNimi, pHaku);
00092
00093         if (iVertailu == 0) {
00094             printf("Nimi '%s' löytyi puusta.\n", ptr->aNimi);
00095             iLoytyi = 1;
00096         }
00097     }
```

```

00096         break;
00097     }
00098
00099     if (ptr->pVasen != NULL) {
00100         pLoppu->pSeuraava = varaaMuistiaJonolle(ptr->pVasen);
00101         pLoppu = pLoppu->pSeuraava;
00102     }
00103
00104     if (ptr->pOikea != NULL) {
00105         pLoppu->pSeuraava = varaaMuistiaJonolle(ptr->pOikea);
00106         pLoppu = pLoppu->pSeuraava;
00107     }
00108
00109     pEdellinen = pAlku;
00110     pAlku = pAlku->pSeuraava;
00111     free(pEdellinen);
00112 }
00113
00114 if (iLoytyi == 0) {
00115     printf("Hakemaasi nimeä ei löytynyt puusta.\n");
00116 }
00117 pAlku = vapautaMuistiJono(pAlku);
00118 return;
00119 }

```

3.5.1.3 syvyyshaku()

```

int syvyyshaku (
    char * pNimi,
    PUU * pAlku,
    int iArvo)

```

Tekee syvyyshaun käyttäjän antaman numeroarvon pohjalta. Kirjoittaa syvyyshaun polun käyttäjän maarittelemaan tiedostoon.

Parameters

<i>pNimi</i>	Käyttäjän antama kirjoitettavan tiedoston nimi.
<i>pAlku</i>	Osoitin puu tietueen alkuun, eli ensimmäiseen solmuun.
<i>iArvo</i>	Arvo, jota etsitään syvyyshaussa.

Returns

int Palauttaa arvon 0 tai 1, riippuen siitä, löytyykö etsitty arvo.

Definition at line 17 of file [haut.c](#).

```

00017                                     {
00018     // Syvyyshaku arvon mukaan, kirjoitetaan käydyt solmut tiedostoon.
00019     int iLoytyi = 0;
00020
00021     if (pAlku == NULL) {
00022         return 0;
00023     }
00024
00025     if (pAlku != NULL) {
00026         kirjoitaTiedostoon(pNimi, pAlku);
00027
00028         if (pAlku->iArvo == iArvo) {
00029             iLoytyi = 1;
00030             printf("Arvo %d löytyi.\n", iArvo);
00031         } else {
00032             if (iLoytyi == 0) {
00033                 iLoytyi = syvyyshaku(pNimi, pAlku->pVasen, iArvo);
00034             }
00035             if (iLoytyi == 0) {
00036                 iLoytyi = syvyyshaku(pNimi, pAlku->pOikea, iArvo);
00037             }
00038         }
00039     }
00040     return (iLoytyi);
00041 }

```

3.5.1.4 tarkistaLoytvykoSyvyysshaulla()

```
void tarkistaLoytvykoSyvyysshaulla (
    char * aNimiKirjoitettava,
    PUU * pJuuriSolmu,
    int iArvo)
```

Tarkistaa, loytvyko syvyysshaussa etsitty arvo. Jos arvoa ei loydy, tulostaa tiedon siita.

Parameters

<i>aNimiKirjoitettava</i>	Kayttajan antama kirjoitettavan tiedoston nimi.
<i>pJuuriSolmu</i>	Osoitin puu tietueen alkuun, eli ensimmaiseen solmuun.
<i>iArvo</i>	Syvyysshaussa haettava numeroarvo.

Returns

void

Definition at line 54 of file [haut.c](#).

```
00054                                     {
00055     int iLoytyi = 0;
00056     iLoytyi = syvyysshaku(aNimiKirjoitettava, pJuuriSolmu, iArvo);
00057     if (iLoytyi == 0) {
00058         printf("Arvoa %d ei loytynyt puusta.\n", iArvo);
00059     }
00060     return;
00061 }
```

3.5.1.5 vapautaMuistiJono()

```
JONO * vapautaMuistiJono (
    JONO * pAlku)
```

Vapauttaa jonon alkioita varten varatun muistin.

Parameters

<i>pAlku</i>	Osoitin jonon ensimmaiseen alkioon.
--------------	-------------------------------------

Returns

pAlku Palauttaa NULL, koska jono on tyhja.

Definition at line 152 of file [haut.c](#).

```
00152                                     {
00153     // Jonon muistin vapauttaminen.
00154     JONO *ptr = pAlku;
00155     JONO *pSeuraava = NULL;
00156
00157     while (ptr != NULL) {
00158         pSeuraava = ptr->pSeuraava;
00159         free(ptr);
00160         ptr = pSeuraava;
00161     }
00162     pAlku = NULL;
00163     return (pAlku);
00164 }
```

3.5.1.6 varaaMuistiaJonolle()

```
JONO * varaaMuistiaJonolle (
    PUU * pSolmu)
```

Varaa muistia jonon uudelle alkiole leveyshakua varten.

Parameters

<i>pSolmu</i>	Osoitin kasiteltavaan puun solmuun.
---------------	-------------------------------------

Returns

pUusi Osoitin uuteen jonon alkioon.

Definition at line 129 of file [haut.c](#).

```
00129                                     {
00130     JONO *pUusi = NULL;
00131
00132     // Muistin varaaminen jonolle.
00133     if ((pUusi = (JONO *)malloc(sizeof(JONO))) == NULL) {
00134         perror("Muistin varaus epäonnistui, lopetetaan");
00135         exit(0);
00136     }
00137
00138     pUusi->pSolmu = pSolmu;
00139     pUusi->pSeuraava = NULL;
00140
00141     return (pUusi);
00142 }
```

3.6 haut.c

[Go to the documentation of this file.](#)

```
00001 #include "haut.h"
00002 #include <stdio.h>
00003 #include <stdlib.h>
00004 #include <string.h>
00005
00017 int syvyyshaku(char *pNimi, PUU *pAlku, int iArvo) {
00018     // Syvyyshaku arvon mukaan, kirjoitetaan käydyt solmut tiedostoon.
00019     int iLoytyi = 0;
00020
00021     if (pAlku == NULL) {
00022         return 0;
00023     }
00024
00025     if (pAlku != NULL) {
00026         kirjoitaTiedostoon(pNimi, pAlku);
00027
00028         if (pAlku->iArvo == iArvo) {
00029             iLoytyi = 1;
00030             printf("Arvo %d löytyi.\n", iArvo);
00031         } else {
00032             if (iLoytyi == 0) {
00033                 iLoytyi = syvyyshaku(pNimi, pAlku->pVasen, iArvo);
00034             }
00035             if (iLoytyi == 0) {
00036                 iLoytyi = syvyyshaku(pNimi, pAlku->pOikea, iArvo);
00037             }
00038         }
00039     }
00040     return (iLoytyi);
00041 }
00042
00054 void tarkistaLoytyykoSyvyysaulla(char *aNimiKirjoitettava, PUU *pJuuriSolmu, int iArvo) {
00055     int iLoytyi = 0;
00056     iLoytyi = syvyyshaku(aNimiKirjoitettava, pJuuriSolmu, iArvo);
```



```

00057     if (iLoytyi == 0) {
00058         printf("Arvoa %d ei löytynyt puusta.\n", iArvo);
00059     }
00060     return;
00061 }
00062
00074 void leveyshaku(char *pNimi, PUU *pJuuriSolmu, char *pHaku) {
00075     // Leveyshaku nimen mukaan, kirjoitetaan käydyt solmut tiedostoon.
00076     JONO *pAlku = varaaMuistiaJonolle(pJuuriSolmu);
00077     JONO *pLoppu = pAlku;
00078     JONO *pEdellinen = NULL;
00079     int iVertailu = 0;
00080     int iLoytyi = 0;
00081
00082     if (pJuuriSolmu == NULL) {
00083         printf("Puu on tyhjä, luo puurakenne ennen leveyshakua.\n");
00084         return;
00085     }
00086
00087     while (pAlku != NULL) {
00088         PUU *ptr = pAlku->pSolmu;
00089         kirjoitaTiedostoon(pNimi, ptr);
00090
00091         iVertailu = strcmp(ptr->aNimi, pHaku);
00092
00093         if (iVertailu == 0) {
00094             printf("Nimi '%s' löytyi puusta.\n", ptr->aNimi);
00095             iLoytyi = 1;
00096             break;
00097         }
00098
00099         if (ptr->pVasen != NULL) {
00100             pLoppu->pSeuraava = varaaMuistiaJonolle(ptr->pVasen);
00101             pLoppu = pLoppu->pSeuraava;
00102         }
00103
00104         if (ptr->pOikea != NULL) {
00105             pLoppu->pSeuraava = varaaMuistiaJonolle(ptr->pOikea);
00106             pLoppu = pLoppu->pSeuraava;
00107         }
00108
00109         pEdellinen = pAlku;
00110         pAlku = pAlku->pSeuraava;
00111         free(pEdellinen);
00112     }
00113
00114     if (iLoytyi == 0) {
00115         printf("Hakemaasi nimeä ei löytynyt puusta.\n");
00116     }
00117     pAlku = vapautaMuistiJono(pAlku);
00118     return;
00119 }
00120
00129 JONO *varaaMuistiaJonolle(PUU *pSolmu) {
00130     JONO *pUusi = NULL;
00131
00132     // Muistin varaaminen jonolle.
00133     if ((pUusi = (JONO *)malloc(sizeof(JONO))) == NULL) {
00134         perror("Muistin varaus epäonnistui, lopetetaan");
00135         exit(0);
00136     }
00137
00138     pUusi->pSolmu = pSolmu;
00139     pUusi->pSeuraava = NULL;
00140
00141     return (pUusi);
00142 }
00143
00152 JONO *vapautaMuistiJono(JONO *pAlku) {
00153     // Jonon muistin vapauttaminen.
00154     JONO *ptr = pAlku;
00155     JONO *pSeuraava = NULL;
00156
00157     while (ptr != NULL) {
00158         pSeuraava = ptr->pSeuraava;
00159         free(ptr);
00160         ptr = pSeuraava;
00161     }
00162     pAlku = NULL;
00163     return (pAlku);
00164 }
00165
00176 int binaarihaku(char *pNimi, PUU *pJuuriSolmu, int iArvo) {
00177
00178     if (pJuuriSolmu == NULL) {
00179         return (0);
00180     }

```

```

00181
00182     kirjoitaTiedostoon(pNimi, pJuuriSolmu);
00183
00184     if (iArvo == pJuuriSolmu->iArvo) {
00185         printf("Hakemasi arvo '%d' löytyi!\n", iArvo);
00186         return (1);
00187     } else if (iArvo < pJuuriSolmu->iArvo) {
00188         return binaarihaku(pNimi, pJuuriSolmu->pVasen, iArvo);
00189     } else {
00190         return binaarihaku(pNimi, pJuuriSolmu->pOikea, iArvo);
00191     }
00192 }

```

3.7 haut.h File Reference

```
#include "binaaripuu.h"
```

Classes

- struct [jono](#)

Typedefs

- typedef struct [jono](#) JONO

Functions

- int [syvyyshaku](#) (char *pKirjoitaTiedostoNimi, [PUU](#) *pJuuriSolmu, int iArvo)
Tekee syvyysshaun numeroarvon pohjalta.
- void [tarkistaLoytvykoSyvyysshaulla](#) (char *aNimiKirjoitettava, [PUU](#) *pJuuriSolmu, int iArvo)
Tarkistaa, löytyykö etsitty arvo syvyysshaussa.
- void [leveyshaku](#) (char *pKirjoitaTiedostoNimi, [PUU](#) *pAlku, char *pHaku)
Tekee leveyshaun nimen pohjalta.
- JONO * [varaaMuistiaJonolle](#) ([PUU](#) *pSolmu)
Varaa muistia jonolle.
- JONO * [vapautaMuistiJono](#) (JONO *pAlku)
Vapauttaa jonon varaaman muistin.
- int [binaarihaku](#) (char *pNimi, [PUU](#) *pJuuriSolmu, int iArvo)
Binaarihaku perustuen numeroarvoon.

3.7.1 Typedef Documentation

3.7.1.1 JONO

```
typedef struct jono JONO
```

3.7.2 Function Documentation

3.7.2.1 binaarihaku()

```
int binaarihaku (
    char * pNimi,
    PUU * pJuuriSolmu,
    int iArvo)
```

Binaarihaku, joka tehdään käyttäjän antaman numeroarvon perusteella.

Parameters

<i>pNimi</i>	Kirjoitettavan tiedoston nimi.
<i>pJuuriSolmu</i>	Osoitin kasiteltavaan puun solmuun.
<i>iArvo</i>	Haettava numeroarvo.

Returns

int Palauttaa joko 0 tai 1, riippuen siitä, löytyykö numeroarvo binaaripuusta.

Definition at line 176 of file [haut.c](#).

```
00176                                     {
00177
00178     if (pJuuriSolmu == NULL) {
00179         return (0);
00180     }
00181
00182     kirjoitaTiedostoon(pNimi, pJuuriSolmu);
00183
00184     if (iArvo == pJuuriSolmu->iArvo) {
00185         printf("Hakemasi arvo '%d' löytyi!\n", iArvo);
00186         return (1);
00187     } else if (iArvo < pJuuriSolmu->iArvo) {
00188         return binaarihaku(pNimi, pJuuriSolmu->pVasen, iArvo);
00189     } else {
00190         return binaarihaku(pNimi, pJuuriSolmu->pOikea, iArvo);
00191     }
00192 }
```

3.7.2.2 leveyshaku()

```
void leveyshaku (
    char * pNimi,
    PUU * pJuuriSolmu,
    char * pHaku)
```

Tekee leveyshaun käyttäjän antaman nimen pohjalta. Kirjoittaa leveyshaun polun käyttäjän maarittelemaan tiedoston.

Parameters

<i>pNimi</i>	Käyttäjän antama kirjoitettavan tiedoston nimi.
<i>pAlku</i>	Osoitin puu tietueen alkuun, eli ensimmäiseen solmuun.
<i>pHaku</i>	Osoitin haettavan nimen merkkijonoon.

Returns

void

Definition at line 74 of file [haut.c](#).

```

00074                                     {
00075     // Leveyshaku nimen mukaan, kirjoitetaan käydyt solmut tiedostoon.
00076     JONO *pAlku = varaaMuistiaJonolle(pJuuriSolmu);
00077     JONO *pLoppu = pAlku;
00078     JONO *pEdellinen = NULL;
00079     int iVertailu = 0;
00080     int iLoytyi = 0;
00081
00082     if (pJuuriSolmu == NULL) {
00083         printf("Puu on tyhjä, luo puurakenne ennen leveyshakua.\n");
00084         return;
00085     }
00086
00087     while (pAlku != NULL) {
00088         PUU *ptr = pAlku->pSolmu;
00089         kirjoitaTiedostoon(pNimi, ptr);
00090
00091         iVertailu = strcmp(ptr->aNimi, pHaku);
00092
00093         if (iVertailu == 0) {
00094             printf("Nimi '%s' löytyi puusta.\n", ptr->aNimi);
00095             iLoytyi = 1;
00096             break;
00097         }
00098
00099         if (ptr->pVasen != NULL) {
00100             pLoppu->pSeuraava = varaaMuistiaJonolle(ptr->pVasen);
00101             pLoppu = pLoppu->pSeuraava;
00102         }
00103
00104         if (ptr->pOikea != NULL) {
00105             pLoppu->pSeuraava = varaaMuistiaJonolle(ptr->pOikea);
00106             pLoppu = pLoppu->pSeuraava;
00107         }
00108
00109         pEdellinen = pAlku;
00110         pAlku = pAlku->pSeuraava;
00111         free(pEdellinen);
00112     }
00113
00114     if (iLoytyi == 0) {
00115         printf("Hakemaasi nimeä ei löytynyt puusta.\n");
00116     }
00117     pAlku = vapautaMuistiJono(pAlku);
00118     return;
00119 }

```

3.7.2.3 syvyyshaku()

```

int syvyyshaku (
    char * pNimi,
    PUU * pAlku,
    int iArvo)

```

Tekee syvyyshaun käyttäjän antaman numeroarvon pohjalta. Kirjoittaa syvyyshaun polun käyttäjän määrittelemään tiedostoon.

Parameters

<i>pNimi</i>	Käyttäjän antama kirjoitettavan tiedoston nimi.
<i>pAlku</i>	Osoitin puu tietueen alkuun, eli ensimmäiseen solmuun.
<i>iArvo</i>	Arvo, jota etsitään syvyyshaussa.

Returns

int Palauttaa arvon 0 tai 1, riippuen siitä, löytyykö etsitty arvo.

Definition at line 17 of file [haut.c](#).

```

00017                                     {
00018     // Syvyyshaku arvon mukaan, kirjoitetaan käydyt solmut tiedostoon.
00019     int iLoytyi = 0;
00020
00021     if (pAlku == NULL) {
00022         return 0;
00023     }
00024
00025     if (pAlku != NULL) {
00026         kirjoitaTiedostoon(pNimi, pAlku);
00027
00028         if (pAlku->iArvo == iArvo) {
00029             iLoytyi = 1;
00030             printf("Arvo %d löytyi.\n", iArvo);
00031         } else {
00032             if (iLoytyi == 0) {
00033                 iLoytyi = syvyyshaku(pNimi, pAlku->pVasen, iArvo);
00034             }
00035             if (iLoytyi == 0) {
00036                 iLoytyi = syvyyshaku(pNimi, pAlku->pOikea, iArvo);
00037             }
00038         }
00039     }
00040     return (iLoytyi);
00041 }
```

3.7.2.4 tarkistaLoytyykoSyvyyshaulla()

```

void tarkistaLoytyykoSyvyyshaulla (
    char * aNimiKirjoitettava,
    PUU * pJuuriSolmu,
    int iArvo)
```

Tarkistaa, löytyykö syvyyshaussa etsitty arvo. Jos arvoa ei löydy, tulostaa tiedon siitä.

Parameters

<i>aNimiKirjoitettava</i>	Kayttajan antama kirjoitettavan tiedoston nimi.
<i>pJuuriSolmu</i>	Osoitin puu tietueen alkuun, eli ensimmäiseen solmuun.
<i>iArvo</i>	Syvyyhaussa haettava numeroarvo.

Returns

void

Definition at line 54 of file [haut.c](#).

```

00054                                     {
00055     int iLoytyi = 0;
00056     iLoytyi = syvyyshaku(aNimiKirjoitettava, pJuuriSolmu, iArvo);
00057     if (iLoytyi == 0) {
00058         printf("Arvoa %d ei löytynyt puusta.\n", iArvo);
00059     }
00060     return;
00061 }
```

3.7.2.5 vapautaMuistiJono()

```
JONO * vapautaMuistiJono (
    JONO * pAlku)
```

Vapauttaa jonon alkioita varten varatun muistin.

Parameters

<i>pAlku</i>	Osoitin jonon ensimmäiseen alkioon.
--------------	-------------------------------------

Returns

pAlku Palauttaa NULL, koska jono on tyhjä.

Definition at line 152 of file [haut.c](#).

```
00152                                     {
00153     // Jonon muistin vapauttaminen.
00154     JONO *ptr = pAlku;
00155     JONO *pSeuraava = NULL;
00156
00157     while (ptr != NULL) {
00158         pSeuraava = ptr->pSeuraava;
00159         free(ptr);
00160         ptr = pSeuraava;
00161     }
00162     pAlku = NULL;
00163     return (pAlku);
00164 }
```

3.7.2.6 varaaMuistiaJonolle()

```
JONO * varaaMuistiaJonolle (
    PUU * pSolmu)
```

Varaa muistia jonon uudelle alkiolle leveyshakua varten.

Parameters

<i>pSolmu</i>	Osoitin kasiteltavaan puun solmuun.
---------------	-------------------------------------

Returns

pUusi Osoitin uuteen jonon alkioon.

Definition at line 129 of file [haut.c](#).

```
00129                                     {
00130     JONO *pUusi = NULL;
00131
00132     // Muistin varaaminen jonolle.
00133     if ((pUusi = (JONO *)malloc(sizeof(JONO))) == NULL) {
00134         perror("Muistin varaus epäonnistui, lopetetaan");
00135         exit(0);
00136     }
00137
00138     pUusi->pSolmu = pSolmu;
00139     pUusi->pSeuraava = NULL;
00140
00141     return (pUusi);
00142 }
```

3.8 haut.h

[Go to the documentation of this file.](#)

```
00001 #ifndef HAUT_H
00002 #define HAUT_H
00003 #include "binaaripuu.h"
00004
00005 typedef struct jono {
00006     PUU *pSolmu;
00007     struct jono *pSeuraava;
00008 } JONO;
00009
00010 int syvyyshaku(char *pKirjoitaTiedostoNimi, PUU *pJuuriSolmu, int iArvo);
00011 void tarkistaLoytvykoSyvyysshaulla(char *aNimiKirjoitettava, PUU *pJuuriSolmu, int iArvo);
00012 void leveyshaku(char *pKirjoitaTiedostoNimi, PUU *pAlku, char *pHaku);
00013 JONO *varaamuistiaJonolle(PUU *pSolmu);
00014 JONO *vapautamuistiJono(JONO *pAlku);
00015 int binaarihaku(char *pNimi, PUU *pJuuriSolmu, int iArvo);
00016
00017 #endif
```

3.9 linkitettylista.c File Reference

```
#include "linkitettylista.h"
#include <ctype.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

Functions

- **TIEDOT * lue** (char *pNimi, **TIEDOT *pAlku**)
Lukee tiedoston.
- **TIEDOT * varaamuistia** (**TIEDOT *pAlku**, char *pNimi, int iMaara)
Varaa muistin ja lisää uuden solmun kaksisuuntaisen linkitetyn listan.
- **TIEDOT * vapautamuisti** (**TIEDOT *pAlku**)
Vapauttaa linkitetyn listan varaaman muistin.
- void **kirjoitaTiedostoAlustaLoppuun** (char *pNimi, **TIEDOT *pAlku**)
Kirjoittaa linkitetyn listan tiedostoon alusta loppuun.
- void **kirjoitaTiedostoLopustaAlkuun** (char *pNimi, **TIEDOT *pAlku**)
Kirjoittaa linkitetyn listan tiedostoon lopusta alkuun.
- **TIEDOT * lisaaListaan** (**TIEDOT *pAlku**, int iIndeksi, char *pNimi, int iArvo)
Lisää linkitettyyn listaan käyttäjän syöttämät tiedot.
- **TIEDOT * poistaListastaAlkio** (**TIEDOT *pAlku**, int iLuvuVaiNimi, char *pTieto)
Poistaa linkitetystä listasta alkion.
- **TIEDOT * poistaListastaLkmPerusteella** (**TIEDOT *pAlku**, int iLKM)
Poistetaan linkitetystä listasta alkio lukumäärän perusteella.
- **TIEDOT * poistaListastaNimenPerusteella** (**TIEDOT *pAlku**, char *pNimi)
Poistetaan linkitetystä listasta alkio nimen perusteella.
- int **useammallaAlkiollaSamaLKM** (**TIEDOT *pAlku**, int iLKM)
Etsii kaikki alkiot, joiden nimien lukumäärä on käyttäjän antama luku.
- int **onkoLukuVaiNimi** (char *Tieto)
Selvittää onko käyttäjän antama syöte luku vai nimi.
- int **laskeListanPituus** (**TIEDOT *pAlku**)

- Laskee linkitetyn listan pituuden.*
 • `TIEDOT * halkaise (TIEDOT *pAlku)`
Halkaisee linkitetyn listan kahteen puoliskoon.
- `TIEDOT * lomitut (TIEDOT *p1, TIEDOT *p2)`
Lomittaa (merge) kaksi lajiteltua listaa yhdeksi.
- `TIEDOT * lomitutLajittelu (TIEDOT *pAlku)`
Lajittelee listan rekursiivisella lomitutlajittelulla (Merge sort).
- `TIEDOT * lisaysLajittelu (TIEDOT *pAlku)`
Lajittelee listan lisayslajittelulla (Insertion sort).

3.9.1 Function Documentation

3.9.1.1 halkaise()

```
TIEDOT * halkaise (
    TIEDOT * pAlku)
```

Parameters

<i>pAlku</i>	Osoitin linkitetyn listan alkuun.
--------------	-----------------------------------

Returns

TIEDOT* Osoitin listan jälkimmäisen puoliskon alkuun.

Definition at line 449 of file [linkitettylista.c](#).

```
00449     {
00450         TIEDOT *p1 = NULL, *p2 = NULL;
00451         int i;
00452         int iPituus = 0;
00453         int iKeskipaikka = 0;
00454
00455         iPituus = laskeListanPituus(pAlku);
00456         iKeskipaikka = iPituus / 2 - 1;
00457
00458         p1 = pAlku;
00459         for (i = 0; i < iKeskipaikka; i++) {
00460             p1 = p1->pSeuraava;
00461         }
00462
00463         // Halkaistaan lista kahtia, p1 päättyy siihen mistä p2 alkaa
00464         p2 = p1->pSeuraava;
00465         p1->pSeuraava = NULL;
00466         if (p2 != NULL) {
00467             p2->pEdellinen = NULL;
00468         }
00469
00470         // palautetaan osoitin toisen listan alkuun
00471         return p2;
00472     }
```


3.9.1.2 kirjoitaTiedostoAlustaLoppuun()

```
void kirjoitaTiedostoAlustaLoppuun (
    char * pNimi,
    TIEDOT * pAlku)
```

Parameters

<i>pNimi</i>	Kirjoitettavan tiedoston nimi.
<i>pAlku</i>	Osoitin linkitetyn listan alkuun.

Returns

void

Definition at line 129 of file [linkitettylista.c](#).

```
00129                                     {
00130     FILE *Tiedosto = NULL;
00131     TIEDOT *ptr = NULL;
00132
00133     if (pAlku == NULL) {
00134         printf("Lista on tyhjä, lue tiedosto ennen kirjoittamista.\n");
00135         return;
00136     }
00137
00138     // Kirjoitus tiedoston avaaminen
00139     if ((Tiedosto = fopen(pNimi, "w")) == NULL) {
00140         perror("Tiedoston avaaminen epäonnistui, lopetetaan");
00141         exit(0);
00142     }
00143
00144     // Tiedostoon kirjoittaminen
00145     ptr = pAlku;
00146     while (ptr != NULL) {
00147         fprintf(Tiedosto, "%s,%d\n", ptr->aNimi, ptr->iYhteensa);
00148         ptr = ptr->pSeuraava;
00149     }
00150
00151     // Tiedoston sulkeminen
00152     fclose(Tiedosto);
00153     printf("Tiedosto %s kirjoitettu.\n", pNimi);
00154     return;
00155 }
```

3.9.1.3 kirjoitaTiedostoLopustaAlkuun()

```
void kirjoitaTiedostoLopustaAlkuun (
    char * pNimi,
    TIEDOT * pAlku)
```

Parameters

<i>pNimi</i>	Kirjoitettavan tiedoston nimi.
<i>pAlku</i>	Osoitin linkitetyn listan alkuun.

Returns

void

Definition at line 164 of file [linkitettylista.c](#).

```

00164                                     {
00165     FILE *Tiedosto = NULL;
00166     TIEDOT *ptr = pAlku;
00167
00168     if (pAlku == NULL) {
00169         printf("Lista on tyhjä, lue tiedosto ennen kirjoittamista.\n");
00170         return;
00171     }
00172
00173     if ((Tiedosto = fopen(pNimi, "w")) == NULL) {
00174         perror("Tiedoston avaaminen epäonnistui, lopetetaan");
00175         exit(0);
00176     }
00177
00178     // Mennään listan loppuun.
00179     while (ptr->pSeuraava != NULL) {
00180         ptr = ptr->pSeuraava;
00181     }
00182
00183     // Lopusta alkuun, kirjoitetaan tiedostoon.
00184     while (ptr != NULL) {
00185         fprintf(Tiedosto, "%s,%d\n", ptr->aNimi, ptr->iYhteensa);
00186         ptr = ptr->pEdellinen;
00187     }
00188
00189     fclose(Tiedosto);
00190     printf("Tiedosto %s kirjoitettu.\n", pNimi);
00191     return;
00192 }

```

3.9.1.4 laskeListanPituus()

```

int laskeListanPituus (
    TIEDOT * pAlku)

```

Parameters

<i>pAlku</i>	Osoitin linkitetyn listan alkuun.
--------------	-----------------------------------

Returns

int Linkitetyn listan alkioden lukumäärä.

Definition at line 434 of file [linkitettylista.c](#).

```

00434                                     {
00435     int i = 0;
00436     while (pAlku != NULL) {
00437         i++;
00438         pAlku = pAlku->pSeuraava;
00439     }
00440     return i;
00441 }

```

3.9.1.5 lisaaListaan()

```

TIEDOT * lisaaListaan (
    TIEDOT * pAlku,
    int iIndeksi,

```

```
char * pNimi,
int iArvo)
```

Lisaa käyttäjän haluamaan kohtaan (indeksi) tiedot lisattavasta nimestä ja niiden lukumaarasta.

Parameters

<i>pAlku</i>	Osoitin linkitetyn listan alkuun.
<i>iIndeksi</i>	Käyttäjän antama indeksi, eli kohta, johon tietue lisataan linkitettyssä listassa. Jos lista on tyhjä, lisataan automaattisesti ensimmäiseksi elementiksi. Jos indeksi on suurempi kuin listan elementtien määrä, lisataan automaattisesti viimeiseksi.
<i>pNimi</i>	Lisattava nimi.
<i>iArvo</i>	Lisattavan nimen lukumaara.

Returns

pAlku Osoitin listan nykyiseen alkuun.

Definition at line 207 of file [linkitettylista.c](#).

```
00207                                     {
00208     TIEDOT *pUusi = NULL;
00209     TIEDOT *ptr = pAlku;
00210     int i = 1;
00211
00212     // Varataan muistia uudelle linkitetyn listan solmulle.
00213     if ((pUusi = (TIEDOT *)malloc(sizeof(TIEDOT))) == NULL) {
00214         perror("Muistin varaus epäonnistui, lopetetaan");
00215         exit(0);
00216     }
00217
00218     // Lisataan tiedot tietueeseen.
00219     strcpy(pUusi->aNimi, pNimi);
00220     pUusi->iYhteensa = iArvo;
00221
00222     // Jos lista on tyhjä, lisataan uusi solmu ensimmäiseksi.
00223     if (pAlku == NULL) {
00224         pUusi->pEdellinen = NULL;
00225         pUusi->pSeuraava = NULL;
00226         pAlku = pUusi;
00227         return (pAlku);
00228     } else {
00229         // Jos lisataan listan alkuun, eli indeksi = 0.
00230         if (iIndeksi <= 1) {
00231             pUusi->pEdellinen = NULL;
00232             pUusi->pSeuraava = pAlku;
00233             pAlku->pEdellinen = pUusi;
00234             pAlku = pUusi;
00235             return (pAlku);
00236         }
00237
00238         // Etsitään listalta oikea kohta, johon tiedot lisataan.
00239         while (ptr->pSeuraava != NULL && i < (iIndeksi - 1)) {
00240             ptr = ptr->pSeuraava;
00241             i++;
00242         }
00243         // Lisataan tiedot oikeaan kohtaan linkitettyä listaa.
00244         // Paivitetaan osoittimet osoittamaan oikeisiin kohtiin.
00245         pUusi->pSeuraava = ptr->pSeuraava;
00246         pUusi->pEdellinen = ptr;
00247
00248         if (ptr->pSeuraava != NULL) {
00249             ptr->pSeuraava->pEdellinen = pUusi;
00250         }
00251
00252         ptr->pSeuraava = pUusi;
00253     }
00254     return (pAlku);
00255 }
```

3.9.1.6 lisaysLajittelu()

```
TIEDOT * lisaysLajittelu (
    TIEDOT * pAlku)
```

Parameters

<i>pAlku</i>	Osoitin lajiteltavan linkitetyn listan alkuun.
--------------	--

Returns

TIEDOT* Lajiteltu lista.

Definition at line 602 of file [linkitettylista.c](#).

```
00602                                     {
00603     // Jos lista on tyhjä
00604     if (pAlku == NULL) {
00605         return pAlku;
00606     }
00607
00608     TIEDOT *pLajittelematon = pAlku; // Osoitin lajiteltavan listan nykyiseen solmuun
00609     TIEDOT *pLajiteltu = NULL;       // Tehdään toinen lista, johon lajitellut tiedot laitetaan
00610     TIEDOT *pSeuraava = NULL;
00611     TIEDOT *ptr = NULL; // Osoitin, jolla läpikäydään lajiteltua listaa.
00612
00613     /* Käydään lajittelematonta listaa läpi ja lisätään alkiot
00614     * lajiteltuun listaan pienenevässä järjestyksessä arvон mukaan.
00615     */
00616     while (pLajittelematon != NULL) {
00617         pSeuraava = pLajittelematon->pSeuraava; // pSeuraava talteen ennen listan läpikäyntiä
00618
00619         // irrotetaan alkio
00620         if (pSeuraava != NULL) {
00621             pSeuraava->pEdellinen = pLajittelematon->pEdellinen;
00622         }
00623         if (pLajittelematon->pEdellinen != NULL) {
00624             pLajittelematon->pEdellinen->pSeuraava = pSeuraava;
00625         }
00626         pLajittelematon->pSeuraava = NULL;
00627         pLajittelematon->pEdellinen = NULL;
00628
00629         if (pLajiteltu == NULL || pLajiteltu->iYhteensa < pLajittelematon->iYhteensa ||
00630             (pLajiteltu->iYhteensa == pLajittelematon->iYhteensa &&
00631              strcmp(pLajiteltu->aNimi, pLajittelematon->aNimi) < 0)) {
00632             // Lisätään lajitellun listan alkuun
00633             pLajittelematon->pSeuraava = pLajiteltu;
00634             if (pLajiteltu != NULL) {
00635                 pLajiteltu->pEdellinen = pLajittelematon;
00636             }
00637             pLajiteltu = pLajittelematon;
00638         } else {
00639             ptr = pLajiteltu;
00640             while (ptr->pSeuraava != NULL &&
00641                 (ptr->pSeuraava->iYhteensa > pLajittelematon->iYhteensa ||
00642                  (ptr->pSeuraava->iYhteensa == pLajittelematon->iYhteensa &&
00643                   strcmp(ptr->pSeuraava->aNimi, pLajittelematon->aNimi) > 0))) {
00644                 ptr = ptr->pSeuraava;
00645             }
00646             pLajittelematon->pSeuraava = ptr->pSeuraava;
00647             if (ptr->pSeuraava != NULL) {
00648                 ptr->pSeuraava->pEdellinen = pLajittelematon;
00649             }
00650             ptr->pSeuraava = pLajittelematon;
00651             pLajittelematon->pEdellinen = ptr;
00652         }
00653         pLajittelematon = pSeuraava;
00654     }
00655     return pLajiteltu;
00656 }
00657 }
```

3.9.1.7 lomitus()

```
TIEDOT * lomitus (
    TIEDOT * p1,
    TIEDOT * p2)
```

Parameters

<i>p1</i>	Osoitin ensimmäisen lajitellun listan alkuun.
<i>p2</i>	Osoitin jälkimmäisen lajitellun listan alkuun.

Returns

TIEDOT* yhdistetty lajiteltu lista.

Definition at line 481 of file [linkitettylista.c](#).

```

00481                                     {
00482     // Jos kumpikaan lista on tyhjä, palautetaan toinen.
00483     if (p1 == NULL)
00484         return p2;
00485     if (p2 == NULL)
00486         return p1;
00487
00488     TIEDOT *p3 = NULL;           // Uuden listan alku.
00489     TIEDOT *p3Loppu = NULL;     // Uuden lista loppu, jotta loppuun lisääminen onnistuu helposti.
00490
00491     // Käydään molempia listoja läpi:
00492     while (p1 != NULL && p2 != NULL) {
00493         int iVertailu =
00494             strcmp(p1->aNimi, p2->aNimi); // Jos sama arvo, lisätään aakkosjärjestyksessä.
00495         if (p1->iYhteensa > p2->iYhteensa || (p1->iYhteensa == p2->iYhteensa && iVertailu > 0)) {
00496             TIEDOT *ptr = p2; // p2[0] talteen
00497             // Poistetaan p2[0] listasta.
00498             p2 = p2->pSeuraava;
00499             if (p2 != NULL) {
00500                 p2->pEdellinen = NULL;
00501             }
00502             // Lisätään p2[0] uuden listan loppuun.
00503             ptr->pSeuraava = NULL;
00504             ptr->pEdellinen = p3Loppu;
00505             if (p3Loppu != NULL) {
00506                 p3Loppu->pSeuraava = ptr;
00507             } else {
00508                 p3 = ptr; // ensimmäinen alkio, jos lista tyhjä
00509             }
00510             p3Loppu = ptr; // päivitetään loppu.
00511         } else {
00512             TIEDOT *ptr = p1; // p1[0] talteen
00513             p1 = p1->pSeuraava;
00514             // Poistetaan p1[0] listasta.
00515             if (p1 != NULL) {
00516                 p1->pEdellinen = NULL;
00517             }
00518             // Lisätään p1[0] uuden listan loppuun.
00519             ptr->pSeuraava = NULL;
00520             ptr->pEdellinen = p3Loppu;
00521             if (p3Loppu != NULL) {
00522                 p3Loppu->pSeuraava = ptr;
00523             } else {
00524                 p3 = ptr; // ensimmäinen alkio, jos lista tyhjä
00525             }
00526             p3Loppu = ptr; // päivitetään loppu.
00527         }
00528     }
00529
00530     // Tässä kohtaa joko p1 tai p2 on tyhjä.
00531
00532     // Lisätään loput p1:stä uuden listan loppuun.
00533     while (p1 != NULL) {
00534         TIEDOT *ptr = p1;
00535         p1 = p1->pSeuraava;
00536         if (p1 != NULL) {
00537             p1->pEdellinen = NULL;
00538         }
00539         ptr->pSeuraava = NULL;
00540         ptr->pEdellinen = p3Loppu;
00541         if (p3Loppu != NULL) {
00542             p3Loppu->pSeuraava = ptr;
00543         } else {
00544             p3 = ptr;
00545         }
00546         p3Loppu = ptr;
00547     }
00548
00549     // Lisätään loput p2:stä uuden listan loppuun.

```

```

00550     while (p2 != NULL) {
00551         TIEDOT *ptr = p2;
00552         p2 = p2->pSeuraava;
00553         if (p2 != NULL) {
00554             p2->pEdellinen = NULL;
00555         }
00556         ptr->pSeuraava = NULL;
00557         ptr->pEdellinen = p3Loppu;
00558         if (p3Loppu != NULL) {
00559             p3Loppu->pSeuraava = ptr;
00560         } else {
00561             p3 = ptr;
00562         }
00563         p3Loppu = ptr;
00564     }
00565     return p3;
00566 }

```

3.9.1.8 lomitusLajittelu()

```

TIEDOT * lomitusLajittelu (
    TIEDOT * pAlku)

```

Parameters

<i>pAlku</i>	Osoitin lajiteltavan linkitetyn listan alkuun.
--------------	--

Returns

TIEDOT* Lajiteltu lista.

Definition at line 574 of file [linkitettylista.c](#).

```

00574     {
00575         TIEDOT *p1 = NULL, *p2 = NULL;
00576
00577         // Jos lista on tyhjä tai sisältää vain yhden alkion
00578         if (pAlku == NULL || pAlku->pSeuraava == NULL) {
00579             return pAlku;
00580         }
00581
00582         // Jaetaan lista kahtia
00583         p2 = halkaise(pAlku);
00584         p1 = pAlku;
00585
00586         // Lajitellaan molemmat puolikkaat
00587         p1 = lomitusLajittelu(p1);
00588         p2 = lomitusLajittelu(p2);
00589
00590         // Yhdistetään lajitellut puolikkaat
00591         return lomitus(p1, p2);
00592     }

```

3.9.1.9 lue()

```

TIEDOT * lue (
    char * pNimi,
    TIEDOT * pAlku)

```

Avaa ja lukee käyttäjän syötteen mukaisen tiedoston. Kutsuu muistinvaraus aliohjelmaa muistin varaamiseksi.

Parameters

<i>pNimi</i>	Luettavan tiedoston nimi.
<i>pAlku</i>	Osoitin linkitetyn listan alkuun.

Returns

pAlku Uusi osoitin linkitetyn listan alkuun.

Definition at line 19 of file [linkitettylista.c](#).

```

00019                                     {
00020     FILE *Tiedosto = NULL;
00021     char aRivi[LEN] = "";
00022     char *p1 = NULL, *p2 = NULL;
00023     int iOtsikko = 0;
00024     int iMaara = 0;
00025
00026     /* Tiedoston avaaminen. */
00027
00028     if ((Tiedosto = fopen(pNimi, "r")) == NULL) {
00029         perror("Tiedoston avaaminen epäonnistui, lopetetaan");
00030         exit(0);
00031     }
00032
00033     /* Tiedoston lukeminen */
00034
00035     while ((fgets(aRivi, LEN, Tiedosto)) != NULL) {
00036
00037         aRivi[strlen(aRivi) - 1] = '\0';
00038
00039         if (iOtsikko == 0) {
00040             iOtsikko = 1;
00041             continue;
00042         }
00043
00044         if ((p1 = strtok(aRivi, ";")) == NULL) {
00045             perror("Tiedoston pilkkominen epäonnistui, lopetetaan");
00046             exit(0);
00047         }
00048
00049         if ((p2 = strtok(NULL, ";")) == NULL) {
00050             perror("Tiedoston pilkkominen epäonnistui, lopetetaan");
00051             exit(0);
00052         }
00053
00054         iMaara = atoi(p2);
00055
00056         /* Muistin varaaminen ja linkitetyn listan luominen. */
00057         pAlku = varaaMuistia(pAlku, p1, iMaara);
00058     }
00059     /* Tiedoston sulkeminen. */
00060     fclose(Tiedosto);
00061     printf("Tiedosto '%s' luettu.\n", pNimi);
00062     return (pAlku);
00063 }
```

3.9.1.10 onkoLukuVaiNimi()

```

int onkoLukuVaiNimi (
    char * Tieto)
```

Parameters

<i>Tieto</i>	Käyttäjän antama syöte.
--------------	-------------------------

Returns

int Tieto, onko luku vai nimi, jos -2, niin käyttäjä antoi luvun, jos joku muu niin se on luku.

Definition at line 403 of file [linkitettylista.c](#).

```

00403                                     {
00404     int iVastaus = 0;
00405     int i = 0;
00406     int iPituus = 0;
00407
00408     iPituus = strlen(Tieto);
00409
00410     for (i = 0; i < iPituus; i++) {
00411         if (isdigit(Tieto[i])) {
00412             iVastaus = -1;
00413         } else {
00414             iVastaus = -2;
00415             break;
00416         }
00417     }
00418
00419     if (iVastaus == -1) {
00420         iVastaus = atoi(Tieto);
00421     }
00422
00423     return (iVastaus);
00424 }
```

3.9.1.11 poistaListastaAlkio()

```

TIEDOT * poistaListastaAlkio (
    TIEDOT * pAlku,
    int iLuvuVaiNimi,
    char * pTieto)
```

Parameters

<i>pAlku</i>	Osoitin linkitetyn listan alkuun.
<i>iLuvuVaiNimi</i>	Tieto siitä, haluaako käyttäjä poistaa alkion nimen vai luvun perusteella.
<i>pTieto</i>	Poistettavan alkion tieto, nimi tai luku.

Returns

TIEDOT* pAlku Osoitin listan nykyiseen alkuun.

Definition at line 265 of file [linkitettylista.c](#).

```

00265                                     {
00266     int iSamojenAlkiodenLkm = 0;
00267     char aPoistettavaNimi[LEN] = "";
00268
00269     if (iLuvuVaiNimi == -2) {
00270         pAlku = poistaListastaNimenPerusteella(pAlku, pTieto);
00271     } else {
00272         iSamojenAlkiodenLkm = useamallaAlkiollaSamaLKM(pAlku, iLuvuVaiNimi);
00273
00274         if (iSamojenAlkiodenLkm > 1) {
00275             kysyNimi("Anna nimi, joka poistetaan: ", aPoistettavaNimi);
00276             pAlku = poistaListastaNimenPerusteella(pAlku, aPoistettavaNimi);
00277         } else {
00278             pAlku = poistaListastaLkmPerusteella(pAlku, iLuvuVaiNimi);
00279         }
00280     }
00281
00282     return (pAlku);
00283 }
```


3.9.1.12 poistaListastaLkmPeruusteella()

```
TIEDOT * poistaListastaLkmPeruusteella (
    TIEDOT * pAlku,
    int iLKM)
```

Käyttäjä on antanut lukumäärän. Alkio, jossa on tämä lukumäärä poistetaan.

Parameters

<i>pAlku</i>	Osoitin linkitetyn listan alkuun.
<i>iLKM</i>	Käyttäjän antama lukumäärä. Alkio, jossa on tämä lukumäärä poistetaan.

Returns

TIEDOT* Palautetaan linkitettylista, josta on poistettu alkio.

Definition at line 292 of file [linkitettylista.c](#).

```
00292                                     {
00293     TIEDOT *ptr = pAlku;
00294     TIEDOT *pEdellinen = NULL;
00295     int iAlkioLoytyi = 0;
00296
00297     // Tarkistetaan, onko poistettava alkio ensimmäinen.
00298     if ((ptr != NULL) && (ptr->iYhteensa == iLKM)) {
00299         // Sijoitetaan osoitin osoittamaan seuraavaan alkioon.
00300         pAlku = ptr->pSeuraava;
00301         free(ptr);
00302         printf("Alkio poistettu.\n");
00303
00304         // Jos poistettava alkio ei ole ensimmäinen.
00305     } else {
00306         // Etsitään kohtaa, jossa poistettava alkio on.
00307         while (ptr != NULL) {
00308             if (ptr->iYhteensa == iLKM) {
00309                 pEdellinen->pSeuraava = ptr->pSeuraava;
00310                 free(ptr);
00311                 iAlkioLoytyi = 1;
00312                 break;
00313             }
00314             pEdellinen = ptr;
00315             ptr = ptr->pSeuraava;
00316         }
00317
00318         // Kerrotaan käyttäjälle, poistettiin alkio.
00319         if (iAlkioLoytyi == 1) {
00320             printf("Alkio poistettu.\n");
00321         } else {
00322             printf("Poistettavaa alkiota ei löydetty.\n");
00323         }
00324     }
00325
00326     return (pAlku);
00327 }
```

3.9.1.13 poistaListastaNimenPerusteella()

```
TIEDOT * poistaListastaNimenPerusteella (
    TIEDOT * pAlku,
    char * pNimi)
```

Käyttäjä on antanut nimen. Alkio, jossa on tämä nimi poistetaan.

Parameters

<i>pAlku</i>	Osoitin linkitetyn listan alkuun.
<i>pNimi</i>	Käyttäjän antama nimi. Alkio, jossa on tämä nimi poistetaan.

Returns

TIEDOT* TIEDOT* Palautetaan linkitettylista, josta on poistettu alkio.

Definition at line 336 of file [linkitettylista.c](#).

```

00336                                     {
00337     TIEDOT *ptr = pAlku;
00338     TIEDOT *pEdellinen = NULL;
00339     int iAlkioLoytyi = 0;
00340
00341     // Tarkistetaan, onko poistettava alkio ensimmäinen.
00342     if ((ptr != NULL) && (strcmp(ptr->aNimi, pNimi) == 0)) {
00343         // Sijoitetaan osoitin osoittamaan seuraavaan alkioon.
00344         pAlku = ptr->pSeuraava;
00345         free(ptr);
00346         printf("Alkio poistettu.\n");
00347
00348         // Jos poistettava alkio ei ole ensimmäinen.
00349     } else {
00350         // Etsitään kohtaa, jossa poistettava alkio on.
00351         while (ptr != NULL) {
00352             if (strcmp(ptr->aNimi, pNimi) == 0) {
00353                 pEdellinen->pSeuraava = ptr->pSeuraava;
00354                 free(ptr);
00355                 iAlkioLoytyi = 1;
00356                 break;
00357             }
00358             pEdellinen = ptr;
00359             ptr = ptr->pSeuraava;
00360         }
00361
00362         // Kerrotaan käyttäjälle, poistettiin alkio.
00363         if (iAlkioLoytyi == 1) {
00364             printf("Alkio poistettu.\n");
00365         } else {
00366             printf("Poistettavaa alkiota ei löydetty.\n");
00367         }
00368     }
00369     return (pAlku);
00370 }
```

3.9.1.14 useammallaAlkiollaSamaLKM()

```

int useammallaAlkiollaSamaLKM (
    TIEDOT * pAlku,
    int iLKM)
```

Parameters

<i>pAlku</i>	Osoitin linkitetyn listan alkuun.
<i>iLKM</i>	Käyttäjän antama lukumäärä, joiden määrä selvitetään.

Returns

int Palauttaa luvun, jossa on kaikki alkiot, joissa on käyttäjän antama luku.

Definition at line 379 of file [linkitettylista.c](#).

```
00379                                     {
00380     int iSamoja = 0;
00381     TIEDOT *ptr = NULL;
00382     ptr = pAlku;
00383
00384     // Käydään linkitettylista läpi
00385     while (ptr != NULL) {
00386         // Lisätään summaan yksi, jos linkitetyn listan alkion lukumäärä on sama kuin käyttäjän
00387         // antama luku.
00388         if (ptr->iYhteensa == iLKM) {
00389             iSamoja++;
00390         }
00391         ptr = ptr->pSeuraava;
00392     }
00393
00394     return (iSamoja);
00395 }
```

3.9.1.15 vapautaMuisti()

```
TIEDOT * vapautaMuisti (
    TIEDOT * pAlku)
```

Parameters

<i>pAlku</i>	Osoitin linkitetyn listan alkuun.
--------------	-----------------------------------

Returns

pAlku Osoitin, joka on NULL, koska lista on tyhjä.

Definition at line 110 of file [linkitettylista.c](#).

```
00110                                     {
00111     /* Muistin vapauttaminen. */
00112     TIEDOT *ptr = pAlku;
00113     while (ptr != NULL) {
00114         pAlku = ptr->pSeuraava;
00115         free(ptr);
00116         ptr = pAlku;
00117     }
00118     pAlku = NULL;
00119     return (pAlku);
00120 }
```

3.9.1.16 varaaMuistia()

```
TIEDOT * varaaMuistia (
    TIEDOT * pAlku,
    char * pNimi,
    int iMaara)
```

Parameters

<i>pAlku</i>	Osoitin linkitetyn listan alkuun.
--------------	-----------------------------------

<i>pNimi</i>	Merkkijono solmun nimeksi.
<i>iMaara</i>	Luku, joka asetetaan solmuun maaraksi.

Returns

pAlku Uusi osoitin linkitetyn listan alkuun.

Definition at line 73 of file [linkitettylista.c](#).

```

00073                                     {
00074     TIEDOT *pUusi = NULL;
00075     TIEDOT *ptr = NULL;
00076
00077     /* Muistin varaaminen. */
00078
00079     if ((pUusi = (TIEDOT *)malloc(sizeof(TIEDOT))) == NULL) {
00080         perror("Muistin varaus epäonnistui, lopetetaan");
00081         exit(0);
00082     }
00083
00084     /* Kaksisuuntaisen linkitetyn listan luominen. */
00085
00086     strcpy(pUusi->aNimi, pNimi);
00087     pUusi->iYhteensa = iMaara;
00088     pUusi->pSeuraava = NULL;
00089
00090     if (pAlku == NULL) {
00091         pUusi->pEdellinen = NULL;
00092         pAlku = pUusi;
00093     } else {
00094         ptr = pAlku;
00095         while (ptr->pSeuraava != NULL) {
00096             ptr = ptr->pSeuraava;
00097         }
00098         ptr->pSeuraava = pUusi;
00099         pUusi->pEdellinen = ptr;
00100     }
00101     return (pAlku);
00102 }
```

3.10 linkitettylista.c

[Go to the documentation of this file.](#)

```

00001 #include "linkitettylista.h"
00002 #include <ctype.h>
00003 #include <stdio.h>
00004 #include <stdlib.h>
00005 #include <string.h>
00006
00007 // Linkitettylista
00008
00019 TIEDOT *lue(char *pNimi, TIEDOT *pAlku) {
00020     FILE *Tiedosto = NULL;
00021     char aRivi[LEN] = "";
00022     char *p1 = NULL, *p2 = NULL;
00023     int iOtsikko = 0;
00024     int iMaara = 0;
00025
00026     /* Tiedoston avaaminen. */
00027
00028     if ((Tiedosto = fopen(pNimi, "r")) == NULL) {
00029         perror("Tiedoston avaaminen epäonnistui, lopetetaan");
00030         exit(0);
00031     }
00032
00033     /* Tiedoston lukeminen */
00034
00035     while ((fgets(aRivi, LEN, Tiedosto)) != NULL) {
00036
00037         aRivi[strlen(aRivi) - 1] = '\0';
00038
00039         if (iOtsikko == 0) {
00040             iOtsikko = 1;
```

```

00041         continue;
00042     }
00043
00044     if ((p1 = strtok(aRivi, ";")) == NULL) {
00045         perror("Tiedoston pilkkominen epäonnistui, lopetetaan");
00046         exit(0);
00047     }
00048
00049     if ((p2 = strtok(NULL, ";")) == NULL) {
00050         perror("Tiedoston pilkkominen epäonnistui, lopetetaan");
00051         exit(0);
00052     }
00053
00054     iMaara = atoi(p2);
00055
00056     /* Muistin varaaminen ja linkitetyn listan luominen. */
00057     pAlku = varaaMuistia(pAlku, p1, iMaara);
00058 }
00059 /* Tiedoston sulkeminen. */
00060 fclose(Tiedosto);
00061 printf("Tiedosto '%s' luettu.\n", pNimi);
00062 return (pAlku);
00063 }
00064
00073 TIEDOT *varaaMuistia(TIEDOT *pAlku, char *pNimi, int iMaara) {
00074     TIEDOT *pUusi = NULL;
00075     TIEDOT *ptr = NULL;
00076
00077     /* Muistin varaaminen. */
00078
00079     if ((pUusi = (TIEDOT *)malloc(sizeof(TIEDOT))) == NULL) {
00080         perror("Muistin varaus epäonnistui, lopetetaan");
00081         exit(0);
00082     }
00083
00084     /* Kaksisuuntaisen linkitetyn listan luominen. */
00085
00086     strcpy(pUusi->aNimi, pNimi);
00087     pUusi->iYhteensa = iMaara;
00088     pUusi->pSeuraava = NULL;
00089
00090     if (pAlku == NULL) {
00091         pUusi->pEdellinen = NULL;
00092         pAlku = pUusi;
00093     } else {
00094         ptr = pAlku;
00095         while (ptr->pSeuraava != NULL) {
00096             ptr = ptr->pSeuraava;
00097         }
00098         ptr->pSeuraava = pUusi;
00099         pUusi->pEdellinen = ptr;
00100     }
00101     return (pAlku);
00102 }
00103
00110 TIEDOT *vapautaMuisti(TIEDOT *pAlku) {
00111     /* Muistin vapauttaminen. */
00112     TIEDOT *ptr = pAlku;
00113     while (ptr != NULL) {
00114         pAlku = ptr->pSeuraava;
00115         free(ptr);
00116         ptr = pAlku;
00117     }
00118     pAlku = NULL;
00119     return (pAlku);
00120 }
00121
00129 void kirjoitaTiedostoAlustaLoppuun(char *pNimi, TIEDOT *pAlku) {
00130     FILE *Tiedosto = NULL;
00131     TIEDOT *ptr = NULL;
00132
00133     if (pAlku == NULL) {
00134         printf("Lista on tyhjä, lue tiedosto ennen kirjoittamista.\n");
00135         return;
00136     }
00137
00138     // Kirjoitus tiedoston avaaminen
00139     if ((Tiedosto = fopen(pNimi, "w")) == NULL) {
00140         perror("Tiedoston avaaminen epäonnistui, lopetetaan");
00141         exit(0);
00142     }
00143
00144     // Tiedostoon kirjoittaminen
00145     ptr = pAlku;
00146     while (ptr != NULL) {
00147         fprintf(Tiedosto, "%s,%d\n", ptr->aNimi, ptr->iYhteensa);
00148         ptr = ptr->pSeuraava;

```

```

00149     }
00150
00151     // Tiedoston sulkeminen
00152     fclose(Tiedosto);
00153     printf("Tiedosto %s kirjoitettu.\n", pNimi);
00154     return;
00155 }
00156
00164 void kirjoitaTiedostoLopustaAlkuun(char *pNimi, TIEDOT *pAlku) {
00165     FILE *Tiedosto = NULL;
00166     TIEDOT *ptr = pAlku;
00167
00168     if (pAlku == NULL) {
00169         printf("Lista on tyhjä, lue tiedosto ennen kirjoittamista.\n");
00170         return;
00171     }
00172
00173     if ((Tiedosto = fopen(pNimi, "w")) == NULL) {
00174         perror("Tiedoston avaaminen epäonnistui, lopetetaan");
00175         exit(0);
00176     }
00177
00178     // Mennään listan loppuun.
00179     while (ptr->pSeuraava != NULL) {
00180         ptr = ptr->pSeuraava;
00181     }
00182
00183     // Lopusta alkuun, kirjoitetaan tiedostoon.
00184     while (ptr != NULL) {
00185         fprintf(Tiedosto, "%s,%d\n", ptr->aNimi, ptr->iYhteensa);
00186         ptr = ptr->pEdellinen;
00187     }
00188
00189     fclose(Tiedosto);
00190     printf("Tiedosto %s kirjoitettu.\n", pNimi);
00191     return;
00192 }
00193
00207 TIEDOT *lisaaListaan(TIEDOT *pAlku, int iIndeksi, char *pNimi, int iArvo) {
00208     TIEDOT *pUusi = NULL;
00209     TIEDOT *ptr = pAlku;
00210     int i = 1;
00211
00212     // Varataan muistia uudelle linkitetyn listan solmulle.
00213     if ((pUusi = (TIEDOT *)malloc(sizeof(TIEDOT))) == NULL) {
00214         perror("Muistin varaus epäonnistui, lopetetaan");
00215         exit(0);
00216     }
00217
00218     // Lisataan tiedot tietueeseen.
00219     strcpy(pUusi->aNimi, pNimi);
00220     pUusi->iYhteensa = iArvo;
00221
00222     // Jos lista on tyhjä, lisataan uusi solmu ensimmäiseksi.
00223     if (pAlku == NULL) {
00224         pUusi->pEdellinen = NULL;
00225         pUusi->pSeuraava = NULL;
00226         pAlku = pUusi;
00227         return (pAlku);
00228     } else {
00229         // Jos lisataan listan alkuun, eli indeksi = 0.
00230         if (iIndeksi <= 1) {
00231             pUusi->pEdellinen = NULL;
00232             pUusi->pSeuraava = pAlku;
00233             pAlku->pEdellinen = pUusi;
00234             pAlku = pUusi;
00235             return (pAlku);
00236         }
00237
00238         // Etsitään listalta oikea kohta, johon tiedot lisataan.
00239         while (ptr->pSeuraava != NULL && i < (iIndeksi - 1)) {
00240             ptr = ptr->pSeuraava;
00241             i++;
00242         }
00243         // Lisataan tiedot oikeaan kohtaan linkitettyä listaa.
00244         // Paivitetään osoittimet osoittamaan oikeisiin kohtiin.
00245         pUusi->pSeuraava = ptr->pSeuraava;
00246         pUusi->pEdellinen = ptr;
00247
00248         if (ptr->pSeuraava != NULL) {
00249             ptr->pSeuraava->pEdellinen = pUusi;
00250         }
00251
00252         ptr->pSeuraava = pUusi;
00253     }
00254     return (pAlku);
00255 }

```

```

00256
00265 TIEDOT *poistaListastaAlkio(TIEDOT *pAlku, int iLuvuVaiNimi, char *pTieto) {
00266     int iSamojenAlkioidenLkm = 0;
00267     char aPoistettavaNimi[LEN] = "";
00268
00269     if (iLuvuVaiNimi == -2) {
00270         pAlku = poistaListastaNimenPerusteella(pAlku, pTieto);
00271     } else {
00272         iSamojenAlkioidenLkm = useammallaAlkiollaSamaLKM(pAlku, iLuvuVaiNimi);
00273
00274         if (iSamojenAlkioidenLkm > 1) {
00275             kysyNimi("Anna nimi, joka poistetaan: ", aPoistettavaNimi);
00276             pAlku = poistaListastaNimenPerusteella(pAlku, aPoistettavaNimi);
00277         } else {
00278             pAlku = poistaListastaLkmPeruusteella(pAlku, iLuvuVaiNimi);
00279         }
00280     }
00281
00282     return (pAlku);
00283 }
00284
00292 TIEDOT *poistaListastaLkmPeruusteella(TIEDOT *pAlku, int iLKM) {
00293     TIEDOT *ptr = pAlku;
00294     TIEDOT *pEdellinen = NULL;
00295     int iAlkioLoytyi = 0;
00296
00297     // Tarkistetaan, onko poistettava alkio ensimmäinen.
00298     if ((ptr != NULL) && (ptr->iYhteensa == iLKM)) {
00299         // Sijoitetaan osoitin osoittamaan seuraavaan alkioon.
00300         pAlku = ptr->pSeuraava;
00301         free(ptr);
00302         printf("Alkio poistettu.\n");
00303
00304         // Jos poistettava alkio ei ole ensimmäinen.
00305     } else {
00306         // Etsitään kohtaa, jossa poistettava alkio on.
00307         while (ptr != NULL) {
00308             if (ptr->iYhteensa == iLKM) {
00309                 pEdellinen->pSeuraava = ptr->pSeuraava;
00310                 free(ptr);
00311                 iAlkioLoytyi = 1;
00312                 break;
00313             }
00314             pEdellinen = ptr;
00315             ptr = ptr->pSeuraava;
00316         }
00317
00318         // Kerrotaan käyttäjälle, poistettiin alkio.
00319         if (iAlkioLoytyi == 1) {
00320             printf("Alkio poistettu.\n");
00321         } else {
00322             printf("Poistettavaa alkioa ei löydetty.\n");
00323         }
00324     }
00325
00326     return (pAlku);
00327 }
00328
00336 TIEDOT *poistaListastaNimenPerusteella(TIEDOT *pAlku, char *pNimi) {
00337     TIEDOT *ptr = pAlku;
00338     TIEDOT *pEdellinen = NULL;
00339     int iAlkioLoytyi = 0;
00340
00341     // Tarkistetaan, onko poistettava alkio ensimmäinen.
00342     if ((ptr != NULL) && (strcmp(ptr->aNimi, pNimi) == 0)) {
00343         // Sijoitetaan osoitin osoittamaan seuraavaan alkioon.
00344         pAlku = ptr->pSeuraava;
00345         free(ptr);
00346         printf("Alkio poistettu.\n");
00347
00348         // Jos poistettava alkio ei ole ensimmäinen.
00349     } else {
00350         // Etsitään kohtaa, jossa poistettava alkio on.
00351         while (ptr != NULL) {
00352             if (strcmp(ptr->aNimi, pNimi) == 0) {
00353                 pEdellinen->pSeuraava = ptr->pSeuraava;
00354                 free(ptr);
00355                 iAlkioLoytyi = 1;
00356                 break;
00357             }
00358             pEdellinen = ptr;
00359             ptr = ptr->pSeuraava;
00360         }
00361
00362         // Kerrotaan käyttäjälle, poistettiin alkio.
00363         if (iAlkioLoytyi == 1) {
00364             printf("Alkio poistettu.\n");

```

```

00365         } else {
00366             printf("Poistettavaa alkiota ei löydetty.\n");
00367         }
00368     }
00369     return (pAlku);
00370 }
00371
00379 int useammallaAlkiollaSamaLKM(TIEDOT *pAlku, int iLKM) {
00380     int iSamoja = 0;
00381     TIEDOT *ptr = NULL;
00382     ptr = pAlku;
00383
00384     // Käydään linkitettylistaa läpi
00385     while (ptr != NULL) {
00386         // Lisätään summaan yksi, jos linkitetyn listan alkion lukumäärä on sama kuin käyttäjän
00387         // antama luku.
00388         if (ptr->iYhteensa == iLKM) {
00389             iSamoja++;
00390         }
00391         ptr = ptr->pSeuraava;
00392     }
00393
00394     return (iSamoja);
00395 }
00396
00403 int onkoLukuVaiNimi(char *Tieto) {
00404     int iVastaus = 0;
00405     int i = 0;
00406     int iPituus = 0;
00407
00408     iPituus = strlen(Tieto);
00409
00410     for (i = 0; i < iPituus; i++) {
00411         if (isdigit(Tieto[i])) {
00412             iVastaus = -1;
00413         } else {
00414             iVastaus = -2;
00415             break;
00416         }
00417     }
00418
00419     if (iVastaus == -1) {
00420         iVastaus = atoi(Tieto);
00421     }
00422
00423     return (iVastaus);
00424 }
00425
00426 // L11 / Lomituslajittelu (merge sort)
00427
00434 int laskeListanPituus(TIEDOT *pAlku) {
00435     int i = 0;
00436     while (pAlku != NULL) {
00437         i++;
00438         pAlku = pAlku->pSeuraava;
00439     }
00440     return i;
00441 }
00442
00449 TIEDOT *halkaise(TIEDOT *pAlku) {
00450     TIEDOT *p1 = NULL, *p2 = NULL;
00451     int i;
00452     int iPituus = 0;
00453     int iKeskipiste = 0;
00454
00455     iPituus = laskeListanPituus(pAlku);
00456     iKeskipiste = iPituus / 2 - 1;
00457
00458     p1 = pAlku;
00459     for (i = 0; i < iKeskipiste; i++) {
00460         p1 = p1->pSeuraava;
00461     }
00462
00463     // Halkaistaan lista kahtia, p1 päättyy siihen mistä p2 alkaa
00464     p2 = p1->pSeuraava;
00465     p1->pSeuraava = NULL;
00466     if (p2 != NULL) {
00467         p2->pEdellinen = NULL;
00468     }
00469
00470     // palautetaan osoitin toisen listan alkuun
00471     return p2;
00472 }
00473
00481 TIEDOT *lomitus(TIEDOT *p1, TIEDOT *p2) {
00482     // Jos kumpikaan lista on tyhjä, palautetaan toinen.
00483     if (p1 == NULL)

```



```

00484     return p2;
00485 if (p2 == NULL)
00486     return p1;
00487
00488 TIEDOT *p3 = NULL; // Uuden listan alku.
00489 TIEDOT *p3Loppu = NULL; // Uuden lista loppu, jotta loppuun lisääminen onnistuu helposti.
00490
00491 // Käydään molempia listoja läpi:
00492 while (p1 != NULL && p2 != NULL) {
00493     int iVertailu =
00494         strcmp(p1->aNimi, p2->aNimi); // Jos sama arvo, lisäämään aakkosjärjestyksessä.
00495     if (p1->iYhteensa > p2->iYhteensa || (p1->iYhteensa == p2->iYhteensa && iVertailu > 0)) {
00496         TIEDOT *ptr = p2; // p2[0] talteen
00497         // Poistetaan p2[0] listasta.
00498         p2 = p2->pSeuraava;
00499         if (p2 != NULL) {
00500             p2->pEdellinen = NULL;
00501         }
00502         // Lisätään p2[0] uuden listan loppuun.
00503         ptr->pSeuraava = NULL;
00504         ptr->pEdellinen = p3Loppu;
00505         if (p3Loppu != NULL) {
00506             p3Loppu->pSeuraava = ptr;
00507         } else {
00508             p3 = ptr; // ensimmäinen alkio, jos lista tyhjä
00509         }
00510         p3Loppu = ptr; // päivitetään loppu.
00511     } else {
00512         TIEDOT *ptr = p1; // p1[0] talteen
00513         p1 = p1->pSeuraava;
00514         // Poistetaan p1[0] listasta.
00515         if (p1 != NULL) {
00516             p1->pEdellinen = NULL;
00517         }
00518         // Lisätään p1[0] uuden listan loppuun.
00519         ptr->pSeuraava = NULL;
00520         ptr->pEdellinen = p3Loppu;
00521         if (p3Loppu != NULL) {
00522             p3Loppu->pSeuraava = ptr;
00523         } else {
00524             p3 = ptr; // ensimmäinen alkio, jos lista tyhjä
00525         }
00526         p3Loppu = ptr; // päivitetään loppu.
00527     }
00528 }
00529
00530 // Tässä kohtaa joko p1 tai p2 on tyhjä.
00531
00532 // Lisätään loput p1:stä uuden listan loppuun.
00533 while (p1 != NULL) {
00534     TIEDOT *ptr = p1;
00535     p1 = p1->pSeuraava;
00536     if (p1 != NULL) {
00537         p1->pEdellinen = NULL;
00538     }
00539     ptr->pSeuraava = NULL;
00540     ptr->pEdellinen = p3Loppu;
00541     if (p3Loppu != NULL) {
00542         p3Loppu->pSeuraava = ptr;
00543     } else {
00544         p3 = ptr;
00545     }
00546     p3Loppu = ptr;
00547 }
00548
00549 // Lisätään loput p2:stä uuden listan loppuun.
00550 while (p2 != NULL) {
00551     TIEDOT *ptr = p2;
00552     p2 = p2->pSeuraava;
00553     if (p2 != NULL) {
00554         p2->pEdellinen = NULL;
00555     }
00556     ptr->pSeuraava = NULL;
00557     ptr->pEdellinen = p3Loppu;
00558     if (p3Loppu != NULL) {
00559         p3Loppu->pSeuraava = ptr;
00560     } else {
00561         p3 = ptr;
00562     }
00563     p3Loppu = ptr;
00564 }
00565 return p3;
00566 }
00567
00574 TIEDOT *lomitusLajittelu(TIEDOT *pAlku) {
00575     TIEDOT *p1 = NULL, *p2 = NULL;
00576

```

```

00577 // Jos lista on tyhjä tai sisältää vain yhden alkion
00578 if (pAlku == NULL || pAlku->pSeuraava == NULL) {
00579     return pAlku;
00580 }
00581
00582 // Jaetaan lista kahtia
00583 p2 = halkaise(pAlku);
00584 p1 = pAlku;
00585
00586 // Lajitellaan molemmat puolikkaat
00587 p1 = lomitusLajittelu(p1);
00588 p2 = lomitusLajittelu(p2);
00589
00590 // Yhdistetään lajitellut puolikkaat
00591 return lomitus(p1, p2);
00592 }
00593
00594 // Lisäyslajittelu (Insertion sort)
00595
00602 TIEDOT *lisaysLajittelu(TIEDOT *pAlku) {
00603     // Jos lista on tyhjä
00604     if (pAlku == NULL) {
00605         return pAlku;
00606     }
00607
00608     TIEDOT *pLajittelematon = pAlku; // Osoitin lajiteltavan listan nykyiseen solmuun
00609     TIEDOT *pLajiteltu = NULL;       // Tehdään toinen lista, johon lajitellut tiedot laitetaan
00610     TIEDOT *pSeuraava = NULL;
00611     TIEDOT *ptr = NULL; // Osoitin, jolla läpikäydään lajiteltua listaa.
00612
00613     /* Käydään lajittelematonta listaa läpi ja lisätään alkiot
00614     * lajiteltuun listaan pienenevässä järjestyksessä arvon mukaan.
00615     */
00616     while (pLajittelematon != NULL) {
00617         pSeuraava = pLajittelematon->pSeuraava; // pSeuraava talteen ennen listan läpikäyntiä
00618
00619         // irrotetaan alkio
00620         if (pSeuraava != NULL) {
00621             pSeuraava->pEdellinen = pLajittelematon->pEdellinen;
00622         }
00623         if (pLajittelematon->pEdellinen != NULL) {
00624             pLajittelematon->pEdellinen->pSeuraava = pSeuraava;
00625         }
00626         pLajittelematon->pSeuraava = NULL;
00627         pLajittelematon->pEdellinen = NULL;
00628
00629         if (pLajiteltu == NULL || pLajiteltu->iYhteensa < pLajittelematon->iYhteensa ||
00630             (pLajiteltu->iYhteensa == pLajittelematon->iYhteensa &&
00631              strcmp(pLajiteltu->aNimi, pLajittelematon->aNimi) < 0)) {
00632             // Lisätään lajitellun listan alkuun
00633             pLajittelematon->pSeuraava = pLajiteltu;
00634             if (pLajiteltu != NULL) {
00635                 pLajiteltu->pEdellinen = pLajittelematon;
00636             }
00637             pLajiteltu = pLajittelematon;
00638
00639         } else {
00640             ptr = pLajiteltu;
00641             while (ptr->pSeuraava != NULL &&
00642                 (ptr->pSeuraava->iYhteensa > pLajittelematon->iYhteensa ||
00643                  (ptr->pSeuraava->iYhteensa == pLajittelematon->iYhteensa &&
00644                   strcmp(ptr->pSeuraava->aNimi, pLajittelematon->aNimi) > 0))) {
00645                 ptr = ptr->pSeuraava;
00646             }
00647             pLajittelematon->pSeuraava = ptr->pSeuraava;
00648             if (ptr->pSeuraava != NULL) {
00649                 ptr->pSeuraava->pEdellinen = pLajittelematon;
00650             }
00651             ptr->pSeuraava = pLajittelematon;
00652             pLajittelematon->pEdellinen = ptr;
00653         }
00654         pLajittelematon = pSeuraava;
00655     }
00656     return pLajiteltu;
00657 }

```

3.11 linkitettylista.h File Reference

```
#include "yleiset.h"
```

Classes

- struct [tiedot](#)

Typedefs

- typedef struct [tiedot](#) [TIEDOT](#)

Functions

- [TIEDOT *](#) [lue](#) (char *pNimi, [TIEDOT *](#)pAlku)
Lukee tiedoston.
- [TIEDOT *](#) [varaaMuistia](#) ([TIEDOT *](#)pAlku, char *pSukunimi, int iMaara)
Varaa muistin ja lisää uuden solmun kaksisuuntaisen linkitetyn listan.
- [TIEDOT *](#) [vapautaMuisti](#) ([TIEDOT *](#)pAlku)
Vapauttaa linkitetyn listan varaaman muistin.
- void [kirjoitaTiedostoAlustaLoppuun](#) (char *pKirjoitaTiedostoNimi, [TIEDOT *](#)pAlku)
Kirjoittaa linkitetyn listan tiedostoon alusta loppuun.
- void [kirjoitaTiedostoLopustaAlkuun](#) (char *pKirjoitaTiedostoNimi, [TIEDOT *](#)pAlku)
Kirjoittaa linkitetyn listan tiedostoon lopusta alkuun.
- [TIEDOT *](#) [lisaaListaan](#) ([TIEDOT *](#)pAlku, int iIndeksi, char *pNimi, int iArvo)
Lisää linkitettyyn listaan käyttäjän syöttämat tiedot.
- [TIEDOT *](#) [poistaListastaLkmPeruusteella](#) ([TIEDOT *](#)pAlku, int iLKM)
Poistetaan linkitetystä listasta alkio lukumäärän perusteella.
- int [useammallaAlkiollaSamaLKM](#) ([TIEDOT *](#)pAlku, int iLKM)
Etsii kaikki alkio, joiden nimien lukumäärä on käyttäjän antama luku.
- [TIEDOT *](#) [poistaListastaNimenPerusteella](#) ([TIEDOT *](#)pAlku, char *pNimi)
Poistetaan linkitetystä listasta alkio nimen perusteella.
- int [laskeListanPituus](#) ([TIEDOT *](#)pAlku)
Laskee linkitetyn listan pituuden.
- [TIEDOT *](#) [halkaise](#) ([TIEDOT *](#)pAlku)
Halkaisee linkitetyn listan kahteen puoliskoon.
- [TIEDOT *](#) [lomitua](#) ([TIEDOT *](#)p1, [TIEDOT *](#)p2)
Lomittaa (merge) kaksi lajiteltua listaa yhdeksi.
- [TIEDOT *](#) [lomituaLajittelu](#) ([TIEDOT *](#)pAlku)
Lajittelee listan rekursiivisella lomitustajittelulla (Merge sort).
- [TIEDOT *](#) [lisaysLajittelu](#) ([TIEDOT *](#)pAlku)
Lajittelee listan lisayslajittelulla (Insertion sort).
- int [onkoLukuVaiNimi](#) (char *Tieto)
Selvittää onko käyttäjän antama syöte luku vai nimi.
- [TIEDOT *](#) [poistaListastaAlkio](#) ([TIEDOT *](#)pAlku, int iLuvuVaiNimi, char *pTieto)
Poistaa linkitetystä listasta alkion.

3.11.1 Typedef Documentation

3.11.1.1 TIEDOT

```
typedef struct tiedot TIEDOT
```

3.11.2 Function Documentation

3.11.2.1 halkaise()

```
TIEDOT * halkaise (
    TIEDOT * pAlku)
```

Parameters

<i>pAlku</i>	Osoitin linkitetyn listan alkuun.
--------------	-----------------------------------

Returns

TIEDOT* Osoitin listan jälkimmäisen puoliskon alkuun.

Definition at line 449 of file [linkitettylista.c](#).

```
00449         {
00450     TIEDOT *p1 = NULL, *p2 = NULL;
00451     int i;
00452     int iPituus = 0;
00453     int iKeskipiste = 0;
00454
00455     iPituus = laskeListanPituus(pAlku);
00456     iKeskipiste = iPituus / 2 - 1;
00457
00458     p1 = pAlku;
00459     for (i = 0; i < iKeskipiste; i++) {
00460         p1 = p1->pSeuraava;
00461     }
00462
00463     // Halkaistaan lista kahtia, p1 päättyy siihen mistä p2 alkaa
00464     p2 = p1->pSeuraava;
00465     p1->pSeuraava = NULL;
00466     if (p2 != NULL) {
00467         p2->pEdellinen = NULL;
00468     }
00469
00470     // palautetaan osoitin toisen listan alkuun
00471     return p2;
00472 }
```

3.11.2.2 kirjoitaTiedostoAlustaLoppuun()

```
void kirjoitaTiedostoAlustaLoppuun (
    char * pNimi,
    TIEDOT * pAlku)
```

Parameters

<i>pNimi</i>	Kirjoitettavan tiedoston nimi.
<i>pAlku</i>	Osoitin linkitetyn listan alkuun.

Returns

void

Definition at line 129 of file [linkitettylista.c](#).

```

00129                                     {
00130     FILE *Tiedosto = NULL;
00131     TIEDOT *ptr = NULL;
00132
00133     if (pAlku == NULL) {
00134         printf("Lista on tyhjä, lue tiedosto ennen kirjoittamista.\n");
00135         return;
00136     }
00137
00138     // Kirjoitus tiedoston avaaminen
00139     if ((Tiedosto = fopen(pNimi, "w")) == NULL) {
00140         perror("Tiedoston avaaminen epäonnistui, lopetetaan");
00141         exit(0);
00142     }
00143
00144     // Tiedostoon kirjoittaminen
00145     ptr = pAlku;
00146     while (ptr != NULL) {
00147         fprintf(Tiedosto, "%s,%d\n", ptr->aNimi, ptr->iYhteensa);
00148         ptr = ptr->pSeuraava;
00149     }
00150
00151     // Tiedoston sulkeminen
00152     fclose(Tiedosto);
00153     printf("Tiedosto %s kirjoitettu.\n", pNimi);
00154     return;
00155 }

```

3.11.2.3 kirjoitaTiedostoLopustaAlkuun()

```

void kirjoitaTiedostoLopustaAlkuun (
    char * pNimi,
    TIEDOT * pAlku)

```

Parameters

<i>pNimi</i>	Kirjoitettavan tiedoston nimi.
<i>pAlku</i>	Osoitin linkitetyn listan alkuun.

Returns

void

Definition at line 164 of file [linkitettylista.c](#).

```

00164                                     {
00165     FILE *Tiedosto = NULL;
00166     TIEDOT *ptr = pAlku;
00167
00168     if (pAlku == NULL) {
00169         printf("Lista on tyhjä, lue tiedosto ennen kirjoittamista.\n");
00170         return;
00171     }
00172
00173     if ((Tiedosto = fopen(pNimi, "w")) == NULL) {
00174         perror("Tiedoston avaaminen epäonnistui, lopetetaan");
00175         exit(0);
00176     }
00177
00178     // Mennään listan loppuun.
00179     while (ptr->pSeuraava != NULL) {
00180         ptr = ptr->pSeuraava;
00181     }
00182
00183     // Lopusta alkuun, kirjoitetaan tiedostoon.

```

```

00184     while (ptr != NULL) {
00185         fprintf(Tiedosto, "%s,%d\n", ptr->aNimi, ptr->iYhteensa);
00186         ptr = ptr->pEdellinen;
00187     }
00188
00189     fclose(Tiedosto);
00190     printf("Tiedosto %s kirjoitettu.\n", pNimi);
00191     return;
00192 }

```

3.11.2.4 laskeListanPituus()

```

int laskeListanPituus (
    TIEDOT * pAlku)

```

Parameters

<i>pAlku</i>	Osoitin linkitetyn listan alkuun.
--------------	-----------------------------------

Returns

int Linkitetyn listan alkioden lukumäärä.

Definition at line 434 of file [linkitettylista.c](#).

```

00434                                     {
00435     int i = 0;
00436     while (pAlku != NULL) {
00437         i++;
00438         pAlku = pAlku->pSeuraava;
00439     }
00440     return i;
00441 }

```

3.11.2.5 lisaaListaan()

```

TIEDOT * lisaaListaan (
    TIEDOT * pAlku,
    int iIndeksi,
    char * pNimi,
    int iArvo)

```

Lisaa käyttäjän haluamaan kohtaan (indeksi) tiedot lisattavasta nimestä ja niiden lukumaarasta.

Parameters

<i>pAlku</i>	Osoitin linkitetyn listan alkuun.
<i>iIndeksi</i>	Käyttäjän antama indeksi, eli kohta, johon tietue lisataan linkitettyssä listassa. Jos lista on tyhjä, lisataan automaattisesti ensimmäiseksi elementiksi. Jos indeksi on suurempi kuin listan elementtien määrä, lisataan automaattisesti viimeiseksi.
<i>pNimi</i>	Lisattava nimi.
<i>iArvo</i>	Lisattavan nimen lukumaara.

Returns

pAlku Osoitin listan nykyiseen alkuun.

Definition at line 207 of file [linkitettylista.c](#).

```

00207                                     {
00208     TIEDOT *pUusi = NULL;
00209     TIEDOT *ptr = pAlku;
00210     int i = 1;
00211
00212     // Varataan muistia uudelle linkitetyn listan solmulle.
00213     if ((pUusi = (TIEDOT *)malloc(sizeof(TIEDOT))) == NULL) {
00214         perror("Muistin varaus epäonnistui, lopetetaan");
00215         exit(0);
00216     }
00217
00218     // Lisataan tiedot tietueeseen.
00219     strcpy(pUusi->aNimi, pNimi);
00220     pUusi->iYhteensa = iArvo;
00221
00222     // Jos lista on tyhjä, lisataan uusi solmu ensimmäiseksi.
00223     if (pAlku == NULL) {
00224         pUusi->pEdellinen = NULL;
00225         pUusi->pSeuraava = NULL;
00226         pAlku = pUusi;
00227         return (pAlku);
00228     } else {
00229         // Jos lisataan listan alkuun, eli indeksi = 0.
00230         if (iIndeksi <= 1) {
00231             pUusi->pEdellinen = NULL;
00232             pUusi->pSeuraava = pAlku;
00233             pAlku->pEdellinen = pUusi;
00234             pAlku = pUusi;
00235             return (pAlku);
00236         }
00237
00238         // Etsitään listalta oikea kohta, johon tiedot lisataan.
00239         while (ptr->pSeuraava != NULL && i < (iIndeksi - 1)) {
00240             ptr = ptr->pSeuraava;
00241             i++;
00242         }
00243         // Lisataan tiedot oikeaan kohtaan linkitettyä listaa.
00244         // Paivitetään osoittimet osoittamaan oikeisiin kohtiin.
00245         pUusi->pSeuraava = ptr->pSeuraava;
00246         pUusi->pEdellinen = ptr;
00247
00248         if (ptr->pSeuraava != NULL) {
00249             ptr->pSeuraava->pEdellinen = pUusi;
00250         }
00251
00252         ptr->pSeuraava = pUusi;
00253     }
00254     return (pAlku);
00255 }
```

3.11.2.6 lisaysLajittelu()

```

TIEDOT * lisaysLajittelu (
    TIEDOT * pAlku)
```

Parameters

<i>pAlku</i>	Osoitin lajiteltavan linkitetyn listan alkuun.
--------------	--

Returns

TIEDOT* Lajiteltu lista.

Definition at line 602 of file [linkitettylista.c](#).

```

00602                                     {
00603     // Jos lista on tyhjä
```

```

00604     if (pAlku == NULL) {
00605         return pAlku;
00606     }
00607
00608     TIEDOT *pLajittelematon = pAlku; // Osoitin lajiteltavan listan nykyiseen solmuun
00609     TIEDOT *pLajiteltu = NULL;      // Tehdään toinen lista, johon lajitellut tiedot laitetaan
00610     TIEDOT *pSeuraava = NULL;
00611     TIEDOT *ptr = NULL; // Osoitin, jolla läpikäydään lajiteltua listaa.
00612
00613     /* Käydään lajittelematonta listaa läpi ja lisätään alkiot
00614     * lajiteltuun listaan pienenevässä järjestyksessä arvon mukaan.
00615     */
00616     while (pLajittelematon != NULL) {
00617         pSeuraava = pLajittelematon->pSeuraava; // pSeuraava talteen ennen listan läpikäyntiä
00618
00619         // irrotetaan alkio
00620         if (pSeuraava != NULL) {
00621             pSeuraava->pEdellinen = pLajittelematon->pEdellinen;
00622         }
00623         if (pLajittelematon->pEdellinen != NULL) {
00624             pLajittelematon->pEdellinen->pSeuraava = pSeuraava;
00625         }
00626         pLajittelematon->pSeuraava = NULL;
00627         pLajittelematon->pEdellinen = NULL;
00628
00629         if (pLajiteltu == NULL || pLajiteltu->iYhteensa < pLajittelematon->iYhteensa ||
00630             (pLajiteltu->iYhteensa == pLajittelematon->iYhteensa &&
00631              strcmp(pLajiteltu->aNimi, pLajittelematon->aNimi) < 0)) {
00632             // Lisätään lajitellun listan alkuun
00633             pLajittelematon->pSeuraava = pLajiteltu;
00634             if (pLajiteltu != NULL) {
00635                 pLajiteltu->pEdellinen = pLajittelematon;
00636             }
00637             pLajiteltu = pLajittelematon;
00638
00639         } else {
00640             ptr = pLajiteltu;
00641             while (ptr->pSeuraava != NULL &&
00642                  (ptr->pSeuraava->iYhteensa > pLajittelematon->iYhteensa ||
00643                   (ptr->pSeuraava->iYhteensa == pLajittelematon->iYhteensa &&
00644                    strcmp(ptr->pSeuraava->aNimi, pLajittelematon->aNimi) > 0))) {
00645                 ptr = ptr->pSeuraava;
00646             }
00647             pLajittelematon->pSeuraava = ptr->pSeuraava;
00648             if (ptr->pSeuraava != NULL) {
00649                 ptr->pSeuraava->pEdellinen = pLajittelematon;
00650             }
00651             ptr->pSeuraava = pLajittelematon;
00652             pLajittelematon->pEdellinen = ptr;
00653         }
00654         pLajittelematon = pSeuraava;
00655     }
00656     return pLajiteltu;
00657 }

```

3.11.2.7 lomitus()

```

TIEDOT * lomitus (
    TIEDOT * p1,
    TIEDOT * p2)

```

Parameters

<i>p1</i>	Osoitin ensimmäisen lajitellun listan alkuun.
<i>p2</i>	Osoitin jälkimmäisen lajitellun listan alkuun.

Returns

TIEDOT* yhdistetty lajiteltu lista.

Definition at line 481 of file [linkitettylista.c](#).


```

00481                                     {
00482 // Jos kumpikaan lista on tyhjä, palautetaan toinen.
00483 if (p1 == NULL)
00484     return p2;
00485 if (p2 == NULL)
00486     return p1;
00487
00488 TIEDOT *p3 = NULL; // Uuden listan alku.
00489 TIEDOT *p3Loppu = NULL; // Uuden lista loppu, jotta loppuun lisääminen onnistuu helposti.
00490
00491 // Käydään molempia listoja läpi:
00492 while (p1 != NULL && p2 != NULL) {
00493     int iVertailu =
00494         strcmp(p1->aNimi, p2->aNimi); // Jos sama arvo, lisätään aakkosjärjestyksessä.
00495     if (p1->iYhteensa > p2->iYhteensa || (p1->iYhteensa == p2->iYhteensa && iVertailu > 0)) {
00496         TIEDOT *ptr = p2; // p2[0] talteen
00497         // Poistetaan p2[0] listasta.
00498         p2 = p2->pSeuraava;
00499         if (p2 != NULL) {
00500             p2->pEdellinen = NULL;
00501         }
00502         // Lisätään p2[0] uuden listan loppuun.
00503         ptr->pSeuraava = NULL;
00504         ptr->pEdellinen = p3Loppu;
00505         if (p3Loppu != NULL) {
00506             p3Loppu->pSeuraava = ptr;
00507         } else {
00508             p3 = ptr; // ensimmäinen alkio, jos lista tyhjä
00509         }
00510         p3Loppu = ptr; // päivitetään loppu.
00511     } else {
00512         TIEDOT *ptr = p1; // p1[0] talteen
00513         p1 = p1->pSeuraava;
00514         // Poistetaan p1[0] listasta.
00515         if (p1 != NULL) {
00516             p1->pEdellinen = NULL;
00517         }
00518         // Lisätään p1[0] uuden listan loppuun.
00519         ptr->pSeuraava = NULL;
00520         ptr->pEdellinen = p3Loppu;
00521         if (p3Loppu != NULL) {
00522             p3Loppu->pSeuraava = ptr;
00523         } else {
00524             p3 = ptr; // ensimmäinen alkio, jos lista tyhjä
00525         }
00526         p3Loppu = ptr; // päivitetään loppu.
00527     }
00528 }
00529
00530 // Tässä kohtaa joko p1 tai p2 on tyhjä.
00531
00532 // Lisätään loput p1:stä uuden listan loppuun.
00533 while (p1 != NULL) {
00534     TIEDOT *ptr = p1;
00535     p1 = p1->pSeuraava;
00536     if (p1 != NULL) {
00537         p1->pEdellinen = NULL;
00538     }
00539     ptr->pSeuraava = NULL;
00540     ptr->pEdellinen = p3Loppu;
00541     if (p3Loppu != NULL) {
00542         p3Loppu->pSeuraava = ptr;
00543     } else {
00544         p3 = ptr;
00545     }
00546     p3Loppu = ptr;
00547 }
00548
00549 // Lisätään loput p2:stä uuden listan loppuun.
00550 while (p2 != NULL) {
00551     TIEDOT *ptr = p2;
00552     p2 = p2->pSeuraava;
00553     if (p2 != NULL) {
00554         p2->pEdellinen = NULL;
00555     }
00556     ptr->pSeuraava = NULL;
00557     ptr->pEdellinen = p3Loppu;
00558     if (p3Loppu != NULL) {
00559         p3Loppu->pSeuraava = ptr;
00560     } else {
00561         p3 = ptr;
00562     }
00563     p3Loppu = ptr;
00564 }
00565 return p3;
00566 }

```

3.11.2.8 lomitusLajittelu()

```
TIEDOT * lomitusLajittelu (
    TIEDOT * pAlku)
```

Parameters

<i>pAlku</i>	Osoitin lajiteltavan linkitetyn listan alkuun.
--------------	--

Returns

TIEDOT* Lajiteltu lista.

Definition at line 574 of file [linkitettylista.c](#).

```
00574                                     {
00575     TIEDOT *p1 = NULL, *p2 = NULL;
00576
00577     // Jos lista on tyhjä tai sisältää vain yhden alkion
00578     if (pAlku == NULL || pAlku->pSeuraava == NULL) {
00579         return pAlku;
00580     }
00581
00582     // Jaetaan lista kahtia
00583     p2 = halkaise(pAlku);
00584     p1 = pAlku;
00585
00586     // Lajitellaan molemmat puolikkaat
00587     p1 = lomitusLajittelu(p1);
00588     p2 = lomitusLajittelu(p2);
00589
00590     // Yhdistetään lajitellut puolikkaat
00591     return lomitus(p1, p2);
00592 }
```

3.11.2.9 lue()

```
TIEDOT * lue (
    char * pNimi,
    TIEDOT * pAlku)
```

Avaa ja lukee käyttäjän syötteen mukaisen tiedoston. Kutsuu muistinvaraus aliohjelmaa muistin varaamiseksi.

Parameters

<i>pNimi</i>	Luettavan tiedoston nimi.
<i>pAlku</i>	Osoitin linkitetyn listan alkuun.

Returns

pAlku Uusi osoitin linkitetyn listan alkuun.

Definition at line 19 of file [linkitettylista.c](#).

```
00019                                     {
00020     FILE *Tiedosto = NULL;
00021     char aRivi[LEN] = "";
00022     char *p1 = NULL, *p2 = NULL;
00023     int iOtsikko = 0;
00024     int iMaara = 0;
00025 }
```

```

00026      /* Tiedoston avaaminen. */
00027
00028      if ((Tiedosto = fopen(pNimi, "r")) == NULL) {
00029          perror("Tiedoston avaaminen epäonnistui, lopetetaan");
00030          exit(0);
00031      }
00032
00033      /* Tiedoston lukeminen */
00034
00035      while ((fgets(aRivi, LEN, Tiedosto)) != NULL) {
00036
00037          aRivi[strlen(aRivi) - 1] = '\0';
00038
00039          if (iOtsikko == 0) {
00040              iOtsikko = 1;
00041              continue;
00042          }
00043
00044          if ((p1 = strtok(aRivi, ";")) == NULL) {
00045              perror("Tiedoston pilkkominen epäonnistui, lopetetaan");
00046              exit(0);
00047          }
00048
00049          if ((p2 = strtok(NULL, ";")) == NULL) {
00050              perror("Tiedoston pilkkominen epäonnistui, lopetetaan");
00051              exit(0);
00052          }
00053
00054          iMaara = atoi(p2);
00055
00056          /* Muistin varaaminen ja linkitetyn listan luominen. */
00057          pAlku = varaaMuistia(pAlku, p1, iMaara);
00058      }
00059      /* Tiedoston sulkeminen. */
00060      fclose(Tiedosto);
00061      printf("Tiedosto '%s' luettu.\n", pNimi);
00062      return (pAlku);
00063 }

```

3.11.2.10 onkoLukuVaiNimi()

```

int onkoLukuVaiNimi (
    char * Tieto)

```

Parameters

<i>Tieto</i>	Käyttäjän antama syöte.
--------------	-------------------------

Returns

int Tieto, onko luku vai nimi, jos -2, niin käyttäjä antoi luvun, jos joku muu niin se on luku.

Definition at line 403 of file [linkitettylista.c](#).

```

00403      {
00404          int iVastaus = 0;
00405          int i = 0;
00406          int iPituus = 0;
00407
00408          iPituus = strlen(Tieto);
00409
00410          for (i = 0; i < iPituus; i++) {
00411              if (isdigit(Tieto[i])) {
00412                  iVastaus = -1;
00413              } else {
00414                  iVastaus = -2;
00415                  break;
00416              }
00417          }
00418
00419          if (iVastaus == -1) {
00420              iVastaus = atoi(Tieto);
00421          }
00422
00423          return (iVastaus);
00424 }

```

3.11.2.11 poistaListastaAlkio()

```
TIEDOT * poistaListastaAlkio (
    TIEDOT * pAlku,
    int iLuvuVaiNimi,
    char * pTieto)
```

Parameters

<i>pAlku</i>	Osoitin linkitetyn listan alkuun.
<i>iLuvuVaiNimi</i>	Tieto siitä, haluaako käyttäjä poistaa alkion nimen vai luvun perusteella.
<i>pTieto</i>	Poistettavan alkion tieto, nimi tai luku.

Returns

TIEDOT* pAlku Osoitin listan nykyiseen alkuun.

Definition at line 265 of file [linkitettylista.c](#).

```
00265                                     {
00266     int iSamojenAlkiodenLkm = 0;
00267     char aPoistettavaNimi[LEN] = "";
00268
00269     if (iLuvuVaiNimi == -2) {
00270         pAlku = poistaListastaNimenPerusteella(pAlku, pTieto);
00271     } else {
00272         iSamojenAlkiodenLkm = useamallaAlkiollaSamaLKM(pAlku, iLuvuVaiNimi);
00273
00274         if (iSamojenAlkiodenLkm > 1) {
00275             kysyNimi("Anna nimi, joka poistetaan: ", aPoistettavaNimi);
00276             pAlku = poistaListastaNimenPerusteella(pAlku, aPoistettavaNimi);
00277         } else {
00278             pAlku = poistaListastaLkmPeruusteella(pAlku, iLuvuVaiNimi);
00279         }
00280     }
00281
00282     return (pAlku);
00283 }
```

3.11.2.12 poistaListastaLkmPeruusteella()

```
TIEDOT * poistaListastaLkmPeruusteella (
    TIEDOT * pAlku,
    int iLKM)
```

Käyttäjä on antanut lukumäärän. Alkio, jossa on tämä lukumäärä poistetaan.

Parameters

<i>pAlku</i>	Osoitin linkitetyn listan alkuun.
<i>iLKM</i>	Käyttäjän antama lukumäärä. Alkio, jossa on tämä lukumäärä poistetaan.

Returns

TIEDOT* Palautetaan linkitettylista, josta on poistettu alkio.

Definition at line 292 of file [linkitettylista.c](#).

```

00292                                     {
00293     TIEDOT *ptr = pAlku;
00294     TIEDOT *pEdellinen = NULL;
00295     int iAlkioLoytyi = 0;
00296
00297     // Tarkistetaan, onko poistettava alkio ensimmäinen.
00298     if ((ptr != NULL) && (ptr->iYhteensa == iLKM)) {
00299         // Sijoitetaan osoitin osoittamaan seuraavaan alkioon.
00300         pAlku = ptr->pSeuraava;
00301         free(ptr);
00302         printf("Alkio poistettu.\n");
00303
00304         // Jos poistettava alkio ei ole ensimmäinen.
00305     } else {
00306         // Etsitään kohtaa, jossa poistettava alkio on.
00307         while (ptr != NULL) {
00308             if (ptr->iYhteensa == iLKM) {
00309                 pEdellinen->pSeuraava = ptr->pSeuraava;
00310                 free(ptr);
00311                 iAlkioLoytyi = 1;
00312                 break;
00313             }
00314             pEdellinen = ptr;
00315             ptr = ptr->pSeuraava;
00316         }
00317
00318         // Kerrotaan käyttäjälle, poistettiin alkio.
00319         if (iAlkioLoytyi == 1) {
00320             printf("Alkio poistettu.\n");
00321         } else {
00322             printf("Poistettavaa alkiota ei löydetty.\n");
00323         }
00324     }
00325
00326     return (pAlku);
00327 }

```

3.11.2.13 poistaListastaNimenPerusteella()

```

TIEDOT * poistaListastaNimenPerusteella (
    TIEDOT * pAlku,
    char * pNimi)

```

Käyttäjä on antanut nimen. Alkio, jossa on tämä nimi poistetaan.

Parameters

<i>pAlku</i>	Osoitin linkitetyn listan alkuun.
<i>pNimi</i>	Käyttäjän antama nimi. Alkio, jossa on tämä nimi poistetaan.

Returns

TIEDOT* TIEDOT* Palautetaan linkitettylista, josta on poistettu alkio.

Definition at line 336 of file [linkitettylista.c](#).

```

00336                                     {
00337     TIEDOT *ptr = pAlku;
00338     TIEDOT *pEdellinen = NULL;
00339     int iAlkioLoytyi = 0;
00340
00341     // Tarkistetaan, onko poistettava alkio ensimmäinen.
00342     if ((ptr != NULL) && (strcmp(ptr->aNimi, pNimi) == 0)) {
00343         // Sijoitetaan osoitin osoittamaan seuraavaan alkioon.

```

```

00344     pAlku = ptr->pSeuraava;
00345     free(ptr);
00346     printf("Alkio poistettu.\n");
00347
00348     // Jos poistettava alkio ei ole ensimmäinen.
00349 } else {
00350     // Etsitään kohtaa, jossa poistettava alkio on.
00351     while (ptr != NULL) {
00352         if (strcmp(ptr->aNimi, pNimi) == 0) {
00353             pEdellinen->pSeuraava = ptr->pSeuraava;
00354             free(ptr);
00355             iAlkioLoytyi = 1;
00356             break;
00357         }
00358         pEdellinen = ptr;
00359         ptr = ptr->pSeuraava;
00360     }
00361
00362     // Kerrotaan käyttäjälle, poistettiinkö alkio.
00363     if(iAlkioLoytyi == 1) {
00364         printf("Alkio poistettu.\n");
00365     } else {
00366         printf("Poistettavaa alkiota ei löydetty.\n");
00367     }
00368 }
00369 return (pAlku);
00370 }

```

3.11.2.14 useammallaAlkiollaSamaLKM()

```

int useammallaAlkiollaSamaLKM (
    TIEDOT * pAlku,
    int iLKM)

```

Parameters

<i>pAlku</i>	Osoitin linkitetyn listan alkuun.
<i>iLKM</i>	Käyttäjän antama lukumäärä, joiden määrä selvitetään.

Returns

int Palauttaa luvun, jossa on kaikki alkio, joissa on käyttäjän antama luku.

Definition at line 379 of file [linkitettylista.c](#).

```

00379     {
00380         int iSamoja = 0;
00381         TIEDOT *ptr = NULL;
00382         ptr = pAlku;
00383
00384         // Käydään linkitettylista läpi
00385         while (ptr != NULL) {
00386             // Lisätään summaan yksi, jos linkitetyn listan alkion lukumäärä on sama kuin käyttäjän
00387             // antama luku.
00388             if (ptr->iYhteensa == iLKM) {
00389                 iSamoja++;
00390             }
00391             ptr = ptr->pSeuraava;
00392         }
00393
00394         return (iSamoja);
00395     }

```

3.11.2.15 vapautaMuisti()

```

TIEDOT * vapautaMuisti (
    TIEDOT * pAlku)

```

Parameters

<i>pAlku</i>	Osoitin linkitetyn listan alkuun.
--------------	-----------------------------------

Returns

pAlku Osoitin, joka on NULL, koska lista on tyhja.

Definition at line 110 of file [linkitettylista.c](#).

```

00110                                     {
00111     /* Muistin vapauttaminen. */
00112     TIEDOT *ptr = pAlku;
00113     while (ptr != NULL) {
00114         pAlku = ptr->pSeuraava;
00115         free(ptr);
00116         ptr = pAlku;
00117     }
00118     pAlku = NULL;
00119     return (pAlku);
00120 }
```

3.11.2.16 varaaMuistia()

```

TIEDOT * varaaMuistia (
    TIEDOT * pAlku,
    char * pNimi,
    int iMaara)
```

Parameters

<i>pAlku</i>	Osoitin linkitetyn listan alkuun.
<i>pNimi</i>	Merkkijono solmun nimeksi.
<i>iMaara</i>	Luku, joka asetetaan solmuun maaraksi.

Returns

pAlku Uusi osoitin linkitetyn listan alkuun.

Definition at line 73 of file [linkitettylista.c](#).

```

00073                                     {
00074     TIEDOT *pUusi = NULL;
00075     TIEDOT *ptr = NULL;
00076
00077     /* Muistin varaaminen. */
00078
00079     if ((pUusi = (TIEDOT *)malloc(sizeof(TIEDOT))) == NULL) {
00080         perror("Muistin varaus epäonnistui, lopetetaan");
00081         exit(0);
00082     }
00083
00084     /* Kaksisuuntaisen linkitetyn listan luominen. */
00085
00086     strcpy(pUusi->aNimi, pNimi);
00087     pUusi->iYhteensa = iMaara;
00088     pUusi->pSeuraava = NULL;
00089
00090     if (pAlku == NULL) {
00091         pUusi->pEdellinen = NULL;
00092         pAlku = pUusi;
00093     } else {
00094         ptr = pAlku;
00095         while (ptr->pSeuraava != NULL) {
00096             ptr = ptr->pSeuraava;
00097         }
00098         ptr->pSeuraava = pUusi;
00099         pUusi->pEdellinen = ptr;
00100     }
00101     return (pAlku);
00102 }
```

3.12 linkitettylista.h

[Go to the documentation of this file.](#)

```

00001 #ifndef LINKITETTYLISTA_H
00002 #define LINKITETTYLISTA_H
00003 #include "yleiset.h"
00004
00005 typedef struct tiedot {
00006     char aNimi[LEN];
00007     int iYhteensa;
00008     struct tiedot *pSeuraava;
00009     struct tiedot *pEdellinen;
00010 } TIEDOT;
00011
00012 TIEDOT *lue(char *pNimi, TIEDOT *pAlku);
00013 TIEDOT *varaaMuistia(TIEDOT *pAlku, char *pSukunimi, int iMaara);
00014 TIEDOT *vapautaMuisti(TIEDOT *pAlku);
00015 void kirjoitaTiedostoAlustaLoppuun(char *pKirjoitaTiedostoNimi, TIEDOT *pAlku);
00016 void kirjoitaTiedostoLopustaAlkuun(char *pKirjoitaTiedostoNimi, TIEDOT *pAlku);
00017 TIEDOT *lisaaListaan(TIEDOT *pAlku, int iIndeksi, char *pNimi, int iArvo);
00018 TIEDOT *poistaListastaLkmPeruusteella(TIEDOT *pAlku, int iLKM);
00019 int useammallaAlkiollaSamaLKM(TIEDOT *pAlku, int iLKM);
00020 TIEDOT *poistaListastaNimenPerusteella(TIEDOT *pAlku, char *pNimi);
00021 int laskeListanPituus(TIEDOT *pAlku);
00022 TIEDOT *halkaise(TIEDOT *pAlku);
00023 TIEDOT *lomitus(TIEDOT *p1, TIEDOT *p2);
00024 TIEDOT *lomitusLajittelu(TIEDOT *pAlku);
00025 TIEDOT *lisaysLajittelu(TIEDOT *pAlku);
00026 int onkoLukuVaiNimi(char *Tieto);
00027 TIEDOT *poistaListastaAlkio(TIEDOT *pAlku, int iLuvuVaiNimi, char *pTieto);
00028
00029 #endif

```

3.13 paaohjelma.c File Reference

```

#include "binaaripuu.h"
#include "linkitettylista.h"
#include "yleiset.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

```

Functions

- `int main (void)`
Paaohjelma, pyorittaa koko ohjelmaa.

3.13.1 Function Documentation

3.13.1.1 main()

```

int main (
    void )

```


Returns

int Palauttaa aina 0, kun ohjelma tulee päätökseen.

Definition at line 13 of file [paaohjelma.c](#).

```
00013     {
00014     int iValinta = 0;
00015
00016     do {
00017         iValinta = valikko();
00018         if (iValinta == 1) {
00019             mainLista();
00020         } else if (iValinta == 2) {
00021             mainBinaaripuu();
00022         } else if (iValinta == 0) {
00023             printf("Lopetetaan.\n");
00024         } else {
00025             printf("Tuntematon valinta, yritä uudestaan.\n");
00026         }
00027         printf("\n");
00028     } while (iValinta != 0);
00029
00030     printf("Kiitos ohjelman käytöstä.\n");
00031     return (0);
00032 }
```

3.14 paaohjelma.c

[Go to the documentation of this file.](#)

```
00001 #include "binaaripuu.h"
00002 #include "linkitettylista.h"
00003 #include "yleiset.h"
00004 #include <stdio.h>
00005 #include <stdlib.h>
00006 #include <string.h>
00007
00013 int main(void) {
00014     int iValinta = 0;
00015
00016     do {
00017         iValinta = valikko();
00018         if (iValinta == 1) {
00019             mainLista();
00020         } else if (iValinta == 2) {
00021             mainBinaaripuu();
00022         } else if (iValinta == 0) {
00023             printf("Lopetetaan.\n");
00024         } else {
00025             printf("Tuntematon valinta, yritä uudestaan.\n");
00026         }
00027         printf("\n");
00028     } while (iValinta != 0);
00029
00030     printf("Kiitos ohjelman käytöstä.\n");
00031     return (0);
00032 }
```

3.15 punamustapuu.c File Reference

```
#include "punamustapuu.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

Functions

- **RBSOLMU * varaaMuistiaRB** (char *pNimi, int iArvo)
Varaa muistia punamustalle puulle.
- **RBSOLMU * vapautaMuistiRB** (RBSOLMU *pJuuriSolmu)
Vapauttaa muistin punamustasta puusta.
- **RBSOLMU * lisaaRBSolmu** (RBSOLMU *pJuuriSolmu, RBSOLMU *pUusi)
Lisää solmun punamustapuuhun.
- **RBSOLMU * luoRBPuu** (RBSOLMU *pJuuriSolmu, char *pNimi)
Luo punamustapuun.
- void **kierraVasemmalle** (RBSOLMU **pJuuriSolmu, RBSOLMU *pSolmu)
Tekee vasemman rotaation punamustalle puulle.
- void **kierraOikealle** (RBSOLMU **pJuuriSolmu, RBSOLMU *pSolmu)
Tekee oikean rotaation punamustalle puulle.
- void **korjaaLisays** (RBSOLMU **pJuuriSolmu, RBSOLMU *pUusi)
Korjaa lisäyksen jälkeen punamustan puun rakenteen ja värit oikeiksi.
- void **kirjoitaRBTiedostoon** (char *pNimi, RBSOLMU *pJuuriSolmu)
Kirjoittaa punamustapuun tiedostoon.
- void **kirjoitaRB** (char *pNimi, RBSOLMU *pJuuriSolmu)
Kirjoittaa punamustapuun.

3.15.1 Function Documentation

3.15.1.1 kierraOikealle()

```
void kierraOikealle (
    RBSOLMU ** pJuuriSolmu,
    RBSOLMU * pSolmu)
```

Parameters

<i>pJuuriSolmu</i>	Osoitin juurisolmun osoittimeen.
<i>pSolmu</i>	Osoitin solmuun. Kierro tehdään tämän solmun ympärille.

Definition at line 170 of file [punamustapuu.c](#).

```
00170                                     {
00171     RBSOLMU *pVasenLapsi = pSolmu->pVasen;
00172     pSolmu->pVasen = pVasenLapsi->pOikea;
00173
00174     if (pSolmu->pVasen != NULL)
00175         pSolmu->pVasen->pVanhempi = pSolmu;
00176
00177     pVasenLapsi->pVanhempi = pSolmu->pVanhempi;
00178
00179     if (pSolmu->pVanhempi == NULL)
00180         *pJuuriSolmu = pVasenLapsi;
00181     else if (pSolmu == pSolmu->pVanhempi->pVasen)
00182         pSolmu->pVanhempi->pVasen = pVasenLapsi;
00183     else
00184         pSolmu->pVanhempi->pOikea = pVasenLapsi;
00185
00186     pVasenLapsi->pOikea = pSolmu;
00187     pSolmu->pVanhempi = pVasenLapsi;
00188
00189     return;
00190 }
```

3.15.1.2 kierraVasemmalle()

```
void kierraVasemmalle (
    RBSOLMU ** pJuurisolmu,
    RBSOLMU * pSolmu)
```

Parameters

<i>pJuurisolmu</i>	Osoitin juurisolmun osoittimeen.
<i>pSolmu</i>	Osoitin solmuun. Kierto tehdään tämän solmun ympärille.

Definition at line 142 of file [punamustapuu.c](#).

```
00142                                     {
00143     RBSOLMU *pOikeaLapsi = pSolmu->pOikea;
00144     pSolmu->pOikea = pOikeaLapsi->pVasen;
00145
00146     if (pSolmu->pOikea != NULL)
00147         pSolmu->pOikea->pVanhempi = pSolmu;
00148
00149     // Solmun oikea lapsi nousee ylös puussa.
00150     pOikeaLapsi->pVanhempi = pSolmu->pVanhempi;
00151
00152     if (pSolmu->pVanhempi == NULL)
00153         *pJuurisolmu = pOikeaLapsi;
00154     else if (pSolmu == pSolmu->pVanhempi->pVasen)
00155         pSolmu->pVanhempi->pVasen = pOikeaLapsi;
00156     else
00157         pSolmu->pVanhempi->pOikea = pOikeaLapsi;
00158
00159     pOikeaLapsi->pVasen = pSolmu;
00160     pSolmu->pVanhempi = pOikeaLapsi;
00161     return;
00162 }
```

3.15.1.3 kirjoitaRB()

```
void kirjoitaRB (
    char * pNimi,
    RBSOLMU * pJuurisolmu)
```

Kutsuu tiedostoon kirjoitettavaa aliohjelmaa ja antaa sille kirjoitettavat arvot.

Parameters

<i>pNimi</i>	Osoitin kirjoitettavan tiedoston nimeen.
<i>pJuurisolmu</i>	Osoitin solmuun, jota aliohjelma käsittelee.

Definition at line 291 of file [punamustapuu.c](#).

```
00291                                     {
00292     if (pJuurisolmu != NULL) {
00293         kirjoitaRBTiedostoon(pNimi, pJuurisolmu);
00294         kirjoitaRB(pNimi, pJuurisolmu->pVasen);
00295         kirjoitaRB(pNimi, pJuurisolmu->pOikea);
00296     }
00297     return;
00298 }
```

3.15.1.4 kirjoitaRBTiedostoon()

```
void kirjoitaRBTiedostoon (
    char * pNimi,
    RBSOLMU * pJuurisolmu)
```

Parameters

<i>pNimi</i>	Osoitin kirjoitettavan tiedoston nimeen.
<i>pJuurisolmu</i>	Osoitin punamustapuun juurisolmuun.

Definition at line 265 of file [punamustapuu.c](#).

```
00265                                     {
00266     FILE *Tiedosto = NULL;
00267
00268     if (pJuurisolmu == NULL) {
00269         return;
00270     }
00271
00272     /* Tiedoston avaaminen. */
00273     if ((Tiedosto = fopen(pNimi, "a")) == NULL) {
00274         perror("Tiedoston avaaminen epäonnistui, lopetetaan");
00275         exit(0);
00276     }
00277     /* Tiedostoon kirjoittaminen. */
00278     fprintf(Tiedosto, "%s,%d\n", pJuurisolmu->aNimi, pJuurisolmu->iArvo);
00279
00280     /* Tiedoston sulkeminen. */
00281     fclose(Tiedosto);
00282     return;
00283 }
```

3.15.1.5 korjaaLisays()

```
void korjaaLisays (
    RBSOLMU ** pJuurisolmu,
    RBSOLMU * pUusi)
```

Parameters

<i>pJuurisolmu</i>	Osoitin juurisolmun osoittimeen.
<i>pUusi</i>	Osoitin lisätyyn solmuun.

Definition at line 198 of file [punamustapuu.c](#).

```
00198                                     {
00199     RBSOLMU *vanhempi = NULL;
00200     RBSOLMU *seta = NULL;
00201     RBSOLMU *isoVanhempi = NULL;
00202
00203     // Pyöritään while:ssa kunnes päädytään juurisolmuun tai vanhempi on mustan värinen.
00204     while ((pUusi != *pJuurisolmu) && (pUusi->pVanhempi->iVariBitti == 0)) {
00205         vanhempi = pUusi->pVanhempi;
00206         isoVanhempi = vanhempi->pVanhempi;
00207
00208         // Vanhempi on isovanhemman vasen lapsi.
00209         if (vanhempi == isoVanhempi->pVasen) {
00210             seta = isoVanhempi->pOikea;
00211
00212             // Tarkastetaan onko setä punainen.
00213             if (seta != NULL && seta->iVariBitti == 0) {
00214                 vanhempi->iVariBitti = 1;
00215                 seta->iVariBitti = 1;
00216                 isoVanhempi->iVariBitti = 0;
00217                 pUusi = isoVanhempi;
00218             } else {
00219
00220                 // Tarkastetaan onko uusi solmu vanhemman solmun oikea lapsi solmu.
```

```

00221         if (pUusi == vanhempi->pOikea) {
00222             pUusi = vanhempi;
00223             kierraVasemmalle(pJuurisolmu, pUusi);
00224             vanhempi = pUusi->pVanhempi;
00225         }
00226         vanhempi->iVariBitti = 1;
00227         isoVanhempi->iVariBitti = 0;
00228         kierraOikealle(pJuurisolmu, isoVanhempi);
00229     }
00230
00231 } else {
00232     seta = isoVanhempi->pVasen;
00233
00234     // Tarkastetaan onko setä punainen.
00235     if (seta != NULL && seta->iVariBitti == 0) {
00236         vanhempi->iVariBitti = 1;
00237         seta->iVariBitti = 1;
00238         isoVanhempi->iVariBitti = 0;
00239         pUusi = isoVanhempi;
00240     } else {
00241
00242         // Tarkastetaan onko uusi solmu vanhemman solmun vasen lapsi solmu.
00243         if (pUusi == vanhempi->pVasen) {
00244             pUusi = vanhempi;
00245             kierraOikealle(pJuurisolmu, pUusi);
00246             vanhempi = pUusi->pVanhempi;
00247         }
00248         vanhempi->iVariBitti = 1;
00249         isoVanhempi->iVariBitti = 0;
00250         kierraVasemmalle(pJuurisolmu, isoVanhempi);
00251     }
00252 }
00253 }
00254 (*pJuurisolmu)->iVariBitti = 1;
00255
00256 return;
00257 }

```

3.15.1.6 lisääRBSolmu()

```

RBSOLMU * lisääRBSolmu (
    RBSOLMU * pJuurisolmu,
    RBSOLMU * pUusi)

```

Parameters

<i>pJuurisolmu</i>	Osoitin punamustapuun juurisolmuun.
<i>pUusi</i>	Osoitin lisättävään solmuun.

Returns

RBSOLMU* Palauttaa osoittimen punamustapuun juurisolmuun.

Katsotaan, ovatko arvot samat. Laitetaan aakkosten perusteella vasemmalle tai oikealle.

Definition at line 57 of file [punamustapuu.c](#).

```

00057                                     {
00058     int iVertailu = 0;
00059
00060     // Jos puu on tyhjä, palautetaan uusi RBSolmu.
00061     if (pJuurisolmu == NULL) {
00062         return pUusi;
00063     }
00064
00065     iVertailu = strcmp(pUusi->aNimi, pJuurisolmu->aNimi);
00066
00067     // Solmujen lisääminen rekursiivisesti
00068     if (pUusi->iArvo < pJuurisolmu->iArvo) {
00069         pJuurisolmu->pVasen = lisääRBSolmu(pJuurisolmu->pVasen, pUusi);
00070         pJuurisolmu->pVasen->pVanhempi = pJuurisolmu;
00071     } else if (pUusi->iArvo > pJuurisolmu->iArvo) {

```

```

00072     pJuurisolmu->pOikea = lisaaRBSolmu(pJuurisolmu->pOikea, pUusi);
00073     pJuurisolmu->pOikea->pVanhempi = pJuurisolmu;
00076 } else if (pUusi->iArvo == pJuurisolmu->iArvo) {
00077     if (iVertailu < 0) {
00078         pJuurisolmu->pVasen = lisaaRBSolmu(pJuurisolmu->pVasen, pUusi);
00079         pJuurisolmu->pVasen->pVanhempi = pJuurisolmu;
00080     } else if (iVertailu > 0) {
00081         pJuurisolmu->pOikea = lisaaRBSolmu(pJuurisolmu->pOikea, pUusi);
00082         pJuurisolmu->pOikea->pVanhempi = pJuurisolmu;
00083     }
00084 }
00085 return (pJuurisolmu);
00086 }

```

3.15.1.7 luoRBPuu()

```

RBSOLMU * luoRBPuu (
    RBSOLMU * pJuurisolmu,
    char * pNimi)

```

Lukee tiedoston, käsittelee tiedot, sekä luo puun.

Parameters

<i>pJuurisolmu</i>	Osoitin punamustapuun solmuun.
<i>pNimi</i>	Osoitin luettava tiedoston nimeen.

Returns

RBSOLMU* Palauttaa solmun osoittimen.

Definition at line 95 of file [punamustapuu.c](#).

```

00095     {
00096     FILE *Tiedosto = NULL;
00097     char aRivi[LEN] = "";
00098     int iOtsikko = 0;
00099     char *p1 = NULL, *p2 = NULL;
00100     int iArvo = 0;
00101
00102     if ((Tiedosto = fopen(pNimi, "r")) == NULL) {
00103         perror("Tiedoston avaaminen epäonnistui, lopetetaan");
00104         exit(0);
00105     }
00106
00107     // Luetaan, pilkotaan ja käsitellään tiedosto.
00108     while ((fgets(aRivi, LEN, Tiedosto)) != NULL) {
00109         aRivi[strlen(aRivi) - 1] = '\0';
00110         if (iOtsikko == 0) {
00111             iOtsikko++;
00112             continue;
00113         }
00114
00115         if ((p1 = strtok(aRivi, ";")) == NULL) {
00116             perror("Tiedoston pilkkominen epäonnistui, lopetetaan");
00117             exit(0);
00118         }
00119
00120         if ((p2 = strtok(NULL, ";")) == NULL) {
00121             perror("Tiedoston pilkkominen epäonnistui, lopetetaan");
00122             exit(0);
00123         }
00124
00125         iArvo = atoi(p2);
00126
00127         RBSOLMU *pUusi = varaaMuistiaRB(p1, iArvo);
00128         pJuurisolmu = lisaaRBSolmu(pJuurisolmu, pUusi);
00129         korjaaLisays(&pJuurisolmu, pUusi);
00130     }
00131
00132     fclose(Tiedosto);
00133     return (pJuurisolmu);
00134 }

```

3.15.1.8 vapautaMuistiRB()

```
RBSOLMU * vapautaMuistiRB (
    RBSOLMU * pJuuriSolmu)
```

Parameters

<i>pJuuriSolmu</i>	Osoitin punamustapuun solmuun.
--------------------	--------------------------------

Returns

RBSOLMU* Palauttaa punamustapuun osoittimen.

Definition at line 40 of file [punamustapuu.c](#).

```
00040                                     {
00041     if (pJuuriSolmu != NULL) {
00042         vapautaMuistiRB (pJuuriSolmu->pVasen);
00043         vapautaMuistiRB (pJuuriSolmu->pOikea);
00044         free (pJuuriSolmu);
00045     }
00046     pJuuriSolmu = NULL;
00047     return (pJuuriSolmu);
00048 }
```

3.15.1.9 varaaMuistiaRB()

```
RBSOLMU * varaaMuistiaRB (
    char * pNimi,
    int iArvo)
```

Parameters

<i>pNimi</i>	Solmun nimi, jolle varataan muistia ja kopioidaan uuteen solmuun.
<i>iArvo</i>	Solmun arvo, jolle varataan muistia ja kopioidaan uuteen solmuun.

Returns

RBSOLMU* Palauttaa osoittimen uuteen alustettuun solmuun.

Definition at line 15 of file [punamustapuu.c](#).

```
00015                                     {
00016     RBSOLMU *pUusi = NULL;
00017
00018     // Muistin varaaminen
00019     if ((pUusi = (RBSOLMU *)malloc(sizeof(RBSOLMU))) == NULL) {
00020         perror("Muistin varaus epäonnistui, lopetetaan");
00021         exit(0);
00022     }
00023
00024     // Alustetaan uusi solmu.
00025     pUusi->iVariBitti = 0; // Uudet solmut aina punaisia.
00026     strcpy(pUusi->aNimi, pNimi);
00027     pUusi->iArvo = iArvo;
00028     pUusi->pVasen = NULL;
00029     pUusi->pOikea = NULL;
00030     pUusi->pVanhempi = NULL;
00031     return (pUusi);
00032 }
```

3.16 punamustapuu.c

[Go to the documentation of this file.](#)

```

00001 #include "punamustapuu.h"
00002 #include <stdio.h>
00003 #include <stdlib.h>
00004 #include <string.h>
00005
00006 // Punamustapuun luominen ja kirjoittaminen tiedostoon.
00007
00015 RBSOLMU *varaaMuistiaRB(char *pNimi, int iArvo) {
00016     RBSOLMU *pUusi = NULL;
00017
00018     // Muistin varaaminen
00019     if ((pUusi = (RBSOLMU *)malloc(sizeof(RBSOLMU))) == NULL) {
00020         perror("Muistin varaus epäonnistui, lopetetaan");
00021         exit(0);
00022     }
00023
00024     // Alustetaan uusi solmu.
00025     pUusi->iVariBitti = 0; // Uudet solmut aina punaisia.
00026     strcpy(pUusi->aNimi, pNimi);
00027     pUusi->iArvo = iArvo;
00028     pUusi->pVasen = NULL;
00029     pUusi->pOikea = NULL;
00030     pUusi->pVanhempi = NULL;
00031     return (pUusi);
00032 }
00033
00040 RBSOLMU *vapautaMuistiRB(RBSOLMU *pJuuriSolmu) {
00041     if (pJuuriSolmu != NULL) {
00042         vapautaMuistiRB(pJuuriSolmu->pVasen);
00043         vapautaMuistiRB(pJuuriSolmu->pOikea);
00044         free(pJuuriSolmu);
00045     }
00046     pJuuriSolmu = NULL;
00047     return (pJuuriSolmu);
00048 }
00049
00057 RBSOLMU *lisaaRBSolmu(RBSOLMU *pJuuriSolmu, RBSOLMU *pUusi) {
00058     int iVertailu = 0;
00059
00060     // Jos puu on tyhjä, palautetaan uusi RBSolmu.
00061     if (pJuuriSolmu == NULL) {
00062         return pUusi;
00063     }
00064
00065     iVertailu = strcmp(pUusi->aNimi, pJuuriSolmu->aNimi);
00066
00067     // Solmujen lisääminen rekursiivisesti
00068     if (pUusi->iArvo < pJuuriSolmu->iArvo) {
00069         pJuuriSolmu->pVasen = lisaaRBSolmu(pJuuriSolmu->pVasen, pUusi);
00070         pJuuriSolmu->pVasen->pVanhempi = pJuuriSolmu;
00071     } else if (pUusi->iArvo > pJuuriSolmu->iArvo) {
00072         pJuuriSolmu->pOikea = lisaaRBSolmu(pJuuriSolmu->pOikea, pUusi);
00073         pJuuriSolmu->pOikea->pVanhempi = pJuuriSolmu;
00074     } else if (pUusi->iArvo == pJuuriSolmu->iArvo) {
00075         if (iVertailu < 0) {
00076             pJuuriSolmu->pVasen = lisaaRBSolmu(pJuuriSolmu->pVasen, pUusi);
00077             pJuuriSolmu->pVasen->pVanhempi = pJuuriSolmu;
00078         } else if (iVertailu > 0) {
00079             pJuuriSolmu->pOikea = lisaaRBSolmu(pJuuriSolmu->pOikea, pUusi);
00080             pJuuriSolmu->pOikea->pVanhempi = pJuuriSolmu;
00081         }
00082     }
00083     return (pJuuriSolmu);
00084 }
00085
00095 RBSOLMU *luoRBPuu(RBSOLMU *pJuuriSolmu, char *pNimi) {
00096     FILE *Tiedosto = NULL;
00097     char aRivi[LEN] = "";
00098     int iOtsikko = 0;
00099     char *p1 = NULL, *p2 = NULL;
00100     int iArvo = 0;
00101
00102     if ((Tiedosto = fopen(pNimi, "r")) == NULL) {
00103         perror("Tiedoston avaaminen epäonnistui, lopetetaan");
00104         exit(0);
00105     }
00106
00107     // Luetaan, pilkotaan ja käsitellään tiedosto.
00108     while ((fgets(aRivi, LEN, Tiedosto)) != NULL) {
00109         aRivi[strlen(aRivi) - 1] = '\0';
00110         if (iOtsikko == 0) {
00111             iOtsikko++;

```



```

00112         continue;
00113     }
00114
00115     if ((p1 = strtok(aRivi, ";")) == NULL) {
00116         perror("Tiedoston pilkkominen epäonnistui, lopetetaan");
00117         exit(0);
00118     }
00119
00120     if ((p2 = strtok(NULL, ";")) == NULL) {
00121         perror("Tiedoston pilkkominen epäonnistui, lopetetaan");
00122         exit(0);
00123     }
00124
00125     iArvo = atoi(p2);
00126
00127     RBSOLMU *pUusi = varaaMuistiaRB(p1, iArvo);
00128     pJuurisolmu = lisaaRBSolmu(pJuurisolmu, pUusi);
00129     korjaaLisays(&pJuurisolmu, pUusi);
00130 }
00131
00132 fclose(Tiedosto);
00133 return (pJuurisolmu);
00134 }
00135
00142 void kierraVasemmalle(RBSOLMU **pJuurisolmu, RBSOLMU *pSolmu) {
00143     RBSOLMU *pOikeaLapsi = pSolmu->pOikea;
00144     pSolmu->pOikea = pOikeaLapsi->pVasen;
00145
00146     if (pSolmu->pOikea != NULL)
00147         pSolmu->pOikea->pVanhempi = pSolmu;
00148
00149     // Solmun oikea lapsi nousee ylös puussa.
00150     pOikeaLapsi->pVanhempi = pSolmu->pVanhempi;
00151
00152     if (pSolmu->pVanhempi == NULL)
00153         *pJuurisolmu = pOikeaLapsi;
00154     else if (pSolmu == pSolmu->pVanhempi->pVasen)
00155         pSolmu->pVanhempi->pVasen = pOikeaLapsi;
00156     else
00157         pSolmu->pVanhempi->pOikea = pOikeaLapsi;
00158
00159     pOikeaLapsi->pVasen = pSolmu;
00160     pSolmu->pVanhempi = pOikeaLapsi;
00161     return;
00162 }
00163
00170 void kierraOikealle(RBSOLMU **pJuurisolmu, RBSOLMU *pSolmu) {
00171     RBSOLMU *pVasenLapsi = pSolmu->pVasen;
00172     pSolmu->pVasen = pVasenLapsi->pOikea;
00173
00174     if (pSolmu->pVasen != NULL)
00175         pSolmu->pVasen->pVanhempi = pSolmu;
00176
00177     pVasenLapsi->pVanhempi = pSolmu->pVanhempi;
00178
00179     if (pSolmu->pVanhempi == NULL)
00180         *pJuurisolmu = pVasenLapsi;
00181     else if (pSolmu == pSolmu->pVanhempi->pVasen)
00182         pSolmu->pVanhempi->pVasen = pVasenLapsi;
00183     else
00184         pSolmu->pVanhempi->pOikea = pVasenLapsi;
00185
00186     pVasenLapsi->pOikea = pSolmu;
00187     pSolmu->pVanhempi = pVasenLapsi;
00188
00189     return;
00190 }
00191
00198 void korjaaLisays(RBSOLMU **pJuurisolmu, RBSOLMU *pUusi) {
00199     RBSOLMU *vanhempi = NULL;
00200     RBSOLMU *seta = NULL;
00201     RBSOLMU *isoVanhempi = NULL;
00202
00203     // Pyöritään while:ssa kunnes päädytään juurisolmuun tai vanhempi on mustan värinen.
00204     while ((pUusi != *pJuurisolmu) && (pUusi->pVanhempi->iVariBitti == 0)) {
00205         vanhempi = pUusi->pVanhempi;
00206         isoVanhempi = vanhempi->pVanhempi;
00207
00208         // Vanhempi on isovanhemman vasen lapsi.
00209         if (vanhempi == isoVanhempi->pVasen) {
00210             seta = isoVanhempi->pOikea;
00211
00212             // Tarkastetaan onko setä punainen.
00213             if (seta != NULL && seta->iVariBitti == 0) {
00214                 vanhempi->iVariBitti = 1;
00215                 seta->iVariBitti = 1;
00216                 isoVanhempi->iVariBitti = 0;

```

```

00217         pUusi = isoVanhempi;
00218     } else {
00219
00220         // Tarkastetaan onko uusi solmu vanhemman solmun oikea lapsi solmu.
00221         if (pUusi == vanhempi->pOikea) {
00222             pUusi = vanhempi;
00223             kierraVasemmalle(pJuurisolmu, pUusi);
00224             vanhempi = pUusi->pVanhempi;
00225         }
00226         vanhempi->iVariBitti = 1;
00227         isoVanhempi->iVariBitti = 0;
00228         kierraOikealle(pJuurisolmu, isoVanhempi);
00229     }
00230
00231 } else {
00232     seta = isoVanhempi->pVasen;
00233
00234     // Tarkastetaan onko setä punainen.
00235     if (seta != NULL && seta->iVariBitti == 0) {
00236         vanhempi->iVariBitti = 1;
00237         seta->iVariBitti = 1;
00238         isoVanhempi->iVariBitti = 0;
00239         pUusi = isoVanhempi;
00240     } else {
00241
00242         // Tarkastetaan onko uusi solmu vanhemman solmun vasen lapsi solmu.
00243         if (pUusi == vanhempi->pVasen) {
00244             pUusi = vanhempi;
00245             kierraOikealle(pJuurisolmu, pUusi);
00246             vanhempi = pUusi->pVanhempi;
00247         }
00248         vanhempi->iVariBitti = 1;
00249         isoVanhempi->iVariBitti = 0;
00250         kierraVasemmalle(pJuurisolmu, isoVanhempi);
00251     }
00252 }
00253 }
00254 (*pJuurisolmu)->iVariBitti = 1;
00255
00256 return;
00257 }
00258
00265 void kirjoitaRBTiedostoon(char *pNimi, RBSOLMU *pJuurisolmu) {
00266     FILE *Tiedosto = NULL;
00267
00268     if (pJuurisolmu == NULL) {
00269         return;
00270     }
00271
00272     /* Tiedoston avaaminen. */
00273     if ((Tiedosto = fopen(pNimi, "a")) == NULL) {
00274         perror("Tiedoston avaaminen epäonnistui, lopetetaan");
00275         exit(0);
00276     }
00277     /* Tiedostoon kirjoittaminen. */
00278     fprintf(Tiedosto, "%s,%d\n", pJuurisolmu->aNimi, pJuurisolmu->iArvo);
00279
00280     /* Tiedoston sulkeminen. */
00281     fclose(Tiedosto);
00282     return;
00283 }
00284
00291 void kirjoitaRB(char *pNimi, RBSOLMU *pJuurisolmu) {
00292     if (pJuurisolmu != NULL) {
00293         kirjoitaRBTiedostoon(pNimi, pJuurisolmu);
00294         kirjoitaRB(pNimi, pJuurisolmu->pVasen);
00295         kirjoitaRB(pNimi, pJuurisolmu->pOikea);
00296     }
00297     return;
00298 }

```

3.17 punamustapuu.h File Reference

```
#include "yleiset.h"
```

Classes

- struct [RBSolmu](#)

Typedefs

- typedef struct [RBSolmu](#) [RBSOLMU](#)

Functions

- [RBSOLMU](#) * [varaaMuistiaRB](#) (char *pNimi, int iArvo)
Varaa muistia punamustalle puulle.
- [RBSOLMU](#) * [vapautaMuistiRB](#) ([RBSOLMU](#) *pJuuriSolmu)
Vapauttaa muistin punamustasta puusta.
- [RBSOLMU](#) * [lisaaRBSolmu](#) ([RBSOLMU](#) *pJuuriSolmu, [RBSOLMU](#) *pUusi)
Lisää solmun punamustapuuhun.
- [RBSOLMU](#) * [luoRBPuu](#) ([RBSOLMU](#) *pJuuriSolmu, char *pNimi)
Luo punamustapuun.
- void [kierraVasemmalle](#) ([RBSOLMU](#) **pJuuriSolmu, [RBSOLMU](#) *pSolmu)
Tekee vasemman rotaation punamustalle puulle.
- void [kierraOikealle](#) ([RBSOLMU](#) **pJuuriSolmu, [RBSOLMU](#) *pSolmu)
Tekee oikean rotaation punamustalle puulle.
- void [korjaaLisays](#) ([RBSOLMU](#) **pJuuriSolmu, [RBSOLMU](#) *pUusi)
Korjaa lisäyksen jälkeen punamustan puun rakenteen ja värit oikeiksi.
- void [kirjoitaRBTiedostoon](#) (char *pNimi, [RBSOLMU](#) *pJuuriSolmu)
Kirjoittaa punamustapuun tiedostoon.
- void [kirjoitaRB](#) (char *pNimi, [RBSOLMU](#) *pJuuriSolmu)
Kirjoittaa punamustapuun.

3.17.1 Typedef Documentation

3.17.1.1 RBSOLMU

```
typedef struct RBSolmu RBSOLMU
```

3.17.2 Function Documentation

3.17.2.1 kierraOikealle()

```
void kierraOikealle (
    RBSOLMU ** pJuuriSolmu,
    RBSOLMU * pSolmu)
```

Parameters

<i>pJuuriSolmu</i>	Osoitin juurisolmun osoittimeen.
<i>pSolmu</i>	Osoitin solmuun. Kierto tehdään tämän solmun ympärille.

Definition at line 170 of file [punamustapuu.c](#).

```
00170                                     {
00171     RBSOLMU *pVasenLapsi = pSolmu->pVasen;
00172     pSolmu->pVasen = pVasenLapsi->pOikea;
```

```

00173
00174     if (pSolmu->pVasen != NULL)
00175         pSolmu->pVasen->pVanhempi = pSolmu;
00176
00177     pVasenLapsi->pVanhempi = pSolmu->pVanhempi;
00178
00179     if (pSolmu->pVanhempi == NULL)
00180         *pJuurisolmu = pVasenLapsi;
00181     else if (pSolmu == pSolmu->pVanhempi->pVasen)
00182         pSolmu->pVanhempi->pVasen = pVasenLapsi;
00183     else
00184         pSolmu->pVanhempi->pOikea = pVasenLapsi;
00185
00186     pVasenLapsi->pOikea = pSolmu;
00187     pSolmu->pVanhempi = pVasenLapsi;
00188
00189     return;
00190 }

```

3.17.2.2 kierraVasemmalle()

```

void kierraVasemmalle (
    RBSOLMU ** pJuurisolmu,
    RBSOLMU * pSolmu)

```

Parameters

<i>pJuurisolmu</i>	Osoitin juurisolmun osoittimeen.
<i>pSolmu</i>	Osoitin solmuun. Kierto tehdään tämän solmun ympärille.

Definition at line 142 of file [punamustapuu.c](#).

```

00142
00143     RBSOLMU *pOikeaLapsi = pSolmu->pOikea;
00144     pSolmu->pOikea = pOikeaLapsi->pVasen;
00145
00146     if (pSolmu->pOikea != NULL)
00147         pSolmu->pOikea->pVanhempi = pSolmu;
00148
00149     // Solmun oikea lapsi nousee ylös puussa.
00150     pOikeaLapsi->pVanhempi = pSolmu->pVanhempi;
00151
00152     if (pSolmu->pVanhempi == NULL)
00153         *pJuurisolmu = pOikeaLapsi;
00154     else if (pSolmu == pSolmu->pVanhempi->pVasen)
00155         pSolmu->pVanhempi->pVasen = pOikeaLapsi;
00156     else
00157         pSolmu->pVanhempi->pOikea = pOikeaLapsi;
00158
00159     pOikeaLapsi->pVasen = pSolmu;
00160     pSolmu->pVanhempi = pOikeaLapsi;
00161     return;
00162 }

```

3.17.2.3 kirjoitaRB()

```

void kirjoitaRB (
    char * pNimi,
    RBSOLMU * pJuurisolmu)

```

Kutsuu tiedostoon kirjoitettavaa aliohjelman ja antaa sille kirjoitettavat arvot.

Parameters

<i>pNimi</i>	Osoitin kirjoitettavan tiedoston nimeen.
--------------	--

<i>pJuurisolmu</i>	Osoitin solmuun, jota aliohjelma käsittelee.
--------------------	--

Definition at line 291 of file [punamustapuu.c](#).

```

00291                                     {
00292     if (pJuurisolmu != NULL) {
00293         kirjoitaRBTiedostoon(pNimi, pJuurisolmu);
00294         kirjoitaRB(pNimi, pJuurisolmu->pVasen);
00295         kirjoitaRB(pNimi, pJuurisolmu->pOikea);
00296     }
00297     return;
00298 }
```

3.17.2.4 kirjoitaRBTiedostoon()

```

void kirjoitaRBTiedostoon (
    char * pNimi,
    RBSOLMU * pJuurisolmu)
```

Parameters

<i>pNimi</i>	Osoitin kirjoitettavan tiedoston nimeen.
<i>pJuurisolmu</i>	Osoitin punamustapuuun juurisolmuun.

Definition at line 265 of file [punamustapuu.c](#).

```

00265                                     {
00266     FILE *Tiedosto = NULL;
00267
00268     if (pJuurisolmu == NULL) {
00269         return;
00270     }
00271
00272     /* Tiedoston avaaminen. */
00273     if ((Tiedosto = fopen(pNimi, "a")) == NULL) {
00274         perror("Tiedoston avaaminen epäonnistui, lopetetaan");
00275         exit(0);
00276     }
00277     /* Tiedostoon kirjoittaminen. */
00278     fprintf(Tiedosto, "%s,%d\n", pJuurisolmu->aNimi, pJuurisolmu->iArvo);
00279
00280     /* Tiedoston sulkeminen. */
00281     fclose(Tiedosto);
00282     return;
00283 }
```

3.17.2.5 korjaaLisays()

```

void korjaaLisays (
    RBSOLMU ** pJuurisolmu,
    RBSOLMU * pUusi)
```

Parameters

<i>pJuurisolmu</i>	Osoitin juurisolmun osoittimeen.
<i>pUusi</i>	Osoitin lisättyyn solmuun.

Definition at line 198 of file [punamustapuu.c](#).

```

00198                                     {
00199     RBSOLMU *vanhempi = NULL;
00200     RBSOLMU *seta = NULL;
```

```

00201     RBSOLMU *isoVanhemp1 = NULL;
00202
00203     // Pyöritään while:ssä kunnes päädytään juurisolmuun tai vanhemp1 on mustan värinen.
00204     while ((pUusi != *pJuurisolmu) && (pUusi->pVanhemp1->iVariBitti == 0)) {
00205         vanhemp1 = pUusi->pVanhemp1;
00206         isoVanhemp1 = vanhemp1->pVanhemp1;
00207
00208         // Vanhemp1 on isovanhemman vasen lapsi.
00209         if (vanhemp1 == isoVanhemp1->pVasen) {
00210             seta = isoVanhemp1->pOikea;
00211
00212             // Tarkastetaan onko setä punainen.
00213             if (seta != NULL && seta->iVariBitti == 0) {
00214                 vanhemp1->iVariBitti = 1;
00215                 seta->iVariBitti = 1;
00216                 isoVanhemp1->iVariBitti = 0;
00217                 pUusi = isoVanhemp1;
00218             } else {
00219
00220                 // Tarkastetaan onko uusi solmu vanhemman solmun oikea lapsi solmu.
00221                 if (pUusi == vanhemp1->pOikea) {
00222                     pUusi = vanhemp1;
00223                     kierraVasemmalle(pJuurisolmu, pUusi);
00224                     vanhemp1 = pUusi->pVanhemp1;
00225                 }
00226                 vanhemp1->iVariBitti = 1;
00227                 isoVanhemp1->iVariBitti = 0;
00228                 kierraOikealle(pJuurisolmu, isoVanhemp1);
00229             }
00230         } else {
00231             seta = isoVanhemp1->pVasen;
00232
00233             // Tarkastetaan onko setä punainen.
00234             if (seta != NULL && seta->iVariBitti == 0) {
00235                 vanhemp1->iVariBitti = 1;
00236                 seta->iVariBitti = 1;
00237                 isoVanhemp1->iVariBitti = 0;
00238                 pUusi = isoVanhemp1;
00239             } else {
00240
00241                 // Tarkastetaan onko uusi solmu vanhemman solmun vasen lapsi solmu.
00242                 if (pUusi == vanhemp1->pVasen) {
00243                     pUusi = vanhemp1;
00244                     kierraOikealle(pJuurisolmu, pUusi);
00245                     vanhemp1 = pUusi->pVanhemp1;
00246                 }
00247                 vanhemp1->iVariBitti = 1;
00248                 isoVanhemp1->iVariBitti = 0;
00249                 kierraVasemmalle(pJuurisolmu, isoVanhemp1);
00250             }
00251         }
00252     }
00253     (*pJuurisolmu)->iVariBitti = 1;
00254
00255     return;
00256 }
00257

```

3.17.2.6 lisaaRBSolmu()

```

RBSOLMU * lisaaRBSolmu (
    RBSOLMU * pJuurisolmu,
    RBSOLMU * pUusi)

```

Parameters

<i>pJuurisolmu</i>	Osoitin punamustapuun juurisolmuun.
<i>pUusi</i>	Osoitin lisättävään solmuun.

Returns

RBSOLMU* Palauttaa osoittimen punamustapuun juurisolmuun.

Katsotaan, ovatko arvot samat. Laitetaan aakkosten perusteella vasemmalle tai oikealle.

Definition at line 57 of file [punamustapuu.c](#).

```

00057                                     {
00058     int iVertailu = 0;
00059
00060     // Jos puu on tyhjä, palautetaan uusi RBSolmu.
00061     if (pJuurisolmu == NULL) {
00062         return pUusi;
00063     }
00064
00065     iVertailu = strcmp(pUusi->aNimi, pJuurisolmu->aNimi);
00066
00067     // Solmujen lisääminen rekursiivisesti
00068     if (pUusi->iArvo < pJuurisolmu->iArvo) {
00069         pJuurisolmu->pVasen = lisaaRBSolmu(pJuurisolmu->pVasen, pUusi);
00070         pJuurisolmu->pVasen->pVanhempi = pJuurisolmu;
00071     } else if (pUusi->iArvo > pJuurisolmu->iArvo) {
00072         pJuurisolmu->pOikea = lisaaRBSolmu(pJuurisolmu->pOikea, pUusi);
00073         pJuurisolmu->pOikea->pVanhempi = pJuurisolmu;
00074     } else if (pUusi->iArvo == pJuurisolmu->iArvo) {
00075         if (iVertailu < 0) {
00076             pJuurisolmu->pVasen = lisaaRBSolmu(pJuurisolmu->pVasen, pUusi);
00077             pJuurisolmu->pVasen->pVanhempi = pJuurisolmu;
00078         } else if (iVertailu > 0) {
00079             pJuurisolmu->pOikea = lisaaRBSolmu(pJuurisolmu->pOikea, pUusi);
00080             pJuurisolmu->pOikea->pVanhempi = pJuurisolmu;
00081         }
00082     }
00083     return (pJuurisolmu);
00084 }
00085
00086 }
```

3.17.2.7 luoRBPuu()

```

RBSOLMU * luoRBPuu (
    RBSOLMU * pJuurisolmu,
    char * pNimi)
```

Lukee tiedoston, käsittelee tiedot, sekä luo puun.

Parameters

<i>pJuurisolmu</i>	Osoitin punamustapuun solmuun.
<i>pNimi</i>	Osoitin luettava tiedoston nimeen.

Returns

RBSOLMU* Palauttaa solmun osoittimen.

Definition at line 95 of file [punamustapuu.c](#).

```

00095                                     {
00096     FILE *Tiedosto = NULL;
00097     char aRivi[LEN] = "";
00098     int iOtsikko = 0;
00099     char *p1 = NULL, *p2 = NULL;
00100     int iArvo = 0;
00101
00102     if ((Tiedosto = fopen(pNimi, "r")) == NULL) {
00103         perror("Tiedoston avaaminen epäonnistui, lopetetaan");
00104         exit(0);
00105     }
00106
00107     // Luetaan, pilkotaan ja käsitellään tiedosto.
00108     while ((fgets(aRivi, LEN, Tiedosto)) != NULL) {
00109         aRivi[strlen(aRivi) - 1] = '\0';
00110         if (iOtsikko == 0) {
00111             iOtsikko++;
00112             continue;
00113         }
```

```

00114
00115     if ((p1 = strtok(aRivi, ";")) == NULL) {
00116         perror("Tiedoston pilkkominen epäonnistui, lopetetaan");
00117         exit(0);
00118     }
00119
00120     if ((p2 = strtok(NULL, ";")) == NULL) {
00121         perror("Tiedoston pilkkominen epäonnistui, lopetetaan");
00122         exit(0);
00123     }
00124
00125     iArvo = atoi(p2);
00126
00127     RBSOLMU *pUusi = varaaMuistiaRB(p1, iArvo);
00128     pJuuriSolmu = lisaaRBSolmu(pJuuriSolmu, pUusi);
00129     korjaaLisays(&pJuuriSolmu, pUusi);
00130 }
00131
00132 fclose(Tiedosto);
00133 return (pJuuriSolmu);
00134 }

```

3.17.2.8 vapautaMuistiRB()

```

RBSOLMU * vapautaMuistiRB (
    RBSOLMU * pJuuriSolmu)

```

Parameters

<i>pJuuriSolmu</i>	Osoitin punamustapuun solmuun.
--------------------	--------------------------------

Returns

RBSOLMU* Palauttaa punamustapuun osoittimen.

Definition at line 40 of file [punamustapuu.c](#).

```

00040
00041     if (pJuuriSolmu != NULL) {
00042         vapautaMuistiRB(pJuuriSolmu->pVasen);
00043         vapautaMuistiRB(pJuuriSolmu->pOikea);
00044         free(pJuuriSolmu);
00045     }
00046     pJuuriSolmu = NULL;
00047     return (pJuuriSolmu);
00048 }

```

3.17.2.9 varaaMuistiaRB()

```

RBSOLMU * varaaMuistiaRB (
    char * pNimi,
    int iArvo)

```

Parameters

<i>pNimi</i>	Solmun nimi, jolle varataan muistia ja kopioidaan uuteen solmuun.
<i>iArvo</i>	Solmun arvo, jolle varataan muistia ja kopioidaan uuteen solmuun.

Returns

RBSOLMU* Palauttaa osoittimen uuteen alustettuun solmuun.

Definition at line 15 of file [punamustapuu.c](#).

```

00015                                     {
00016     RBSOLMU *pUusi = NULL;
00017
00018     // Muistin varaaminen
00019     if ((pUusi = (RBSOLMU *)malloc(sizeof(RBSOLMU))) == NULL) {
00020         perror("Muistin varaus epäonnistui, lopetetaan");
00021         exit(0);
00022     }
00023
00024     // Alustetaan uusi solmu.
00025     pUusi->iVariBitti = 0; // Uudet solmut aina punaisia.
00026     strcpy(pUusi->aNimi, pNimi);
00027     pUusi->iArvo = iArvo;
00028     pUusi->pVasen = NULL;
00029     pUusi->pOikea = NULL;
00030     pUusi->pVanhempi = NULL;
00031     return (pUusi);
00032 }
```

3.18 punamustapuu.h

[Go to the documentation of this file.](#)

```

00001 #ifndef PUNAMUSTAPUU_H
00002 #define PUNAMUSTAPUU_H
00003 #include "yleiset.h"
00004
00005 typedef struct RBSolmu {
00006     char aNimi[LEN];
00007     int iArvo;
00008     int iVariBitti;
00009     struct RBSolmu *pOikea;
00010     struct RBSolmu *pVasen;
00011     struct RBSolmu *pVanhempi;
00012 } RBSOLMU;
00013
00014
00015 RBSOLMU *varaaMuistiRB(char *pNimi, int iArvo);
00016 RBSOLMU *vapautaMuistiRB(RBSOLMU *pJuuriSolmu);
00017 RBSOLMU *lisaaRBSolmu(RBSOLMU *pJuuriSolmu, RBSOLMU *pUusi);
00018 RBSOLMU *luoRBPuu(RBSOLMU *pJuuriSolmu, char *pNimi);
00019 void kierraVasemmalle(RBSOLMU **pJuuriSolmu, RBSOLMU *pSolmu);
00020 void kierraOikealle(RBSOLMU **pJuuriSolmu, RBSOLMU *pSolmu);
00021 void korjaaLisays(RBSOLMU **pJuuriSolmu, RBSOLMU *pUusi);
00022 void kirjoitaRBTiedostoon(char *pNimi, RBSOLMU *pJuuriSolmu);
00023 void kirjoitaRB(char *pNimi, RBSOLMU *pJuuriSolmu);
00024
00025 #endif
```

3.19 yleiset.c File Reference

```

#include "yleiset.h"
#include "binaaripuu.h"
#include "haut.h"
#include "linkitettylista.h"
#include "punamustapuu.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

Functions

- void [mainLista](#) (void)
Tekee käyttäjän valinnan perusteella listan toiminnot.
- void [mainBinaaripuu](#) (void)
Tekee käyttäjän valinnan perusteella binääripuun toiminnot.
- int [valikko](#) (void)
Ensimmäinen valikko, jossa käyttäjä valitsee listan tai binääripuun.
- int [listaValikko](#) (void)
Tulostaa lista valikon ja kysyy käyttäjän valinnan.
- int [binaaripuuValikko](#) (void)
Tulostaa binaariluku valikon ja kysyy käyttäjältä valinnan.
- char * [kysyNimi](#) (char *pPrompti, char *pNimi)
Kysyy tiedoston/haettavan nimen.
- int [kysyArvo](#) (char *pPrompti, int iArvo)
Kysyy arvon, joka halutaan etsiä/poistaa puusta.

3.19.1 Function Documentation

3.19.1.1 binaaripuuValikko()

```
int binaaripuuValikko (
    void )
```

Parameters

<i>iValinta</i>	Käyttäjän antama syöte vaihtoehtojen pohjalta.
-----------------	--

Returns

int Palauttaa käyttäjän valinnan.

Definition at line 245 of file [yleiset.c](#).

```
00245     {
00246         int iValinta = 0;
00247         printf("\nValitse haluamasi toiminto:\n");
00248         printf("1) Luo puu\n");
00249         printf("2) Kirjoita AVL puu tiedostoon\n");
00250         printf("3) Syvyyshaku\n");
00251         printf("4) Leveyshaku\n");
00252         printf("5) Tyhjennä puu\n");
00253         printf("6) Poista solmu\n");
00254         printf("7) Binäärihaku AVL puusta\n");
00255         printf("8) Punamustapuu\n");
00256         printf("0) Takaisin\n");
00257         printf("Anna valintasi: ");
00258         scanf("%d", &iValinta);
00259         getchar();
00260         return iValinta;
00261     }
```

3.19.1.2 kysyArvo()

```
int kysyArvo (  
    char * pPrompti,  
    int iArvo)
```

Parameters

<i>pPrompti</i>	Kysymys esim. "Anna haettava numeroarvo: "
<i>iArvo</i>	Arvo, joka halutaan poistaa/etsiä.

Returns

int Palauttaa käyttäjän antaman arvon.

Definition at line 284 of file [yleiset.c](#).

```
00284                                     {  
00285     printf("%s", pPrompti);  
00286     scanf("%d", &iArvo);  
00287     getchar();  
00288     return (iArvo);  
00289 }
```

3.19.1.3 kysyNimi()

```
char * kysyNimi (  
    char * pPrompti,  
    char * pNimi)
```

Parameters

<i>pPrompti</i>	Kysymys esim. "Anna kirjoitettavan tiedoston nimi: "
<i>pNimi</i>	Tiedoston nimi/haettava nimi, jota halutaan käyttää.

Returns

char Palauttaa käyttäjän antaman merkkijonon.

Definition at line 270 of file [yleiset.c](#).

```
00270                                     {  
00271     printf("%s", pPrompti);  
00272     scanf("%s", pNimi);  
00273     getchar();  
00274     return (pNimi);  
00275 }
```

3.19.1.4 listaValikko()

```
int listaValikko (
    void )
```

Parameters

<i>iValinta</i>	Käyttäjän antama syöte vaihtoehtojen pohjalta.
-----------------	--

Returns

int Palauttaa käyttäjän valinnan.

Definition at line 221 of file [yleiset.c](#).

```
00221         {
00222     int iValinta = 0;
00223     printf("\nValitse haluamasi toiminto:\n");
00224     printf("1) Lue tiedosto\n");
00225     printf("2) Kirjoita tiedosto alusta loppuun\n");
00226     printf("3) Kirjoita tiedosto lopusta alkuun\n");
00227     printf("4) Tyhjennä taulukko\n");
00228     printf("5) Lajittele nousevasti\n");
00229     printf("6) Lajittele laskevasti\n");
00230     printf("7) Lisää tietoja listaan\n");
00231     printf("8) Poista listalta tietoja\n");
00232     printf("0) Takaisin\n");
00233     printf("Anna valintasi: ");
00234     scanf("%d", &iValinta);
00235     getchar();
00236     return iValinta;
00237 }
```

3.19.1.5 mainBinaaripuu()

```
void mainBinaaripuu (
    void )
```

Funktio näyttää valikon ja suorittaa käyttäjän valinnan mukaiset binääripuuhun liittyvät toiminnot.

Definition at line 101 of file [yleiset.c](#).

```
00101     {
00102     char aNimiLuettava[LEN] = "";
00103     char aNimiKirjoitettava[LEN] = "";
00104     char aHaettavaNimi[LEN] = "";
00105     PUU *pJuuriSolmu = NULL;
00106     RBSOLMU *pRBJuuri = NULL;
00107     int iValinta = 0;
00108     int iArvo = 0;
00109
00110     do {
00111         iValinta = binaaripuuValikko();
00112         if (iValinta == 1) {
00113             // Luo AVL puun.
00114             kysyNimi("Tiedoston nimi: ", aNimiLuettava);
00115             pJuuriSolmu = luoPuu(aNimiLuettava, pJuuriSolmu);
00116
00117         } else if (iValinta == 2) {
00118             // Kirjoittaa AVL puun tiedostoon.
00119             if (pJuuriSolmu == NULL) {
00120                 printf("Luo binääripuu ennen sen kirjoittamista.\n");
00121             } else {
00122                 kysyNimi("Anna kirjoitettavan tiedoston nimi: ", aNimiKirjoitettava);
00123                 kirjoitaBinaaripuu(aNimiKirjoitettava, pJuuriSolmu);
00124             }
00125
00126         } else if (iValinta == 3) {
00127             // Syvyyshaku binääripuulle.
00128             if (pJuuriSolmu == NULL) {
00129                 printf("Luo binääripuu ennen syvyyshakua.\n");
00130             } else {
```

```

00131         iArvo = kysyArvo("Anna haettava numeroarvo: ", iArvo);
00132         kysyNimi("Anna kirjoitettavan tiedoston nimi: ", aNimiKirjoitettava);
00133         tarkistaLoytyykoSyvvyysaulla(aNimiKirjoitettava, pJuuriSolmu, iArvo);
00134     }
00135
00136     } else if (iValinta == 4) {
00137         // Leveyshaku binääripuulle.
00138         if (pJuuriSolmu == NULL) {
00139             printf("Luo binääripuu ennen leveyshakua.\n");
00140         } else {
00141             kysyNimi("Anna haettava nimi: ", aHaettavaNimi);
00142             kysyNimi("Anna kirjoitettavan tiedoston nimi: ", aNimiKirjoitettava);
00143             leveyshaku(aNimiKirjoitettava, pJuuriSolmu, aHaettavaNimi);
00144         }
00145
00146     } else if (iValinta == 5) {
00147         // Vapauttaa muistin.
00148         pJuuriSolmu = vapautaMuistiPuu(pJuuriSolmu);
00149         printf("Muisti vapautettu.\n");
00150
00151     } else if (iValinta == 6) {
00152         // Poistaa solmun binääripuusta.
00153         if (pJuuriSolmu == NULL) {
00154             printf("Luo binääripuu ennen poistoa.\n");
00155         } else {
00156             kysyNimi("Anna poistettava nimi tai arvo: ", aHaettavaNimi);
00157             pJuuriSolmu = poistaSolmu(aHaettavaNimi, pJuuriSolmu);
00158         }
00159
00160     } else if (iValinta == 7) {
00161         // Binäärihaku
00162         if (pJuuriSolmu == NULL) {
00163             printf("Luo binääripuu ennen binäärihakua.\n");
00164         } else {
00165             kysyNimi("Anna kirjoitettavan tiedoston nimi: ", aNimiKirjoitettava);
00166             iArvo = kysyArvo("Anna haettava numeroarvo: ", iArvo);
00167             iArvo = binaarihaku(aNimiKirjoitettava, pJuuriSolmu, iArvo);
00168             if (iArvo == 0) {
00169                 printf("Hakemaasi arvoa ei löytynyt binääripuusta.\n");
00170             }
00171         }
00172
00173     } else if (iValinta == 8) {
00174         // Toteuttaa punamustapuun.
00175         kysyNimi("Anna luettavan tiedoston nimi: ", aNimiLuettava);
00176         kysyNimi("Anna kirjoitettavan tiedoston nimi: ", aNimiKirjoitettava);
00177         pRBJuuri = luoRBPuu(pRBJuuri, aNimiLuettava);
00178         kirjoitaRB(aNimiKirjoitettava, pRBJuuri);
00179         printf("Tiedosto '%s' kirjoitettu.\n", aNimiKirjoitettava);
00180         pRBJuuri = vapautaMuistiRB(pRBJuuri);
00181
00182     } else if (iValinta == 0) {
00183         printf("Palataan takaisin päävalikkoon.\n");
00184
00185     } else {
00186         printf("Tuntematon valinta, yritä uudestaan.\n");
00187     }
00188 } while (iValinta != 0);
00189
00190 /* Vapautetaan puun varaama muisti varmuuden vuoksi vielä täällä lopussa erikseen, jos
00191  * käyttäjä ei muista sitä tehdä. */
00192
00193 pJuuriSolmu = vapautaMuistiPuu(pJuuriSolmu);
00194 return;
00195 }

```

3.19.1.6 mainLista()

```

void mainLista (
    void )

```

Funktio näyttää valikon ja suorittaa käyttäjän valinnan mukaiset listaan liittyvät toiminnot.

Definition at line 18 of file [yleiset.c](#).

```

00018     {
00019         char aNimiLuettava[LEN] = "";
00020         char aNimiKirjoitettava[LEN] = "";
00021         char aTietoPoistettava[LEN] = "";
00022         int iArvo = 0;
00023         int iIndeksi = 0;

```

```

00024     int iValinta = 0;
00025     TIEDOT *pAlku = NULL;
00026
00027     do {
00028         iValinta = listaValikko();
00029         if (iValinta == 1) {
00030             // Luekee tiedoston.
00031             kysyNimi("Anna luettavan tiedoston nimi: ", aNimiLuettava);
00032             pAlku = lue(aNimiLuettava, pAlku);
00033
00034         } else if (iValinta == 2) {
00035             // Kirjoittaa tiedoston alusta loppuun.
00036             if (pAlku == NULL) {
00037                 printf("Lue tiedosto ennen kirjoittamista.\n");
00038             } else {
00039                 kysyNimi("Anna kirjoitettavan tiedoston nimi: ", aNimiKirjoitettava);
00040                 kirjoitaTiedostoAlustaLoppuun(aNimiKirjoitettava, pAlku);
00041             }
00042
00043         } else if (iValinta == 3) {
00044             // Kirjoittaa tiedoston lopusta alkuun.
00045             if (pAlku == NULL) {
00046                 printf("Lue tiedosto ennen kirjoittamista.\n");
00047             } else {
00048                 kysyNimi("Anna kirjoitettavan tiedoston nimi: ", aNimiKirjoitettava);
00049                 kirjoitaTiedostoLopustaAlkuun(aNimiKirjoitettava, pAlku);
00050             }
00051
00052         } else if (iValinta == 4) {
00053             // Vapauttaa muistin,
00054             pAlku = vapautaMuisti(pAlku);
00055             printf("Muisti vapautettu.\n");
00056
00057         } else if (iValinta == 5) {
00058             // Lajittelee nousevasti lomistuslajittelulla.
00059             pAlku = lomitusLajittelu(pAlku);
00060             printf("Numerot lajiteltu nousevasti.\n");
00061
00062         } else if (iValinta == 6) {
00063             // Lajittelee kasjetavastu lisäyslajittelulla.
00064             pAlku = lisaysLajittelu(pAlku);
00065             printf("Numerot lajiteltu laskevasti.\n");
00066
00067         } else if (iValinta == 7) {
00068             // Lisää tietoja listaan.
00069             iIndeksi = kysyArvo("Mille riville haluat lisätä tietoja: ", iIndeksi);
00070             kysyNimi("Anna lisättävä nimi: ", aNimiKirjoitettava);
00071             iArvo = kysyArvo("Anna tämän nimisten lukumäärä: ", iArvo);
00072             pAlku = lisaalistaan(pAlku, iIndeksi, aNimiKirjoitettava, iArvo);
00073
00074         } else if (iValinta == 8) {
00075             // Poistaminen listalta.
00076             kysyNimi("Anna poistettava tieto: ", aTietoPoistettava);
00077             iArvo = onkoLukuVaiNimi(aTietoPoistettava);
00078             pAlku = poistaListastaAlkio(pAlku, iArvo, aTietoPoistettava);
00079
00080         } else if (iValinta == 0) {
00081             printf("Palataan takaisin päävalikkoon.\n");
00082
00083         } else {
00084             printf("Tuntematon valinta, yritä uudestaan.\n");
00085         }
00086     } while (iValinta != 0);
00087
00088     /* Vapautetaan listan varaama muisti varmuuden vuoksi vielä täällä lopussa erikseen, jos
00089     * käyttäjä ei muista sitä tehdä. */
00090
00091     pAlku = vapautaMuisti(pAlku);
00092     return;
00093 }

```

3.19.1.7 valikko()

```

int valikko (
    void )

```

Parameters

<i>iValinta</i>	Käyttäjän antama syöte.
-----------------	-------------------------

Returns

int Palauttaa käyttäjän valinnan.

Definition at line 203 of file [yleiset.c](#).

```
00203     {
00204     int iValinta = 0;
00205     printf("Valitse haluamasi toiminto:\n");
00206     printf("1) Lista\n");
00207     printf("2) Binääripuu\n");
00208     printf("0) Lopeta\n");
00209     printf("Anna valintasi: ");
00210     scanf("%d", &iValinta);
00211     getchar();
00212     return iValinta;
00213 }
```

3.20 yleiset.c

[Go to the documentation of this file.](#)

```
00001 #include "yleiset.h"
00002 #include "binaaripuu.h"
00003 #include "haut.h"
00004 #include "linkitettylista.h"
00005 #include "punamustapuu.h"
00006 #include <stdio.h>
00007 #include <stdlib.h>
00008 #include <string.h>
00009
00010 // Yleisiä ja usein käytettyjä aliohjelmia.
00011
00018 void mainLista(void) {
00019     char aNimiLuettava[LEN] = "";
00020     char aNimiKirjoitettava[LEN] = "";
00021     char aTietoPoistettava[LEN] = "";
00022     int iArvo = 0;
00023     int iIndeksi = 0;
00024     int iValinta = 0;
00025     TIEDOT *pAlku = NULL;
00026
00027     do {
00028         iValinta = listaValikko();
00029         if (iValinta == 1) {
00030             // Luekee tiedoston.
00031             kysyNimi("Anna luettavan tiedoston nimi: ", aNimiLuettava);
00032             pAlku = lue(aNimiLuettava, pAlku);
00033
00034         } else if (iValinta == 2) {
00035             // Kirjoittaa tiedoston alusta loppuun.
00036             if (pAlku == NULL) {
00037                 printf("Lue tiedosto ennen kirjoittamista.\n");
00038             } else {
00039                 kysyNimi("Anna kirjoitettavan tiedoston nimi: ", aNimiKirjoitettava);
00040                 kirjoitaTiedostoAlustaLoppuun(aNimiKirjoitettava, pAlku);
00041             }
00042
00043         } else if (iValinta == 3) {
00044             // Kirjoittaa tiedoston lopusta alkuun.
00045             if (pAlku == NULL) {
00046                 printf("Lue tiedosto ennen kirjoittamista.\n");
00047             } else {
00048                 kysyNimi("Anna kirjoitettavan tiedoston nimi: ", aNimiKirjoitettava);
00049                 kirjoitaTiedostoLopustaAlkuun(aNimiKirjoitettava, pAlku);
00050             }
00051
00052         } else if (iValinta == 4) {
00053             // Vapauttaa musitin,
00054             pAlku = vapautaMuisti(pAlku);
00055             printf("Muisti vapautettu.\n");
00056
00057         } else if (iValinta == 5) {
00058             // Lajittelee nousevasti lomistuslajittelulla.
00059             pAlku = lomitusLajittelu(pAlku);
00060             printf("Numerot lajiteltu nousevasti.\n");
00061
00062         } else if (iValinta == 6) {
00063             // Lajittelee kasjetavastu lisäyslajittelulla.
00064             pAlku = lisaysLajittelu(pAlku);
```

```

00065         printf("Numerot lajiteltu laskevasti.\n");
00066
00067     } else if (iValinta == 7) {
00068         // Lisää tietoja listaan.
00069         iIndeksi = kysyArvo("Mille riville haluat lisätä tietoja: ", iIndeksi);
00070         kysyNimi("Anna lisättävä nimi: ", aNimiKirjoitettava);
00071         iArvo = kysyArvo("Anna tämän nimisten lukumäärä: ", iArvo);
00072         pAlku = lisaalistaan(pAlku, iIndeksi, aNimiKirjoitettava, iArvo);
00073
00074     } else if (iValinta == 8) {
00075         // Poistaminen listalta.
00076         kysyNimi("Anna poistettava tieto: ", aTietoPoistettava);
00077         iArvo = onkoLukuVainNimi(aTietoPoistettava);
00078         pAlku = poistaListastaAlkio(pAlku, iArvo, aTietoPoistettava);
00079
00080     } else if (iValinta == 0) {
00081         printf("Palataan takaisin päävalikkoon.\n");
00082
00083     } else {
00084         printf("Tuntematon valinta, yritä uudestaan.\n");
00085     }
00086 } while (iValinta != 0);
00087
00088 /* Vapautetaan listan varaama muisti varmuuden vuoksi vielä täällä lopussa erikseen, jos
00089  * käyttäjä ei muista sitä tehdä. */
00090
00091 pAlku = vapautaMuisti(pAlku);
00092 return;
00093 }
00094
00101 void mainBinaaripuu(void) {
00102     char aNimiLuettava[LEN] = "";
00103     char aNimiKirjoitettava[LEN] = "";
00104     char aHaettavaNimi[LEN] = "";
00105     PUU *pJuuriSolmu = NULL;
00106     RBSOLMU *pRBJuuri = NULL;
00107     int iValinta = 0;
00108     int iArvo = 0;
00109
00110     do {
00111         iValinta = binaaripuuValikko();
00112         if (iValinta == 1) {
00113             // Luo AVL puun.
00114             kysyNimi("Tiedoston nimi: ", aNimiLuettava);
00115             pJuuriSolmu = luoPuu(aNimiLuettava, pJuuriSolmu);
00116
00117         } else if (iValinta == 2) {
00118             // Kirjoittaa AVL puun tiedostoon.
00119             if (pJuuriSolmu == NULL) {
00120                 printf("Luo binääripuu ennen sen kirjoittamista.\n");
00121             } else {
00122                 kysyNimi("Anna kirjoitettavan tiedoston nimi: ", aNimiKirjoitettava);
00123                 kirjoitaBinaaripuu(aNimiKirjoitettava, pJuuriSolmu);
00124             }
00125
00126         } else if (iValinta == 3) {
00127             // Syvyyshaku binääripuulle.
00128             if (pJuuriSolmu == NULL) {
00129                 printf("Luo binääripuu ennen syvyyshakua.\n");
00130             } else {
00131                 iArvo = kysyArvo("Anna haettava numeroarvo: ", iArvo);
00132                 kysyNimi("Anna kirjoitettavan tiedoston nimi: ", aNimiKirjoitettava);
00133                 tarkistaLoytyykoSyvyysshauilla(aNimiKirjoitettava, pJuuriSolmu, iArvo);
00134             }
00135
00136         } else if (iValinta == 4) {
00137             // Leveyshaku binääripuulle.
00138             if (pJuuriSolmu == NULL) {
00139                 printf("Luo binääripuu ennen leveyshakua.\n");
00140             } else {
00141                 kysyNimi("Anna haettava nimi: ", aHaettavaNimi);
00142                 kysyNimi("Anna kirjoitettavan tiedoston nimi: ", aNimiKirjoitettava);
00143                 leveyshaku(aNimiKirjoitettava, pJuuriSolmu, aHaettavaNimi);
00144             }
00145
00146         } else if (iValinta == 5) {
00147             // Vapauttaa muistin.
00148             pJuuriSolmu = vapautaMuistiPuu(pJuuriSolmu);
00149             printf("Muisti vapautettu.\n");
00150
00151         } else if (iValinta == 6) {
00152             // Poistaa solmun binääripuusta.
00153             if (pJuuriSolmu == NULL) {
00154                 printf("Luo binääripuu ennen poistoa.\n");
00155             } else {
00156                 kysyNimi("Anna poistettava nimi tai arvo: ", aHaettavaNimi);
00157                 pJuuriSolmu = poistaSolmu(aHaettavaNimi, pJuuriSolmu);

```



```

00158         }
00159
00160     } else if (iValinta == 7) {
00161         // Binäärihaku
00162         if (pJuuriSolmu == NULL) {
00163             printf("Luo binääripuu ennen binäärihakua.\n");
00164         } else {
00165             kysyNimi("Anna kirjoitettavan tiedoston nimi: ", aNimiKirjoitettava);
00166             iArvo = kysyArvo("Anna haettava numeroarvo: ", iArvo);
00167             iArvo = binaarihaku(aNimiKirjoitettava, pJuuriSolmu, iArvo);
00168             if (iArvo == 0) {
00169                 printf("Hakemaasi arvoa ei löytynyt binääripuusta.\n");
00170             }
00171         }
00172
00173     } else if (iValinta == 8) {
00174         // Toteuttaa punamustapuun.
00175         kysyNimi("Anna luettavan tiedoston nimi: ", aNimiLuettava);
00176         kysyNimi("Anna kirjoitettavan tiedoston nimi: ", aNimiKirjoitettava);
00177         pRBJuuri = luoRBPuu(pRBJuuri, aNimiLuettava);
00178         kirjoitaRB(aNimiKirjoitettava, pRBJuuri);
00179         printf("Tiedosto '%s' kirjoitettu.\n", aNimiKirjoitettava);
00180         pRBJuuri = vapautaMuistiRB(pRBJuuri);
00181
00182     } else if (iValinta == 0) {
00183         printf("Palataan takaisin päävalikkoon.\n");
00184
00185     } else {
00186         printf("Tuntematon valinta, yritä uudestaan.\n");
00187     }
00188 } while (iValinta != 0);
00189
00190 /* Vapautetaan puun varaama muisti varmuuden vuoksi vielä täällä lopussa erikseen, jos
00191  * käyttäjä ei muista sitä tehdä. */
00192
00193 pJuuriSolmu = vapautaMuistiPuu(pJuuriSolmu);
00194 return;
00195 }
00196
00203 int valikko(void) {
00204     int iValinta = 0;
00205     printf("Valitse haluamasi toiminto:\n");
00206     printf("1) Lista\n");
00207     printf("2) Binääripuu\n");
00208     printf("0) Lopeta\n");
00209     printf("Anna valintasi: ");
00210     scanf("%d", &iValinta);
00211     getchar();
00212     return iValinta;
00213 }
00214
00221 int listaValikko(void) {
00222     int iValinta = 0;
00223     printf("\nValitse haluamasi toiminto:\n");
00224     printf("1) Lue tiedosto\n");
00225     printf("2) Kirjoita tiedosto alusta loppuun\n");
00226     printf("3) Kirjoita tiedosto lopusta alkuun\n");
00227     printf("4) Tyhjennä taulukko\n");
00228     printf("5) Lajittele nousevasti\n");
00229     printf("6) Lajittele laskevasti\n");
00230     printf("7) Lisää tietoja listaan\n");
00231     printf("8) Poista listalta tietoja\n");
00232     printf("0) Takaisin\n");
00233     printf("Anna valintasi: ");
00234     scanf("%d", &iValinta);
00235     getchar();
00236     return iValinta;
00237 }
00238
00245 int binaaripuuValikko(void) {
00246     int iValinta = 0;
00247     printf("\nValitse haluamasi toiminto:\n");
00248     printf("1) Luo puu\n");
00249     printf("2) Kirjoita AVL puu tiedostoon\n");
00250     printf("3) Syvyyshaku\n");
00251     printf("4) Leveyshaku\n");
00252     printf("5) Tyhjennä puu\n");
00253     printf("6) Poista solmu\n");
00254     printf("7) Binäärihaku AVL puusta\n");
00255     printf("8) Punamustapuun\n");
00256     printf("0) Takaisin\n");
00257     printf("Anna valintasi: ");
00258     scanf("%d", &iValinta);
00259     getchar();
00260     return iValinta;
00261 }
00262

```

```
00270 char *kysyNimi(char *pPrompti, char *pNimi) {
00271     printf("%s", pPrompti);
00272     scanf("%s", pNimi);
00273     getchar();
00274     return (pNimi);
00275 }
00276
00284 int kysyArvo(char *pPrompti, int iArvo) {
00285     printf("%s", pPrompti);
00286     scanf("%d", &iArvo);
00287     getchar();
00288     return (iArvo);
00289 }
```

3.21 yleiset.h File Reference

Macros

- #define [LEN](#) 50

Functions

- void [mainLista](#) (void)
Tekee käyttäjän valinnan perusteella listan toiminnot.
- void [mainBinaaripuu](#) (void)
Tekee käyttäjän valinnan perusteella binääripuun toiminnot.
- int [valikko](#) (void)
Ensimmäinen valikko, jossa käyttäjä valitsee listan tai binääripuun.
- int [listaValikko](#) (void)
Tulostaa lista valikon ja kysyy käyttäjän valinnan.
- int [binaaripuuValikko](#) (void)
Tulostaa binaariluku valikon ja kysyy käyttäjältä valinnan.
- char * [kysyNimi](#) (char *pPrompti, char *pNimi)
Kysyy tiedoston/haettavan nimen.
- int [kysyArvo](#) (char *pPrompti, int iArvo)
Kysyy arvon, joka halutaan etsiä/poistaa puusta.

3.21.1 Macro Definition Documentation

3.21.1.1 LEN

```
#define LEN 50
```

Definition at line 4 of file [yleiset.h](#).

3.21.2 Function Documentation

3.21.2.1 binaaripuuValikko()

```
int binaaripuuValikko (
    void )
```

Parameters

<i>iValinta</i>	Käyttäjän antama syöte vaihtoehtojen pohjalta.
-----------------	--

Returns

int Palauttaa käyttäjän valinnan.

Definition at line 245 of file [yleiset.c](#).

```
00245     {
00246     int iValinta = 0;
00247     printf("\nValitse haluamasi toiminto:\n");
00248     printf("1) Luo puu\n");
00249     printf("2) Kirjoita AVL puu tiedostoon\n");
00250     printf("3) Syvyyshaku\n");
00251     printf("4) Leveyshaku\n");
00252     printf("5) Tyhjennä puu\n");
00253     printf("6) Poista solmu\n");
00254     printf("7) Binäärihaku AVL puusta\n");
00255     printf("8) Punamustapuu\n");
00256     printf("0) Takaisin\n");
00257     printf("Anna valintasi: ");
00258     scanf("%d", &iValinta);
00259     getchar();
00260     return iValinta;
00261 }
```

3.21.2.2 kysyArvo()

```
int kysyArvo (
    char * pPrompti,
    int iArvo)
```

Parameters

<i>pPrompti</i>	Kysymys esim. "Anna haettava numeroarvo: "
<i>iArvo</i>	Arvo, joka halutaan poistaa/etsiä.

Returns

int Palauttaa käyttäjän antaman arvon.

Definition at line 284 of file [yleiset.c](#).

```
00284     {
00285     printf("%s", pPrompti);
00286     scanf("%d", &iArvo);
00287     getchar();
00288     return (iArvo);
00289 }
```

3.21.2.3 kysyNimi()

```
char * kysyNimi (
    char * pPrompti,
    char * pNimi)
```

Parameters

<i>pPrompti</i>	Kysymys esim. "Anna kirjoitettavan tiedoston nimi: "
<i>pNimi</i>	Tiedoston nimi/haettava nimi, jota halutaan käyttää.

Returns

char Palauttaa käyttäjän antaman merkkijonon.

Definition at line 270 of file [yleiset.c](#).

```
00270                                     {
00271     printf("%s", pPrompti);
00272     scanf("%s", pNimi);
00273     getchar();
00274     return (pNimi);
00275 }
```

3.21.2.4 listaValikko()

```
int listaValikko (
    void )
```

Parameters

<i>iValinta</i>	Käyttäjän antama syöte vaihtoehtojen pohjalta.
-----------------	--

Returns

int Palauttaa käyttäjän valinnan.

Definition at line 221 of file [yleiset.c](#).

```
00221     {
00222     int iValinta = 0;
00223     printf("\nValitse haluamasi toiminto:\n");
00224     printf("1) Lue tiedosto\n");
00225     printf("2) Kirjoita tiedosto alusta loppuun\n");
00226     printf("3) Kirjoita tiedosto lopusta alkuun\n");
00227     printf("4) Tyhjennä taulukko\n");
00228     printf("5) Lajittele nousevasti\n");
00229     printf("6) Lajittele laskevasti\n");
00230     printf("7) Lisää tietoja listaan\n");
00231     printf("8) Poista listalta tietoja\n");
00232     printf("0) Takaisin\n");
00233     printf("Anna valintasi: ");
00234     scanf("%d", &iValinta);
00235     getchar();
00236     return iValinta;
00237 }
```

3.21.2.5 mainBinaaripuu()

```
void mainBinaaripuu (
    void )
```

Funktio näyttää valikon ja suorittaa käyttäjän valinnan mukaiset binääripuuhun liittyvät toiminnot.

Definition at line 101 of file [yleiset.c](#).

```
00101     {
00102     char aNimiLuettava[LEN] = "";
00103     char aNimiKirjoitettava[LEN] = "";
00104     char aHaettavaNimi[LEN] = "";
00105     PUU *pJuuriSolmu = NULL;
00106     RBSOLMU *pRBJuuri = NULL;
00107     int iValinta = 0;
00108     int iArvo = 0;
00109
00110     do {
00111         iValinta = binaaripuuValikko();
00112         if (iValinta == 1) {
00113             // Luo AVL puun.
00114             kysyNimi("Tiedoston nimi: ", aNimiLuettava);
00115             pJuuriSolmu = luoPuu(aNimiLuettava, pJuuriSolmu);
00116
00117         } else if (iValinta == 2) {
00118             // Kirjoittaa AVL puun tiedostoon.
00119             if (pJuuriSolmu == NULL) {
00120                 printf("Luo binääripuu ennen kirjoittamista.\n");
00121             } else {
00122                 kysyNimi("Anna kirjoitettavan tiedoston nimi: ", aNimiKirjoitettava);
00123                 kirjoitaBinaaripuu(aNimiKirjoitettava, pJuuriSolmu);
00124             }
00125
00126         } else if (iValinta == 3) {
00127             // Syvyyshaku binääripuulle.
00128             if (pJuuriSolmu == NULL) {
00129                 printf("Luo binääripuu ennen syvyyshakua.\n");
00130             } else {
00131                 iArvo = kysyArvo("Anna haettava numeroarvo: ", iArvo);
00132                 kysyNimi("Anna kirjoitettavan tiedoston nimi: ", aNimiKirjoitettava);
00133                 tarkistaLoytvykoSyvyyshaula(aNimiKirjoitettava, pJuuriSolmu, iArvo);
00134             }
00135
00136         } else if (iValinta == 4) {
00137             // Leveyshaku binääripuulle.
00138             if (pJuuriSolmu == NULL) {
00139                 printf("Luo binääripuu ennen leveyshakua.\n");
00140             } else {
00141                 kysyNimi("Anna haettava nimi: ", aHaettavaNimi);
00142                 kysyNimi("Anna kirjoitettavan tiedoston nimi: ", aNimiKirjoitettava);
00143                 leveyshaku(aNimiKirjoitettava, pJuuriSolmu, aHaettavaNimi);
00144             }
00145
00146         } else if (iValinta == 5) {
00147             // Vapauttaa muistin.
00148             pJuuriSolmu = vapautaMuistiPuu(pJuuriSolmu);
00149             printf("Muisti vapautettu.\n");
00150
00151         } else if (iValinta == 6) {
00152             // Poistaa solmun binääripuusta.
00153             if (pJuuriSolmu == NULL) {
00154                 printf("Luo binääripuu ennen poistoa.\n");
00155             } else {
00156                 kysyNimi("Anna poistettava nimi tai arvo: ", aHaettavaNimi);
00157                 pJuuriSolmu = poistaSolmu(aHaettavaNimi, pJuuriSolmu);
00158             }
00159
00160         } else if (iValinta == 7) {
00161             // Binäärihaku
00162             if (pJuuriSolmu == NULL) {
00163                 printf("Luo binääripuu ennen binäärihakua.\n");
00164             } else {
00165                 kysyNimi("Anna kirjoitettavan tiedoston nimi: ", aNimiKirjoitettava);
00166                 iArvo = kysyArvo("Anna haettava numeroarvo: ", iArvo);
00167                 iArvo = binaarihaku(aNimiKirjoitettava, pJuuriSolmu, iArvo);
00168                 if (iArvo == 0) {
00169                     printf("Hakemaasi arvoa ei löytynyt binääripuusta.\n");
00170                 }
00171             }
00172
00173         } else if (iValinta == 8) {
00174             // Toteuttaa punamustapuun.
```

```

00175         kysyNimi("Anna luettavan tiedoston nimi: ", aNimiLuettava);
00176         kysyNimi("Anna kirjoitettavan tiedoston nimi: ", aNimiKirjoitettava);
00177         pRBJuuri = luoRBPuu(pRBJuuri, aNimiLuettava);
00178         kirjoitaRB(aNimiKirjoitettava, pRBJuuri);
00179         printf("Tiedosto '%s' kirjoitettu.\n", aNimiKirjoitettava);
00180         pRBJuuri = vapautaMuistiRB(pRBJuuri);
00181
00182     } else if (iValinta == 0) {
00183         printf("Palataan takaisin päävalikkoon.\n");
00184
00185     } else {
00186         printf("Tuntematon valinta, yritä uudestaan.\n");
00187     }
00188 } while (iValinta != 0);
00189
00190 /* Vapautetaan puun varaama muisti varmuuden vuoksi vielä täällä lopussa erikseen, jos
00191  * käyttäjä ei muista sitä tehdä. */
00192
00193 pJuuriSolmu = vapautaMuistiPuu(pJuuriSolmu);
00194 return;
00195 }

```

3.21.2.6 mainLista()

```

void mainLista (
    void )

```

Funktio näyttää valikon ja suorittaa käyttäjän valinnan mukaiset listaan liittyvät toiminnot.

Definition at line 18 of file [yleiset.c](#).

```

00018     {
00019         char aNimiLuettava[LEN] = "";
00020         char aNimiKirjoitettava[LEN] = "";
00021         char aTietoPoistettava[LEN] = "";
00022         int iArvo = 0;
00023         int iIndeksi = 0;
00024         int iValinta = 0;
00025         TIEDOT *pAlku = NULL;
00026
00027     do {
00028         iValinta = listaValikko();
00029         if (iValinta == 1) {
00030             // Luekee tiedoston.
00031             kysyNimi("Anna luettavan tiedoston nimi: ", aNimiLuettava);
00032             pAlku = lue(aNimiLuettava, pAlku);
00033
00034         } else if (iValinta == 2) {
00035             // Kirjoittaa tiedoston alusta loppuun.
00036             if (pAlku == NULL) {
00037                 printf("Lue tiedosto ennen kirjoittamista.\n");
00038             } else {
00039                 kysyNimi("Anna kirjoitettavan tiedoston nimi: ", aNimiKirjoitettava);
00040                 kirjoitaTiedostoAlustaLoppuun(aNimiKirjoitettava, pAlku);
00041             }
00042
00043         } else if (iValinta == 3) {
00044             // Kirjoittaa tiedoston lopusta alkuun.
00045             if (pAlku == NULL) {
00046                 printf("Lue tiedosto ennen kirjoittamista.\n");
00047             } else {
00048                 kysyNimi("Anna kirjoitettavan tiedoston nimi: ", aNimiKirjoitettava);
00049                 kirjoitaTiedostoLopustaAlkuun(aNimiKirjoitettava, pAlku);
00050             }
00051
00052         } else if (iValinta == 4) {
00053             // Vapauttaa musitin,
00054             pAlku = vapautaMuisti(pAlku);
00055             printf("Muisti vapautettu.\n");
00056
00057         } else if (iValinta == 5) {
00058             // Lajittelee nousevasti lomistuslajittelulla.
00059             pAlku = lomitusLajittelu(pAlku);
00060             printf("Numerot lajiteltu nousevasti.\n");
00061
00062         } else if (iValinta == 6) {
00063             // Lajittelee kasjetavastu lisäyslajittelulla.
00064             pAlku = lisaysLajittelu(pAlku);
00065             printf("Numerot lajiteltu laskevasti.\n");
00066
00067         } else if (iValinta == 7) {

```

```

00068         // Lisää tietoja listaan.
00069         iIndeksi = kysyArvo("Mille riville haluat lisätä tietoja: ", iIndeksi);
00070         kysyNimi("Anna lisättävä nimi: ", aNimiKirjoitettava);
00071         iArvo = kysyArvo("Anna tämän nimisten lukumäärä: ", iArvo);
00072         pAlku = lisaaListaan(pAlku, iIndeksi, aNimiKirjoitettava, iArvo);
00073
00074     } else if (iValinta == 8) {
00075         // Poistaminen listalta.
00076         kysyNimi("Anna poistettava tieto: ", aTietoPoistettava);
00077         iArvo = onkoLukuVaiNimi(aTietoPoistettava);
00078         pAlku = poistaListastaAlkio(pAlku, iArvo, aTietoPoistettava);
00079
00080     } else if (iValinta == 0) {
00081         printf("Palataan takaisin päävalikkoon.\n");
00082
00083     } else {
00084         printf("Tuntematon valinta, yritä uudestaan.\n");
00085     }
00086 } while (iValinta != 0);
00087
00088 /* Vapautetaan listan varaama muisti varmuuden vuoksi vielä täällä lopussa erikseen, jos
00089  * käyttäjä ei muista sitä tehdä. */
00090
00091 pAlku = vapautaMuisti(pAlku);
00092 return;
00093 }

```

3.21.2.7 valikko()

```

int valikko (
    void )

```

Parameters

<i>iValinta</i>	Käyttäjän antama syöte.
-----------------	-------------------------

Returns

int Palauttaa käyttäjän valinnan.

Definition at line 203 of file [yleiset.c](#).

```

00203     {
00204         int iValinta = 0;
00205         printf("Valitse haluamasi toiminto:\n");
00206         printf("1) Lista\n");
00207         printf("2) Binääripuu\n");
00208         printf("0) Lopeta\n");
00209         printf("Anna valintasi: ");
00210         scanf("%d", &iValinta);
00211         getchar();
00212         return iValinta;
00213     }

```

3.22 yleiset.h

[Go to the documentation of this file.](#)

```

00001 #ifndef YLEISET_H
00002 #define YLEISET_H
00003
00004 #define LEN 50
00005
00006 void mainLista(void);
00007 void mainBinaaripuu(void);
00008 int valikko(void);
00009 int listaValikko(void);
00010 int binaaripuuValikko(void);
00011 char *kysyNimi(char *pPrompti, char *pNimi);
00012 int kysyArvo(char *pPrompti, int iArvo);
00013
00014 #endif

```


Index

aNimi

puu, [4](#)
RBSolmu, [5](#)
tiedot, [6](#)

binaarihaku

haut.c, [36](#)
haut.h, [43](#)

binaaripuu.c, [9](#), [20](#)

etsiPienin, [10](#)
kirjoitaBinaaripuu, [10](#)
kirjoitaTiedostoon, [10](#)
lisaaSolmu, [11](#)
luoPuu, [12](#)
nimenArvo, [13](#)
oikeaPuoli, [14](#)
onkoLuku, [14](#)
paivitaPuu, [15](#)
poistaSolmu, [15](#)
puunPituus, [16](#)
suurempiLukuVertailu, [17](#)
tasapainoitaPuu, [17](#)
vapautaMuistiPuu, [18](#)
varaaMuistiaPuulle, [18](#)
vasenPuoli, [19](#)

binaaripuu.h, [24](#), [35](#)

etsiPienin, [25](#)
kirjoitaBinaaripuu, [26](#)
kirjoitaTiedostoon, [26](#)
lisaaSolmu, [26](#)
luoPuu, [27](#)
nimenArvo, [28](#)
oikeaPuoli, [29](#)
onkoLuku, [30](#)
paivitaPuu, [30](#)
poistaSolmu, [31](#)
PUU, [25](#)
puunPituus, [32](#)
suurempiLukuVertailu, [32](#)
tasapainoitaPuu, [33](#)
vapautaMuistiPuu, [33](#)
varaaMuistiaPuulle, [34](#)
vasenPuoli, [34](#)

binaaripuuValikko

yleiset.c, [98](#)
yleiset.h, [107](#)

etsiPienin

binaaripuu.c, [10](#)
binaaripuu.h, [25](#)

halkaise

linkitettylista.c, [48](#)
linkitettylista.h, [68](#)

haut.c, [36](#), [40](#)

binaarihaku, [36](#)
leveyshaku, [37](#)
syvyyshaku, [38](#)
tarkistaLoytvykoSyvyysshaulla, [38](#)
vapautaMuistiJono, [39](#)
varaaMuistiaJonolle, [39](#)

haut.h, [42](#), [47](#)

binaarihaku, [43](#)
JONO, [42](#)
leveyshaku, [43](#)
syvyyshaku, [44](#)
tarkistaLoytvykoSyvyysshaulla, [45](#)
vapautaMuistiJono, [45](#)
varaaMuistiaJonolle, [46](#)

iArvo

puu, [4](#)
RBSolmu, [5](#)

iPituus

puu, [4](#)

iVariBitti

RBSolmu, [5](#)

iYhteensa

tiedot, [6](#)

JONO

haut.h, [42](#)

jono, [3](#)

pSeuraava, [3](#)
pSolmu, [3](#)

kierraOikealle

punamustapuu.c, [82](#)
punamustapuu.h, [91](#)

kierraVasemmalle

punamustapuu.c, [82](#)
punamustapuu.h, [92](#)

kirjoitaBinaaripuu

binaaripuu.c, [10](#)
binaaripuu.h, [26](#)

kirjoitaRB

punamustapuu.c, [83](#)
punamustapuu.h, [92](#)

kirjoitaRBTiedostoon

punamustapuu.c, [83](#)
punamustapuu.h, [93](#)

- kirjoitaTiedostoAlustaLoppuun
 - linkitettylista.c, [48](#)
 - linkitettylista.h, [68](#)
- kirjoitaTiedostoLopustaAlkuun
 - linkitettylista.c, [49](#)
 - linkitettylista.h, [69](#)
- kirjoitaTiedostoon
 - binaaripuu.c, [10](#)
 - binaaripuu.h, [26](#)
- korjaaLisays
 - punamustapuu.c, [84](#)
 - punamustapuu.h, [93](#)
- kysyArvo
 - yleiset.c, [98](#)
 - yleiset.h, [107](#)
- kysyNimi
 - yleiset.c, [99](#)
 - yleiset.h, [107](#)
- laskeListanPituus
 - linkitettylista.c, [50](#)
 - linkitettylista.h, [70](#)
- LEN
 - yleiset.h, [106](#)
- leveyshaku
 - haut.c, [37](#)
 - haut.h, [43](#)
- linkitettylista.c, [47](#), [60](#)
 - halkaise, [48](#)
 - kirjoitaTiedostoAlustaLoppuun, [48](#)
 - kirjoitaTiedostoLopustaAlkuun, [49](#)
 - laskeListanPituus, [50](#)
 - lisaaListaan, [50](#)
 - lisaysLajittelu, [51](#)
 - lomit, [52](#)
 - lomitLajittelu, [54](#)
 - lue, [54](#)
 - onkoLukuVaiNimi, [55](#)
 - poistaListastaAlkio, [56](#)
 - poistaListastaLkmPerusteella, [56](#)
 - poistaListastaNimenPerusteella, [57](#)
 - useammallaAlkiollaSamaLKM, [58](#)
 - vapautaMuisti, [59](#)
 - varaaMuistia, [59](#)
- linkitettylista.h, [66](#), [80](#)
 - halkaise, [68](#)
 - kirjoitaTiedostoAlustaLoppuun, [68](#)
 - kirjoitaTiedostoLopustaAlkuun, [69](#)
 - laskeListanPituus, [70](#)
 - lisaaListaan, [70](#)
 - lisaysLajittelu, [71](#)
 - lomit, [72](#)
 - lomitLajittelu, [73](#)
 - lue, [74](#)
 - onkoLukuVaiNimi, [75](#)
 - poistaListastaAlkio, [75](#)
 - poistaListastaLkmPerusteella, [76](#)
 - poistaListastaNimenPerusteella, [77](#)
 - TIEDOT, [67](#)
- useammallaAlkiollaSamaLKM, [78](#)
- vapautaMuisti, [78](#)
- varaaMuistia, [79](#)
- lisaaListaan
 - linkitettylista.c, [50](#)
 - linkitettylista.h, [70](#)
- lisaaRBSolmu
 - punamustapuu.c, [85](#)
 - punamustapuu.h, [94](#)
- lisaaSolmu
 - binaaripuu.c, [11](#)
 - binaaripuu.h, [26](#)
- lisaysLajittelu
 - linkitettylista.c, [51](#)
 - linkitettylista.h, [71](#)
- listaValikko
 - yleiset.c, [99](#)
 - yleiset.h, [108](#)
- lomit
 - linkitettylista.c, [52](#)
 - linkitettylista.h, [72](#)
- lomitLajittelu
 - linkitettylista.c, [54](#)
 - linkitettylista.h, [73](#)
- lue
 - linkitettylista.c, [54](#)
 - linkitettylista.h, [74](#)
- luoPuu
 - binaaripuu.c, [12](#)
 - binaaripuu.h, [27](#)
- luoRBPuu
 - punamustapuu.c, [86](#)
 - punamustapuu.h, [95](#)
- main
 - paaohjelma.c, [80](#)
- mainBinaaripuu
 - yleiset.c, [100](#)
 - yleiset.h, [108](#)
- mainLista
 - yleiset.c, [101](#)
 - yleiset.h, [110](#)
- nimenArvo
 - binaaripuu.c, [13](#)
 - binaaripuu.h, [28](#)
- oikeaPuoli
 - binaaripuu.c, [14](#)
 - binaaripuu.h, [29](#)
- onkoLuku
 - binaaripuu.c, [14](#)
 - binaaripuu.h, [30](#)
- onkoLukuVaiNimi
 - linkitettylista.c, [55](#)
 - linkitettylista.h, [75](#)
- paaohjelma.c, [80](#), [81](#)
 - main, [80](#)

- paivitaPuu
 - binaaripuu.c, [15](#)
 - binaaripuu.h, [30](#)
- pEdellinen
 - tiedot, [7](#)
- pOikea
 - puu, [4](#)
 - RBSolmu, [5](#)
- poistaListastaAlkio
 - linkitettylista.c, [56](#)
 - linkitettylista.h, [75](#)
- poistaListastaLkmPeruusteella
 - linkitettylista.c, [56](#)
 - linkitettylista.h, [76](#)
- poistaListastaNimenPerusteella
 - linkitettylista.c, [57](#)
 - linkitettylista.h, [77](#)
- poistaSolmu
 - binaaripuu.c, [15](#)
 - binaaripuu.h, [31](#)
- pSeuraava
 - jono, [3](#)
 - tiedot, [7](#)
- pSolmu
 - jono, [3](#)
- punamustapuu.c, [81](#), [88](#)
 - kierraOikealle, [82](#)
 - kierraVasemmalle, [82](#)
 - kirjoitaRB, [83](#)
 - kirjoitaRBTiedostoon, [83](#)
 - korjaaLisays, [84](#)
 - lisaaRBSolmu, [85](#)
 - luoRBPuu, [86](#)
 - vapautaMuistiRB, [86](#)
 - varaaMuistiaRB, [87](#)
- punamustapuu.h, [90](#), [97](#)
 - kierraOikealle, [91](#)
 - kierraVasemmalle, [92](#)
 - kirjoitaRB, [92](#)
 - kirjoitaRBTiedostoon, [93](#)
 - korjaaLisays, [93](#)
 - lisaaRBSolmu, [94](#)
 - luoRBPuu, [95](#)
 - RBSOLMU, [91](#)
 - vapautaMuistiRB, [96](#)
 - varaaMuistiaRB, [96](#)
- PUU
 - binaaripuu.h, [25](#)
- puu, [4](#)
 - aNimi, [4](#)
 - iArvo, [4](#)
 - iPituus, [4](#)
 - pOikea, [4](#)
 - pVasen, [4](#)
- puunPituus
 - binaaripuu.c, [16](#)
 - binaaripuu.h, [32](#)
- pVanhempi
 - RBSolmu, [6](#)
- pVasen
 - puu, [4](#)
 - RBSolmu, [6](#)
- RBSOLMU
 - punamustapuu.h, [91](#)
- RBSolmu, [5](#)
 - aNimi, [5](#)
 - iArvo, [5](#)
 - iVariBitti, [5](#)
 - pOikea, [5](#)
 - pVanhempi, [6](#)
 - pVasen, [6](#)
- src Directory Reference, [1](#)
- suurempiLukuVertailu
 - binaaripuu.c, [17](#)
 - binaaripuu.h, [32](#)
- syvyyshaku
 - haut.c, [38](#)
 - haut.h, [44](#)
- tarkistaLoytyykoSyvyyshauulla
 - haut.c, [38](#)
 - haut.h, [45](#)
- tasapainoitaPuu
 - binaaripuu.c, [17](#)
 - binaaripuu.h, [33](#)
- TIEDOT
 - linkitettylista.h, [67](#)
- tiedot, [6](#)
 - aNimi, [6](#)
 - iYhteensa, [6](#)
 - pEdellinen, [7](#)
 - pSeuraava, [7](#)
- useammallaAlkiollaSamaLKM
 - linkitettylista.c, [58](#)
 - linkitettylista.h, [78](#)
- valikko
 - yleiset.c, [102](#)
 - yleiset.h, [111](#)
- vapautaMuisti
 - linkitettylista.c, [59](#)
 - linkitettylista.h, [78](#)
- vapautaMuistiJono
 - haut.c, [39](#)
 - haut.h, [45](#)
- vapautaMuistiPuu
 - binaaripuu.c, [18](#)
 - binaaripuu.h, [33](#)
- vapautaMuistiRB
 - punamustapuu.c, [86](#)
 - punamustapuu.h, [96](#)
- varaaMuistia
 - linkitettylista.c, [59](#)
 - linkitettylista.h, [79](#)

varaaMuistiaJonolle
 haut.c, [39](#)
 haut.h, [46](#)
varaaMuistiaPuulle
 binaaripuu.c, [18](#)
 binaaripuu.h, [34](#)
varaaMuistiaRB
 punamustapuu.c, [87](#)
 punamustapuu.h, [96](#)
vasenPuoli
 binaaripuu.c, [19](#)
 binaaripuu.h, [34](#)

yleiset.c, [97](#), [103](#)
 binaaripuuValikko, [98](#)
 kysyArvo, [98](#)
 kysyNimi, [99](#)
 listaValikko, [99](#)
 mainBinaaripuu, [100](#)
 mainLista, [101](#)
 valikko, [102](#)
yleiset.h, [106](#), [111](#)
 binaaripuuValikko, [107](#)
 kysyArvo, [107](#)
 kysyNimi, [107](#)
 LEN, [106](#)
 listaValikko, [108](#)
 mainBinaaripuu, [108](#)
 mainLista, [110](#)
 valikko, [111](#)